



UNIVERSIDAD DE LAS CIENCIAS INFORMÁTICAS
FACULTAD DE TECNOLOGÍAS INTERACTIVAS

**SERVICIO BACKEND BASADO EN EL PARADIGMA DE LÍNEAS DE PRODUCTOS
DE SOFTWARE Y MODELOS DE CARACTERÍSTICAS PARA LA GESTIÓN
SISTEMÁTICA DE LA VARIABILIDAD CURRICULAR EN INSTITUCIONES DE
EDUCACIÓN SUPERIOR**

Trabajo de diploma para optar por el título de Ingeniero en Ciencias Informáticas

Autor: Jose Luis Echemendia López
Tutores: MSc. Yadira Ramírez Rodríguez
Ing. Liany Sobrino Miranda

La Habana, 2026

What is well set on your mind will be for sure well set on your lips
José Martí

Borrador

To my wife, my endless love

I am greatly indebted to professor John...

Declaración de auditoría

Declaramos ser autores de la presente tesis y reconocemos a la Universidad de las Ciencias Informáticas los derechos patrimoniales sobre esta, con carácter exclusivo.

Para que así conste firmamos la presente a los ____ días del mes de _____ del año _____.

Jose Luis Echemendia López
Autor

MSc. Yadira Ramírez Rodríguez
Tutor

Ing. Liany Sobrino Miranda
Tutor

En el contexto de la rápida evolución tecnológica y las transformaciones sociales del siglo XXI, la Educación Superior enfrenta el desafío de gestionar currículos flexibles que respondan a demandas cambiantes. Sin embargo, esta tarea se ve constantemente superada por el vertiginoso ritmo del cambio tecnológico, que amenaza con dejar obsoleto cualquier modelo educativo, por más innovador que sea en su concepción, antes de que pueda ser plenamente implementado y consolidado. Además, el mantenimiento de la variabilidad curricular se realiza actualmente mediante prácticas artesanales basadas en documentos estáticos y procesos manuales, un enfoque que resulta insostenible por su lentitud, propensión a errores e incapacidad para garantizar la trazabilidad y coherencia de los planes de estudio.

Esta investigación aborda este problema aplicando los principios de la Ingeniería de Líneas de Productos de Software (SPL) y los Modelos de Características (*Feature Models* - FM) al dominio de la planificación curricular. El trabajo se centra en el diseño, implementación y validación de un *backend* que permita a los gestores académicos modelar currículos dando lugar a la variabilidad curricular mediante FMs, con el objetivo de generar "Líneas de Productos Educativos". El sistema cuenta con una API RESTful que sirve como interfaz para las operaciones de creación, análisis, validación y configuración de FMs, permitiendo representar características y relaciones de obligatoriedad, optatividad y prerrequisitos como restricciones formales.

El resultado principal es un servicio backend especializado que no solo soporta el modelado y la persistencia de FMs adaptados al ámbito educativo, sino que también proporciona la base computacional para automatizar la generación y validación de planes de estudio diversos y consistentes a partir de un tronco común. Este desarrollo sienta las bases técnicas para una nueva generación de herramientas de gestión académica, demostrando la viabilidad de transferir técnicas avanzadas de ingeniería de software al dominio educativo.

Palabras clave: Líneas de Productos Educativos, Líneas de Productos de Software, Modelos Característicos, variabilidad curricular.

Lista de tareas pendientes	xv
Introducción	1
1 Fundamentación Teórica	5
1.1 Conceptos asociados al tema	5
1.1.1 Ingeniería de Líneas de Productos de Software (SPLE)	5
1.1.2 Modelos de Características (<i>Feature Models</i>)	7
1.1.3 Lenguaje Universal de Variabilidad (UVL)	14
1.1.4 Configuraciones y configurador curricular	17
1.1.5 Gestión y flexibilidad curricular. Línea de Productos Educativos	18
1.2 Fortalezas de los Modelos de Característica aplicado en la gestión curricular	19
1.2.1 Analogía formal curricular	20
1.2.2 Beneficios de las Líneas de Productos Educativos	26
1.3 Análisis del mercado	26
1.3.1 Panorama General del Mercado	27
1.3.2 Análisis de Soluciones Existentes	27
1.3.3 Oportunidades de Mercado Identificadas	29
1.4 Herramientas y tecnologías	30
1.4.1 Lenguaje de Programación	31
1.4.2 <i>Framework</i> de desarrollo	32
1.4.3 Base de Datos y Gestor de Base de Datos	34
1.4.4 Contenerización y Orquestación	35
1.4.5 Almacenamiento en Memoria, <i>Caching</i> y Mensajería	36
1.4.6 Almacenamiento de Archivos y Objetos	37
1.4.7 Procesamiento Asíncrono	38
1.4.8 Algoritmos, Motor de Reglas y Validación	40
1.4.9 Gestor de dependencias	42
1.4.10 Herramienta para el control de versiones	42
1.4.11 Lenguaje de modelado	43

1.4.12	Herramienta CASE	43
1.4.13	Herramientas para diseñar y ejecutar pruebas	44
1.4.14	Entorno Integrado de Desarrollo	44
1.5	Metodología de desarrollo del software	45
1.5.1	Metodologías de desarrollo de software tradicionales	45
1.5.2	Metodologías de desarrollo de software ágiles	46
1.5.3	Elección de la metodología	47
1.5.4	Programación Extrema	49
1.6	Conclusiones del capítulo	51
2	Análisis y diseño de la propuesta de solución	52
2.1	Descripción de la propuesta de solución	52
2.1.1	Sistemas de roles estáticos	53
2.1.2	Componentes funcionales de la lógica de negocio del sistema	56
2.2	Levantamiento de requisitos	59
2.2.1	Requisitos funcionales	59
2.2.2	Requisitos no funcionales	73
2.3	Historias de usuario	75
2.3.1	Estimación de esfuerzo por historia de usuarios	77
2.3.2	Desarrollo del plan de iteraciones	78
2.3.3	Desarrollo del plan de entregas	78
2.4	Tarjetas Clase-Responsabilidad-Colaboración (CRC)	79
2.5	Arquitectura de software	79
2.5.1	Representación arquitectónica de la solución	80
2.6	Patrones de diseños	84
2.6.1	Patrones GRASP	85
2.6.2	Patrones Banda de los Cuatro (GoF)	85
2.7	Conclusiones del capítulo	87
3	Resultados	88
	Conclusiones	90
	Recomendaciones	91
	Glossary	92
	Acrónimos	93
	Referencias bibliográficas	95

Apéndices	104
A Historias de usuario	105
B Tarjetas CRC	115
C Proyectos y Avaes	122
D Otro reporte	123
D.1 Otra sección de muestra	123
D.2 Otro ensamble Industrial	124

Índice de figuras

1.1	Símbolos gráficos para representar las relaciones estructurales.	10
1.2	Representación gráfica del Modelo de Características (FM) del modelo Sandwich especificado en UVL. Fuente: elaboración propia.	16
1.3	Representación simplificada #1 del FM de la asignatura Estructura de Datos I. Fuente: elaboración propia.	23
1.4	Representación simplificada #2 del FM de la asignatura Estructura de Datos I. Fuente: elaboración propia.	24
2.1	Componentes de la lógica del negocio del sistema. Fuente: elaboración propia.	56
2.2	Arquitectura en capas de la propuesta de solución. Fuente: elaboración propia.	81
2.3	Arquitectura de Celery. Fuente: elaboración propia.	83

Índice de cuadros

1.1	Notación semántica de las relaciones entre características de los FMs.	8
1.2	Analogía de conceptos de Línea de Productos de Software (SPL) en el Dominio Educativo. Fuente: Elaboración propia.	20
1.3	Comparación entre <i>frameworks</i> de Python	33
1.4	Tabla Comparativa: FastAPI vs FastAPI + Celery. Fuente: Elaboración propia.	39
1.5	Comparación, Metodología Ágil VS Metodología Tradicional.	47
1.6	Comparación de la Metodología Ágil y la Metodología Tradicional con respecto al proyecto.	48
1.7	Comparación de Metodología Ágiles. Fuente: Elaboración propia	48
2.1	Funciones y misión de los roles del sistema. Fuente: elaboración propia.	54
2.2	Jerarquía de roles estáticos del sistema. Fuente: elaboración propia.	54
2.3	Requisitos funcionales del sistema. Fuente: Elaboración propia.	59
2.4	Historia de usuario # 1	76
2.5	Estimación de esfuerzo por historia de usuario	77
2.6	Plan de duración de las iteraciones	78
2.7	Plan de entregas.	79
2.8	Tarjeta CRC # 1	79
A.1	Historia de usuario # 2	105
A.2	Historia de usuario # 3	106
A.3	Historia de usuario # 4	106
A.4	Historia de usuario # 5	107
A.5	Historia de usuario # 6	108
A.6	Historia de usuario # 7	108
A.7	Historia de usuario # 8	109
A.8	Historia de usuario # 9	110
A.9	Historia de usuario # 10	111
A.10	Historia de usuario # 11	111
A.11	Historia de usuario # 12	112
A.12	Historia de usuario # 13	112
A.13	Historia de usuario # 14	113

A.14 Historia de usuario # 15	114
B.1 Tarjeta CRC # 2	115
B.2 Tarjeta CRC # 3	115
B.3 Tarjeta CRC # 4	115
B.4 Tarjeta CRC # 5	116
B.5 Tarjeta CRC # 6	116
B.6 Tarjeta CRC # 7	116
B.7 Tarjeta CRC # 8	117
B.8 Tarjeta CRC # 9	117
B.9 Tarjeta CRC # 10	117
B.10 Tarjeta CRC # 11	118
B.11 Tarjeta CRC # 12	118
B.12 Tarjeta CRC # 13	118
B.13 Tarjeta CRC # 14	118
B.14 Tarjeta CRC # 15	119
B.15 Tarjeta CRC # 16	119
B.16 Tarjeta CRC # 17	119
B.17 Tarjeta CRC # 18	120
B.18 Tarjeta CRC # 19	120
B.19 Tarjeta CRC # 20	120
B.20 Tarjeta CRC # 21	121
B.21 Tarjeta CRC # 22	121

Lista de códigos fuentes

1.1	Ejemplo de modelo de variabilidad en Lenguaje de Variabilidad Universal (UVL). Fuente UVL Community, [s.f].	15
1.2	Ejemplo de modelo de variabilidad en UVL para la asignatura Estructura de Datos I.	21

Lista de tareas pendientes

El panorama de la Educación Superior en el siglo XXI está inmerso en una dinámica de cambio sin precedentes, impulsada por rápidos avances tecnológicos y profundas transformaciones sociales. La aceleración tecnológica y las fluctuantes demandas del mercado laboral exigen que las instituciones académicas migren de currículos rígidos hacia planes de estudio dinámicos y flexibles (UNESCO, 2022). Esta necesidad de flexibilidad introduce un desafío crítico que se sitúa en el núcleo de la gestión universitaria moderna: la gestión de la variabilidad curricular.

Para abordar este desafío, es necesario comprender sus dos dimensiones fundamentales. Por un lado, la innovación educativa, la cual se refiere a la introducción de nuevas ideas, enfoques, metodologías y tecnologías en el ámbito educativo; busca mejorar la calidad de la enseñanza y el aprendizaje, fomentando la creatividad, el pensamiento crítico y la resolución de problemas en los estudiantes (Gómez, 2023). Por otro lado, la gestión curricular, que implica el diseño, desarrollo e implementación de planes de estudio y programas educativos. Esta se ocupa de organizar y estructurar los contenidos, las habilidades y las competencias que se enseñan, así como la evaluación y mejora continua de los programas para asegurar su relevancia y efectividad (*ibíd.*). Ambas son aspectos fundamentales para el desarrollo de sistemas educativos actuales.

Para que la innovación educativa y la gestión curricular puedan responder efectivamente a las demandas de un entorno cambiante, la literatura especializada señala la necesidad de adoptar un enfoque de gestión por procesos y mejora continua en las instituciones de Educación Superior (Del Águila et al., 2014). Este enfoque, inspirado en modelos de gestión de calidad como la norma ISO 9001, implica superar la visión fragmentada de la administración académica para concebir el currículo como un sistema integrado de procesos interrelacionados que deben ser evaluados y optimizados permanentemente (Soto Grant, 2022). La gestión por procesos permite identificar con precisión las actividades clave —desde el diseño de planes de estudio hasta la evaluación del aprendizaje—, establecer responsables, y documentar procedimientos para garantizar la trazabilidad y la coherencia institucional (Veiga Méndez, 2024). Investigaciones aplicadas en universidades latinoamericanas demuestran que la implementación de este enfoque contribuye a reducir los “cuellos de botella” administrativos, optimizar el uso de recursos y alinear las operaciones cotidianas con los objetivos estratégicos de la institución (Gutama; Ortiz González; Moscoso Berna y Ayabaca Landi, 2025). Bajo esta óptica, la gestión curricular deja de ser una tarea puramente administrativa para convertirse en un ejercicio estratégico de diseño y rediseño continuo, fundamentado en ciclos de planificación, ejecución, verificación y actuación.

Este enfoque estratégico de gestión por procesos cobra especial relevancia al comprender que la innova-

ción educativa y la gestión curricular no son dimensiones independientes, sino que se encuentran estrechamente interrelacionadas. La introducción de enfoques innovadores en la enseñanza y el aprendizaje requiere una gestión curricular flexible y adaptable, que permita revisar y actualizar constantemente los planes de estudio para integrar nuevas metodologías, tecnologías y recursos educativos. Un ejemplo claro es la integración de las [Tecnologías de la Información y Comunicación \(TIC\)](#) en el aula, cuyo máximo beneficio se logra cuando la gestión curricular planifica su uso estratégico (Gómez, 2023). Esta interdependencia se ha vuelto crítica con la proliferación de nuevas modalidades educativas (presenciales, híbridas, en línea, [Curso en Línea Masivo y Abierto \(MOOC\)](#), *micro-learning*) y enfoques pedagógicos (aprendizaje basado en proyectos, *flipped classroom*, gamificación), lo que ha creado un panorama de extrema variabilidad instruccional (Bates, 2019).

A pesar de esta creciente complejidad, la definición y el mantenimiento de la variabilidad curricular en las instituciones de Educación Superior se gestionan actualmente mediante prácticas obsoletas. La dependencia de documentos estáticos, hojas de cálculo y procesos manuales constituye un enfoque fragmentado que resulta costoso, lento y altamente propenso a errores. Esta gestión artesanal dificulta la actualización oportuna de la información, genera inconsistencias entre versiones y complica la trazabilidad de los cambios. En la práctica, cada nuevo curso o recurso educativo se desarrolla prácticamente desde cero, desaprovechando la oportunidad de reutilizar componentes comunes y construir conocimiento de manera incremental sobre bases previamente validadas.

Las consecuencias de este paradigma obsoleto son profundas y de gran alcance. La falta de un enfoque sistemático provoca la divulgación de información desactualizada, compromete la coherencia institucional y puede llevar al diseño de rutas formativas incoherentes que afectan la progresión del estudiante. Esto incrementa la carga de trabajo administrativo y docente, limita la visibilidad global del currículo y tiene un impacto directo en la satisfacción del estudiante y en los procesos de acreditación. Irónicamente, la misma tecnología que ha posibilitado la explosión de modalidades y enfoques pedagógicos carece, en la actualidad, de los mecanismos sistemáticos para gestionar la complejidad que ella misma ha generado. Existe una batalla contrarreloj, donde la velocidad de la innovación tecnológica supera la capacidad de reacción de los sistemas educativos tradicionales, volviendo potencialmente obsoleta cualquier renovación curricular incluso antes de su implementación (Sánchez; Gutiérrez y Cabrera, 2020). Esta situación deja de manifiesto que el paradigma artesanal de diseño curricular ha llegado a su límite.

De este análisis, se desprende la siguiente interrogante que define el **problema de investigación**: ¿Cómo gestionar sistemáticamente la variabilidad curricular en instituciones de Educación Superior, de manera que se superen las limitaciones de los enfoques basados en documentación manual y se garantice la coherencia, trazabilidad y sostenibilidad de los planes de estudio? Tomando como **objeto de estudio**: la gestión de la variabilidad curricular en instituciones de Educación Superior. Enmarcado en el **campo de acción**: La aplicación del paradigma de Líneas de Productos de Software para la modelación y gestión de la variabilidad curricular. Con el fin de solucionar la problemática planteada, se define como **objetivo general**: desarrollar un servicio *backend* basado en el paradigma de [SPL](#) y [FM](#) para la gestión sistemática de la variabilidad

curricular en instituciones de Educación Superior.

Para dar cumplimiento al objetivo planteado se proponen las siguientes **tareas investigativas**:

1. **Estudio** del estado del arte en la [Ingeniería de Líneas de Productos de Software \(SPLE\)](#), [FM](#) y su aplicación al dominio educativo.
2. **Análisis** de diferentes soluciones homólogas para la identificación de características generales y tendencias en este tipo de aplicaciones.
3. **Selección** de las herramientas, tecnologías y metodología a emplear, para garantizar una excelente y eficiente experiencia de desarrollo y alineado con los objetivos del proyecto.
4. **Identificación** de los requisitos funcionales y no funcionales necesarios para un sistema capaz de gestionar la variabilidad de Líneas de Productos Educativos.
5. **Diseño** de la arquitectura del servicio *backend*, definiendo los componentes principales y el modelo de datos para representar [FM](#) en el contexto educativo.
6. **Implementación** del servicio *backend* para materializar el sistema que dará respuesta a la problemática planteada, asegurando la creación, análisis, validación y configuración de [FM](#).
7. **Validación** del servicio desarrollado mediante casos de uso representativos en el ámbito educativo, evaluando que cumpla con el funcionamiento esperado.

Para darle solución a los objetivos trazados en las tareas de investigación se emplearon los siguientes **métodos científicos**:

- **Métodos teóricos:**

- **Modelación:** se utilizó para la elaboración de los diagramas del lenguaje de modelado y en la representación de las características de la solución.
- **Histórico-Lógico:** Se empleó para comprender la evolución de las técnicas de gestión de variabilidad en [SPL](#), desde sus orígenes en ingeniería de software hasta su aplicación potencial en dominios educativos, analizando su desarrollo histórico y lógica interna.
- **Analítico-sintético:** Utilizado en el examen crítico de la bibliografía especializada sobre [SPL](#), [FM](#) y sistemas educativos, permitiendo sintetizar un marco conceptual adaptado al dominio educativo.

- **Métodos empíricos:**

- **Observación:** Mediante este método se analizaron herramientas existentes de gestión [SPL](#) (como *pure::variants*, *FeatureIDE*) para identificar limitaciones técnicas y requisitos necesarios para el contexto educativo específico.
- **Entrevista:** Se entrevista especialistas responsables de diseños curriculares que compran aspectos técnicos para identificar los desafíos existentes en la flexibilidad y variabilidad de los currículos.
- **Experimentación:** Mediante la implementación de prototipos funcionales para validar diferentes enfoques arquitectónicos y algoritmos de validación.

Estructura de la investigación:

Capítulo 1: Fundamentación teórica, capítulo encargado de definir los elementos teóricos necesarios para el desarrollo de la investigación y los principales conceptos que se emplearán durante todo el trabajo. Se realiza un análisis de las soluciones similares y se selecciona la metodología de desarrollo de software y las herramientas y tecnologías a utilizar.

Capítulo 2: Características y diseño del sistema, capítulo poseedor de los diferentes requisitos funcionales y no funcionales de la aplicación a desarrollar, así como de la modelación de la solución mediante los artefactos planteados por la metodología de desarrollo a utilizar.

Capítulo 3: Implementación y pruebas, capítulo que contiene los estándares de codificación utilizados en la implementación, así como las diferentes pruebas a las que será sometida la aplicación una vez terminada para verificar su correcto funcionamiento.

Fundamentación Teórica

La base teórica funciona como el almacén conceptual sobre el cual se erige todo trabajo investigativo. Este capítulo expone los conceptos asociados al tema, los fundamentos teóricos existentes y el estado del arte que respaldan el estudio, proporcionando un sistema de referencia que orienta la comprensión e interpretación del fenómeno abordado. En tal sentido, se establece el marco conceptual necesario para contextualizar y dar sentido a la propuesta de solución que se desarrolla en los capítulos subsiguientes.

Además, se efectúa una comparación entre soluciones homólogas existentes para determinar características predominantes y conocer las tendencias en este tipo de sistemas. También se realiza un análisis de las diferentes herramientas y tecnologías para dar a conocer el conjunto de utensilios que serán protagonistas en el desarrollo de la propuesta de solución, así como la metodología de desarrollo de software que guiará todo el proceso práctico.

1.1. Conceptos asociados al tema

En este epígrafe se desglosan los conceptos esenciales que convergen en el núcleo del estudio. La clarificación de estos términos no solo delimita el campo de acción, sino que también proporciona el lenguaje común y la base teórica necesaria para el diseño de una solución coherente y efectiva. Esta precisión conceptual resulta fundamental, pues sienta las bases sobre las cuales se articularán posteriormente las decisiones de diseño e implementación de la propuesta.

1.1.1. Ingeniería de Líneas de Productos de Software (SPLE)

La **SPLE** es una estrategia de ingeniería de software que busca producir familias de sistemas de software similares a partir de un conjunto común de activos centrales mediante un proceso gestionado de desarrollo y gestión de la variabilidad. El objetivo principal es lograr la reutilización a gran escala, lo que se traduce en una reducción de costos, menor tiempo de comercialización y mayor calidad de los productos (SG Buzz, [s.f.]). El concepto de **SPL** surgió a finales de los años 90 como respuesta directa a la crisis de productividad

que enfrentaba la industria del software a pesar de las técnicas de reutilización orientadas a objetos. La [SPLE](#) formaliza una práctica que ya existía intuitivamente en muchas empresas: construir un software base (plataforma) para generar rápidamente múltiples productos similares con variaciones controladas (AcademiaLab, [\[s.f.\]](#))

Para comprender cabalmente el paradigma de [SPLE](#), es necesario examinar los conceptos fundamentales que constituyen su base teórica y que permiten articular la gestión de la variabilidad como eje central de la estrategia. Estos conceptos son:

1. **Línea de Productos:** Es la familia completa de sistemas de software que pueden ser generados. Representa el espacio de posibilidades definido por el conjunto de activos.
2. **Activos Centrales (*Core Assets*):** Son los componentes fundamentales (código, arquitectura, modelos de diseño, documentación, planes de prueba) diseñados específicamente para ser compartidos y reutilizados en toda la línea. La inversión inicial en estos activos es clave para el éxito de la estrategia.
3. **Variabilidad:** Es la propiedad esencial de una [SPL](#). Define la capacidad de la línea para ser adaptada, personalizada o configurada para generar productos específicos que satisfagan las necesidades particulares de un cliente o escenario. La variabilidad es, en esencia, el conjunto de diferencias controladas entre los productos posibles.
4. **Puntos de Variabilidad (*Variability Points*):** Son las ubicaciones explícitas dentro de los Activos Centrales donde se deben tomar decisiones para configurar un producto. Estos puntos actúan como interruptores.º .opciones"que guían el proceso de personalización.

La motivación detrás de la [SPL](#) es fundamentalmente económica y técnica, pues permite el ahorro de costos, ya que al compartir la gran mayoría de los activos (entre el 70 % y 90 % del sistema), se evita la reimplementación del mismo código y diseño para cada nuevo producto, además, provee de una reducción del tiempo de comercialización (*Time-to-Market*) pues la creación de un nuevo producto se reduce de un ciclo completo de desarrollo a un simple proceso de configuración y ensamblaje de activos preexistentes y permite una mejora de la calidad, pues como los activos centrales, al ser reutilizados y probados en múltiples productos, alcanzan un nivel de madurez y robustez superior al que podría lograrse en un desarrollo de producto único (Ingeniería de Software, [\[s.f.\]](#)).

La [SPL](#) opera a través de dos procesos interdependientes que deben gestionarse de manera continua, siendo el primero de estos la Ingeniería del Dominio (*Domain Engineering*), proceso hacia arriba que se enfoca en la creación y el mantenimiento de los Activos Centrales, cuya tarea principal es el modelado de la variabilidad para comprender y documentar todas las posibles configuraciones que la línea puede soportar y que constituye la base teórica y formal de la línea, y un segundo, la Ingeniería de la Aplicación (*Application Engineering*), el cual es el proceso hacia abajo que utiliza los Activos Centrales para configurar y generar productos específicos, que implica tomar las decisiones de variabilidad definidas en el modelo (seleccionar las opciones) para producir una instancia concreta de la línea de productos. La herramienta principal y estándar de facto para formalizar y gestionar la variabilidad durante la Ingeniería del Dominio es el [FM](#).

Este modelo establece el puente formal entre las opciones de la línea de productos y la capacidad de generar una instancia válida, lo cual es el principio que se busca trasladar al diseño curricular.

La búsqueda de la variabilidad curricular ha llevado a la exploración de paradigmas de ingeniería de software. El concepto de **SPL** ofrece un enfoque estructurado y probado para gestionar familias de productos a partir de un conjunto común de activos (Apel; Batory; Kästner y Saake, 2013). Cuando este paradigma se aplica al dominio educativo, surge el concepto de "Líneas de Productos Educativos", donde un tronco curricular común puede derivar en múltiples planes de estudio especializados mediante la gestión explícita de su variabilidad.

Evolución y Mantenimiento de Líneas de Productos

Las **SPL** no son entidades estáticas; evolucionan con el tiempo para adaptarse a nuevos requisitos, tecnologías y mercados. La evolución de **SPL** se refiere al proceso de modificar los activos centrales y los modelos de variabilidad para incorporar cambios, corregir defectos o mejorar la línea (*ibíd.*).

Los principales desafíos en la evolución de **SPL** incluyen:

- Trazabilidad de cambios: Mantener registro de qué cambios se realizaron, por qué y cuándo, así como su impacto en los productos derivados.
- Impacto en productos existentes: Determinar cómo las modificaciones en la línea afectan a los productos ya desplegados y si es necesario migrarlos.
- Consistencia del modelo: Garantizar que después de los cambios, el **FM** siga siendo consistente y represente fielmente la variabilidad deseada.
- Gestión de versiones: Coordinar diferentes versiones de la línea que puedan coexistir para dar soporte a productos en distintos estados de madurez.

En el dominio educativo, la evolución curricular es una realidad constante: nuevos planes de estudio, actualización de contenidos, cambios en normativas, etc. Un sistema de gestión de variabilidad curricular debe contemplar mecanismos para:

- Registrar el histórico de modificaciones curriculares.
- Evaluar el impacto de los cambios en los planes de estudio existentes.
- Gestionar múltiples versiones de un mismo plan (por ejemplo, planes de estudio con diferentes años de implementación).
- Asegurar la trazabilidad entre las versiones del modelo y los productos generados.

1.1.2. Modelos de Características (*Feature Models*)

Es necesario contar con un modelo que represente de manera explícita las variaciones y opciones disponibles en la línea de productos. Este modelo debe ser capaz de capturar la complejidad de las relaciones entre las características y sus variaciones, así como las restricciones que pueden existir entre ellas (Pohl; Böckle

y Linden, 2005). La representación gráfica de estas relaciones es fundamental para facilitar la comprensión y el análisis de la variabilidad en la línea de productos.

El Modelo de Características (*Feature Model* - FM) es la técnica de modelado central en SPLE. Su formalismo es indispensable para pasar de la noción abstracta de variabilidad a una representación estructurada y analizable (PTC, 2025). Los FMs fueron formalmente introducidos por Kang; Cohen; Hess; Novak y Peterson (1990) como parte de la metodología *Análisis de Dominio Orientado a Características* (FODA), un enfoque pionero para la ingeniería de dominios. Desde su concepción, un FM ha sido reconocido como el artefacto primario para la captura de la variabilidad funcional de un sistema. En esencia, un FM es un modelo de dominio que no solo documenta las posibles funcionalidades, sino que también define las reglas de composición entre ellas (GeeksforGeeks, [s.f.]).

Formalmente, un FM se define como un árbol de decisiones jerárquico donde cada nodo es una característica (*feature*). Una característica se entiende como una unidad de funcionalidad, un requisito, o un aspecto distintivo de un sistema, tal como es percibido por el usuario final o el analista del dominio.

La estructura del FM es un árbol de composición, donde:

- La característica raíz representa el sistema o la línea de producto completa (ej., la carrera de Ingeniería Informática).
- Las características descendientes detallan la composición de sus características padre hasta alcanzar las características hoja, que son las unidades más pequeñas y configurables.

Relaciones estructurales

La potencia del FM reside en su notación gráfica concisa, la cual utiliza la posición y símbolos específicos para codificar las restricciones de configuración. Las relaciones entre una característica padre (letra P) y sus hijos (letra C) definen las decisiones de inclusión o exclusión; véase la tabla 1.1 (D. Benavides; S. Segura y A. Ruiz-Cortés, 2010; David Benavides; Sergio Segura y Antonio Ruiz-Cortés, 2010; Kang; Cohen; Hess; Novak y Peterson, 1990; Pohl; Böckle y Linden, 2005; Wang; L. Zhao y J. Hu, 2007).

Cuadro 1.1. Notación semántica de las relaciones entre características de los FMs.

Relación	Semántica Formal	Implicación en la Configuración
Obligatoria	$P \rightarrow C$	Si el padre P es seleccionado, el hijo obligatorio C debe incluirse. No hay variabilidad en este punto
Opcional	$C \rightarrow P$	El hijo C puede ser incluido o excluido, si se selecciona el padre P. Introduce variabilidad

Continúa en la siguiente página

Cuadro 1.1 – Continuación de la página anterior..

Relación	Semántica Formal	Implicación en la Configuración
Agrupación AND	$(P \rightarrow \bigwedge_{i=1}^n C_i) \wedge \bigwedge_{i=1}^n (C_i \rightarrow P)$	Si se selecciona la característica padre P, todos los hijos C_i deben incluirse; cada hijo seleccionado implica la presencia del padre. No introduce variabilidad dentro del grupo
Agrupación OR	$(P \rightarrow \bigvee_{i=1}^n C_i) \wedge \bigwedge_{i=1}^n (C_i \rightarrow P)$	Al seleccionar el padre P, debe elegirse al menos un hijo C_i . Cada hijo seleccionado requiere que el padre esté presente. Sí introduce variabilidad
Agrupación alternativa	$(P \rightarrow \bigvee_{i=1}^n C_i) \wedge \bigwedge_{i \neq j} \neg(C_i \wedge C_j) \wedge \bigwedge_{i=1}^n (C_i \rightarrow P)$	Cuando el padre P se selecciona, exactamente un hijo C_i puede activarse y la elección de cualquier hijo obliga a seleccionar al padre. Sí introduce variabilidad

Las relaciones descritas en la tabla 1.1 constituyen la base semántica sobre la cual se construye cualquier FM. Sin embargo, para que esta semántica sea útil en la práctica del modelado, es necesario contar con una notación gráfica estandarizada que permita visualizar de forma intuitiva la estructura jerárquica, las restricciones entre características e identificar los puntos de variabilidad. Esta representación visual proporciona una vista de alto nivel que complementa la especificación formal, pues es la encargada de dotar a los FMs de su poder comunicativo y analítico. La figura 1.1 ilustra la simbología convencionalmente aceptada en la literatura para representar las relaciones estructurales de los FMs, donde cada símbolo codifica visualmente una de las relaciones semánticas previamente explicadas.

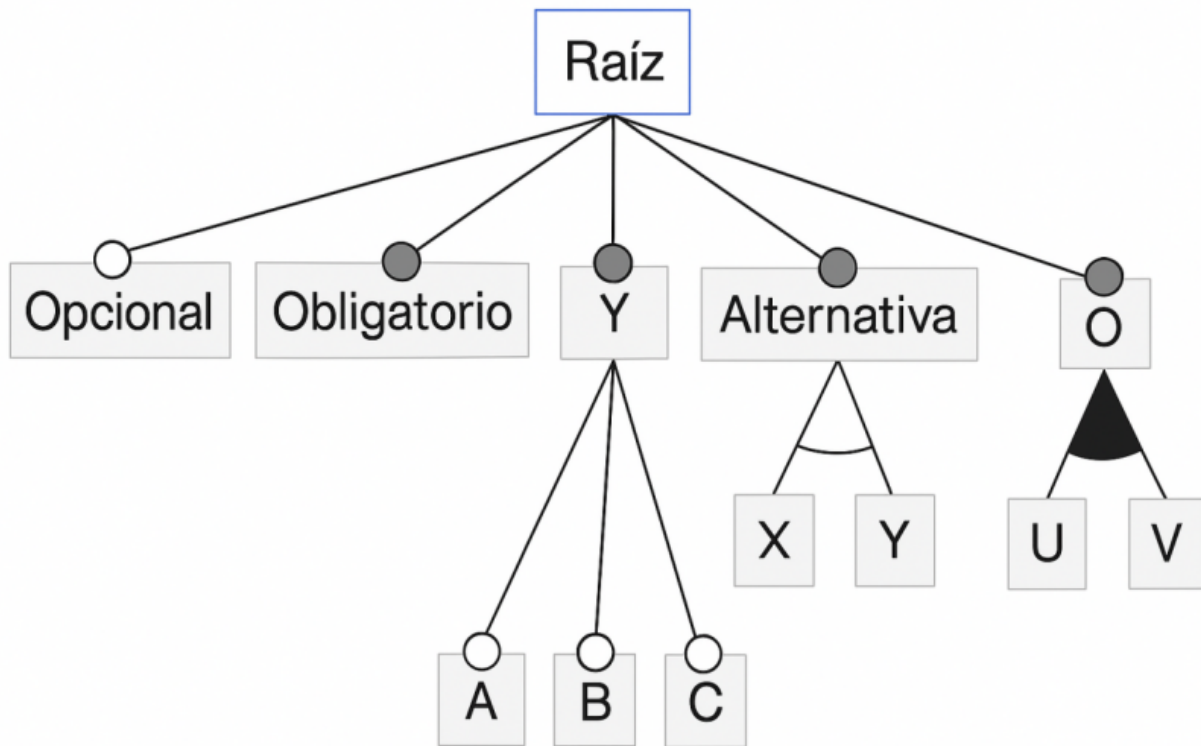


Figura 1.1. Símbolos gráficos para representar las relaciones estructurales.

Restricciones (*Cross-Tree*)

Las relaciones jerárquicas son insuficientes para capturar todas las reglas de negocio complejas. Por ello, los **FM**s permiten definir las restricciones transversales (*cross-tree*); que son reglas lógicas que definen dependencias entre características que no están directamente conectadas en la jerarquía padre-hijo del árbol de características. Su función es garantizar la validez de una configuración al capturar las relaciones complejas del mundo real que la estructura jerárquica por sí sola no puede expresar (David Benavides; Trinidad y Antonio Ruiz-Cortés, 2005; C. Sundermann; Dörr; Junker y Kehrer, 2023).

Las dos formas más comunes y fundamentales son:

- **Requiere:** Esta restricción establece una dependencia de presencia. Si una característica A requiere una característica B, entonces siempre que A sea seleccionada en una configuración, B también debe estarlo. Por ejemplo, en un **FM** para un automóvil, la característica "Navegación" puede requerir la característica "Pantalla Táctil". Esta relación se modela en lógica proposicional como el condicional $A \rightarrow B$ (David Benavides; Trinidad y Antonio Ruiz-Cortés, 2005; C. Sundermann; Dörr; Junker y Kehrer, 2023).
- **Excluye:** Esta restricción establece un conflicto o incompatibilidad. Si una característica C excluye

una característica D, entonces ambas no pueden formar parte de la misma configuración. Por ejemplo, el motor "Diésel" puede excluir la transmisión eléctrica.^{en} un vehículo híbrido. Su representación lógica es la negación de la conjunción: $\neg(C \wedge D)$ (David Benavides; Trinidad y Antonio Ruiz-Cortés, 2005; C. Sundermann; Dörr; Junker y Kehr, 2023).

Semántica Formal y Solucionadores SAT (*SAT Solvers*)

La semántica formal de un FM proporciona un significado matemático preciso a todos sus elementos (jerarquía, relaciones y restricciones), lo que permite un análisis automático y libre de ambigüedades.

- Traducción a Lógica Proposicional: La base de la semántica formal es la traducción de todo el FM a una fórmula en lógica proposicional. Cada característica se representa como una variable proposicional (que puede ser Verdadera o Falsa, indicando si está seleccionada o no). Las relaciones padre-hijo y las restricciones *cross-tree* se convierten en una serie de fórmulas lógicas que, en conjunto, definen el conjunto de todas las configuraciones válidas (C. Sundermann; Dörr; Junker y Kehr, 2023; T. Thüm; Kästner; Benduhn; Meinicke; Saake y Leich, 2014).
- Satisfacibilidad (*Satisfacibilidad Booleana (SAT)*) y *Conteo de Modelos (SHARPSAT)*: Una vez el FM está representado como una fórmula lógica F, se pueden realizar diversos análisis:
 - Análisis de SAT: Consiste en verificar si existe al menos una asignación de valores (Verdadero/Falso) a las variables que haga que la fórmula F sea verdadera. Esto responde a la pregunta fundamental de si el FM tiene al menos una configuración válida (D. Benavides; S. Segura y A. Ruiz-Cortés, 2010; Mendonca y Wasowski, 2009; C. Sundermann; Dörr; Junker y Kehr, 2023).
 - Análisis de SHARPSAT: Consiste en el conteo de modelos (*Model Counting*) que satisfacen F. Esto es crucial para determinar el número total de configuraciones válidas que define un FM, una métrica esencial para comprender su variabilidad y complejidad (D. Benavides; S. Segura y A. Ruiz-Cortés, 2010; Mendonca y Wasowski, 2009; C. Sundermann; Dörr; Junker y Kehr, 2023).
- Solucionadores SAT (*SAT solvers*): Un solucionador SAT (*SAT solver*) es una herramienta algorítmica altamente optimizada diseñada para resolver problemas de satisfacibilidad. Dada una fórmula F, el solucionador (*solver*) determina si es satisfacible (SAT) o insatisfacible (UNSAT). Para el análisis de SHARPSAT, se emplean solucionadores especializados de conteo de modelos (*#SAT solvers*) como sharpSAT y Ganak, que han demostrado un rendimiento excelente al calcular el número de configuraciones en modelos industriales (Soos; Gocht y al., 2020; C. Sundermann; Dörr; Junker y Kehr, 2023).

Operaciones sobre Modelos de Características

Para que un FM sea útil en un entorno de desarrollo, debe soportar un conjunto de operaciones que permitan manipularlo y consultarlo. Estas operaciones están agrupadas por grupos y son:

- **Operaciones de creación y edición:**

- Creación del modelo: Inicialización de un FM con su característica raíz.
- Inserción/eliminación de características: Añadir o remover nodos en la jerarquía.
- Modificación de relaciones: Cambiar el tipo de relación entre características padre-hijo.
- Gestión de restricciones: Añadir, eliminar o modificar restricciones *cross-tree*.

- **Operaciones de consulta y análisis:**

- Validación del modelo: Verificar la consistencia global del FM.
- Cálculo del número de configuraciones: Determinar la cardinalidad del espacio de productos.
- Extracción de configuraciones: Obtener configuraciones válidas aleatorias o siguiendo criterios específicos.
- Verificación de restricciones: Comprobar si una configuración candidata cumple todas las reglas.

- **Operaciones de transformación:**

- Exportación/importación: Serializar el modelo a formatos estándar (UVL, XML, JSON).
- Refactorización: Reorganizar la estructura del modelo sin alterar su semántica.
- Especialización: Generar un submás modelo a partir de decisiones parciales.

Modelos de Características con atributos

Los FMs básicos, también conocidos como FMs booleanos, capturan únicamente la presencia o ausencia de características. Sin embargo, muchos dominios requieren modelar información adicional asociada a cada característica, como costos, pesos, tiempos, o en el caso educativo, créditos, horas lectivas o puntuaciones (David Benavides; Trinidad y Antonio Ruiz-Cortés, 2005).

Los FMs con atributos extienden el formalismo básico permitiendo asociar a cada característica un conjunto de atributos con valores. Estos atributos pueden ser:

- Numéricos: Enteros o reales (ej., 4 créditos, 2.5 horas).
- Booleanos: Valores verdadero/falso.
- Enumerados: Conjuntos finitos de valores (ej., semestre: 1,2,3,4,5).
- Cadenas: Texto descriptivo.

La verdadera utilidad de los atributos emerge cuando se combinan con restricciones que los involucran. Por ejemplo, en un modelo curricular se pueden expresar reglas como:

- La suma de créditos de las asignaturas optativas no debe exceder 30.
- El costo de los materiales de laboratorio debe ser inferior a \$200.
- La carga horaria semanal debe estar entre 20 y 25 horas.

El análisis de FM con atributos requiere técnicas más sofisticadas, como la programación con restricciones (*constraint programming*) o la verificación de modelos (*model checking*), especialmente cuando se

buscan configuraciones que optimicen funciones objetivo (D. Benavides; S. Segura y A. Ruiz-Cortés, 2010). Lenguajes como UVL incorporan soporte nativo para atributos y restricciones sobre ellos, como se ilustra en el ejemplo del Sándwich 1.1.

Análisis Automatizado de Modelos de Características (AAFM)

La verdadera potencia de los FMs reside en la posibilidad de realizar análisis automatizados sobre ellos. El análisis de FMs (Análisis Automatizado de Feature Models (AAFM)) es un área de investigación que estudia cómo extraer información útil de un FM mediante algoritmos y herramientas automáticas (ibíd.). Estos análisis permiten responder preguntas fundamentales sobre la línea de productos antes de invertir en su implementación.

D. Benavides; S. Segura y A. Ruiz-Cortés (ibíd.) clasifican los análisis en varias categorías:

- **Análisis de propiedades básicas:** Verifican características elementales del modelo:
 - Consistencia (*consistency*): Determina si existe al menos una configuración válida (modelo no vacío).
 - Modelo muerto (*void model*): Detecta si el FM carece por completo de configuraciones válidas debido a restricciones contradictorias.
 - Características muertas (*dead features*): Identifica características que no pueden ser seleccionadas en ninguna configuración válida.
 - Características falsas opcionales (*false optional features*): Detecta características declaradas como opcionales pero que, debido a restricciones, resultan obligatorias en todas las configuraciones.
- **Análisis de configuración:** Asisten en el proceso de seleccionar características:
 - Validación de configuraciones: Verifica si una selección parcial o completa de características constituye una configuración válida.
 - Propagación de decisiones: Calcula automáticamente las consecuencias de seleccionar o excluir una característica.
 - Explicación de inconsistencias: Identifica las restricciones responsables de que una configuración sea inválida.
- **Análisis de optimización:** Buscan configuraciones que maximicen o minimicen ciertos criterios:
 - Selección óptima de características: Encuentra configuraciones que optimicen funciones objetivo (costo, rendimiento, calidad).
 - Análisis multi-objetivo: Considera múltiples criterios simultáneamente para encontrar el conjunto de configuraciones Pareto-óptimas.

1.1.3. Lenguaje Universal de Variabilidad (UVL)

La diversidad de notaciones y lenguajes para el modelado de la variabilidad ha sido históricamente un obstáculo para la colaboración científica y la transferencia tecnológica en el área de [SPL](#). Ante esta situación, y como resultado de una iniciativa comunitaria denominada MODEVAR, surgió el Lenguaje Universal de Variabilidad (*Universal Variability Language* - [UVL](#)), un esfuerzo colaborativo que ha involucrado a investigadoras e investigadores de universidades de España, Alemania, Austria y otros países durante más de seis años (David Benavides; Chico Sundermann; Feichtinger; Galindo; Rabiser y Thomas Thüm, [2025](#); Chico Sundermann; Feichtinger; Engelhardt; Rabiser y Thomas Thüm, [2021](#)).

El [UVL](#) se define como un lenguaje abierto e independiente de plataforma, diseñado específicamente para especificar modelos de variabilidad de forma textual. Su principal objetivo es proporcionar una representación común que pueda ser utilizada tanto en el ámbito académico como en la industria, facilitando así el intercambio de modelos, la comparación de resultados y el desarrollo de herramientas interoperables (UVL Community, [\[s.f.\]](#)). La creación de [UVL](#) responde a la necesidad de superar la fragmentación provocada por la existencia de múltiples lenguajes propietarios o académicos con capacidades y expresividad heterogéneas.

Aunque [UVL](#) nace en el contexto del software, su naturaleza genérica lo hace aplicable a cualquier dominio que requiera modelar variabilidad. El laboratorio *Diverso Lab* de la Universidad de Sevilla, liderado por el catedrático David Benavides, ha aplicado [UVL](#) en dominios tan diversos como las aplicaciones móviles, la automatización del marketing, la personalización de sensores para sistemas de riego, y recientemente, en la exploración de su aplicación al ámbito educativo para el modelado de currículos flexibles (Oficina de Transferencia de Resultados de Investigación - Universidad de Sevilla, [2025](#)). Esta versatilidad convierte a [UVL](#) en una opción particularmente adecuada para la presente investigación, donde se utilizará como notación base para representar la variabilidad curricular.

Diseño del lenguaje y niveles de expresividad

[UVL](#) especifica modelos de variabilidad con una estructura similar a un árbol para representar la estructura jerárquica de los modelos de variabilidad. El núcleo del lenguaje viene con varios conceptos para especificar restricciones, y son exactamente los mismos que se explican en la tabla [1.1](#) junto a las limitaciones de árboles cruzados (restricciones *cross-tree*).

El diseño de [UVL](#) se estructura en torno, además de su núcleo, en un sistema de niveles lingüísticos (*language levels*) que permiten incrementar su expresividad de forma controlada. Su sintaxis básica representa la estructura jerárquica de un [FM](#) mediante una notación de árbol indentado, similar a la utilizada en lenguajes de programación como Python (Chico Sundermann; Vill; Thomas Thüm; Feichtinger; Agarwal; Rabiser; Galindo y David Benavides, [2023](#)).

El lenguaje contempla los siguientes niveles principales (UVL Community, [\[s.f.\]](#)):

- **Nivel Booleano (*Boolean Level*):** Constituye la base del lenguaje e incluye:

- *Núcleo Booleano (Boolean Core)*: Soporta las relaciones fundamentales (obligatoria, opcional, OR, alternativa) y restricciones transversales proposicionales.
- *Cardinalidad de Grupo (Group Cardinality)*: Extiende las relaciones padre-hijo permitiendo especificar que se seleccionen entre n y m características hijas.
- **Nivel Aritmético (Arithmetic Level)**: Introduce la posibilidad de trabajar con atributos numéricos asociados a las características:
 - *Núcleo Aritmético (Arithmetic Core)*: Permite definir restricciones sobre atributos mediante operaciones aritméticas estándar (+, -, *, /, =, !=, >, <).
 - *Agregados de Atributos (Attribute aggregates)*: Simplifica la especificación de restricciones globales mediante funciones como `sum()` y `avg()`.
 - *Cardinalidad de Característica (Feature Cardinality)*: Permite que una misma característica pueda ser seleccionada múltiples veces dentro de un rango $[n..m]$.
- **Nivel Tipado (Type Level)**: Amplía el lenguaje con tipos de datos más complejos:
 - *Núcleo Tipado (Type Core)*: Introduce características de tipo cadena (*String Features*).
 - *Restricciones Numéricas (Numeric-Constraints)*: Añade operaciones como `floor()` y `ceil()` dentro de las restricciones.
 - *Restricciones de Cadena (String-Constraints)*: Permite operaciones como `len()` y comparación entre cadenas.

Además, [UVL](#) incorpora un mecanismo de importación que permite referenciar submodelos, facilitando así la composición de modelos de variabilidad de gran tamaño a partir de módulos más pequeños y gestionables (UVL Community, [s.f.]). El FM representado en las figuras 1.1 y 1.2 ejemplifica la sintaxis [UVL](#) y la correspondencia con su representación gráfica respectivamente.

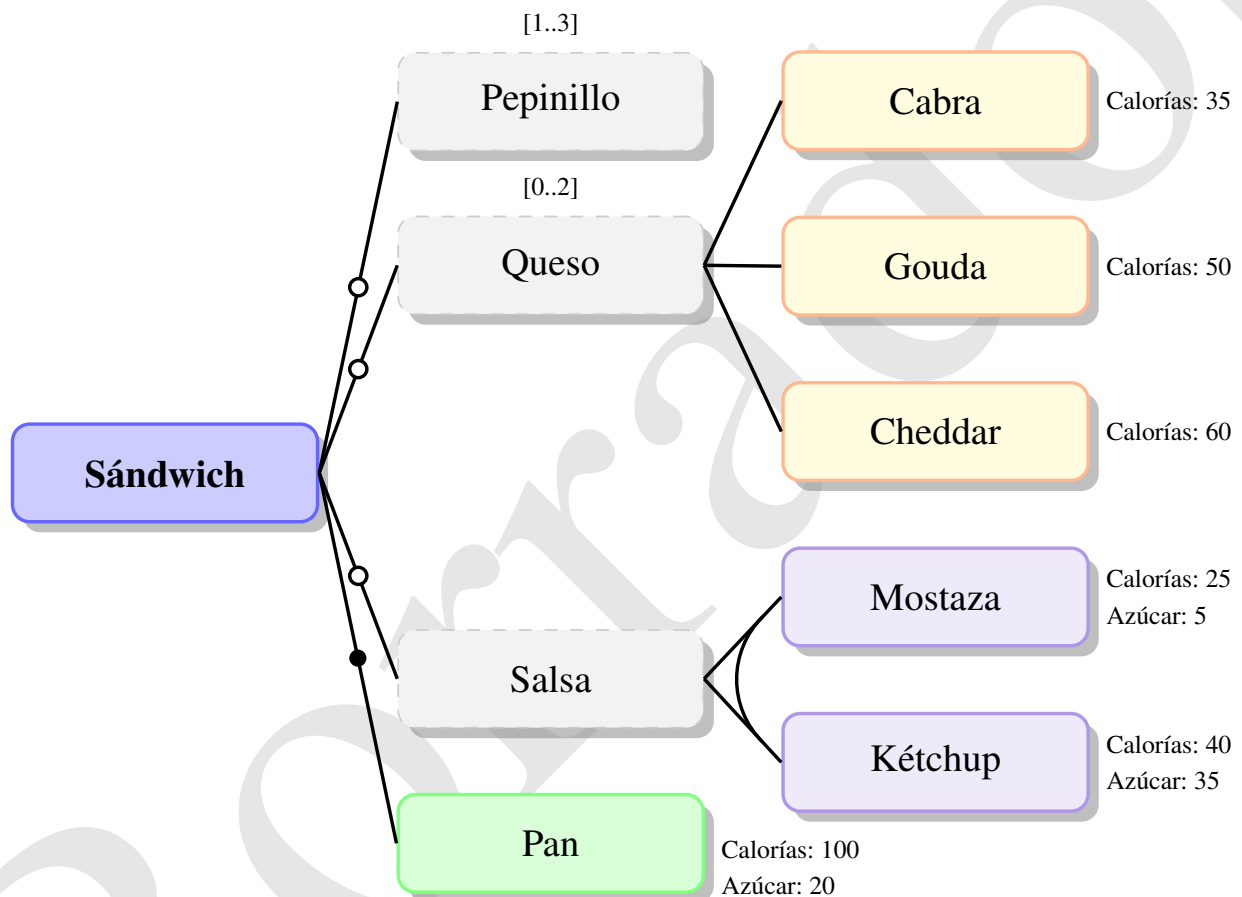
Código fuente 1.1. Ejemplo de modelo de variabilidad en [UVL](#). Fuente [ibíd.](#)

```
include
  Boolean.group-cardinality
  Arithmetic.aggregate-function
  Arithmetic.feature-cardinality
  Type
features
  Sandwich
    mandatory
      Pan {Calorias 100, Azucar 20}
    optional
      Salsa
        or
          Ketchup {Calorias 40, Azucar 35}
          Mostaza {Calorias 25, Azucar 5}
      Queso
```

```

[0..2] // Group cardinality
  Cheddar {Calorias 60}
  Gouda {Calorias 50}
  Cabra {Calorias 35}
  Pepinillo cardinality [1..3] // Feature cardinality
constraints
  Ketchup => Queso
  Pan.Azucar + Ketchup.Azucar + Mostaza.Azucar < 60 // Attribute constraints
  sum(Calorias) < 160 // Attribute aggregate

```



ID	Restricciones Cruzadas (UVL / Lógica Proposicional)
R1	Ketchup $\Rightarrow$ Queso
R2	Pan.Azúcar + Ketchup.Azúcar + Mostaza.Azúcar $\leq$ 60
R3	sum(Calorías) < 60

Figura 1.2. Representación gráfica del FM del modelo Sandwich especificado en UVL. Fuente: elaboración propia.

1.1.4. Configuraciones y configurador curricular

En el contexto de las [SPL](#), la configuración se refiere al proceso sistemático de seleccionar un conjunto específico de características de un [FM](#) para derivar un producto individual que satisfaga requisitos particulares (Apel; Batory; Kästner y Saake, 2013). Este proceso implica tomar decisiones de configuración que respeten todas las restricciones definidas en el modelo, asegurando que la selección final de características sea válida y consistente (D. Benavides; S. Segura y A. Ruiz-Cortés, 2010).

La configuración en [SPL](#) se caracteriza por ser un proceso guiado por restricciones donde las dependencias entre características (mandatorias, opcionales, alternativas) y las restricciones *cross-tree* (requiere, excluye) determinan el espacio de configuraciones posibles (Chen; Babar y Ali, 2020). La complejidad de este proceso aumenta exponencialmente con el tamaño del modelo, haciendo necesario el uso de herramientas automatizadas para garantizar la corrección de las configuraciones generadas (D. Benavides; S. Segura y A. Ruiz-Cortés, 2010).

Un configurador curricular es una herramienta especializada que aplica los principios de configuración de [SPL](#) al dominio de la planificación educativa, permitiendo la generación semi-automatizada de planes de estudio personalizados a partir de un modelo base que captura la variabilidad curricular (Sánchez; Gutiérrez y Cabrera, 2020). Este sistema opera como un motor de decisiones académicas que guía al usuario (gestor académico, coordinador de programa) a través del espacio de configuraciones válidas, garantizando que el plan de estudio resultante cumpla con todas las restricciones pedagógicas y administrativas (Chen; Babar y Ali, 2020).

El valor fundamental de un configurador curricular reside en su capacidad para reducir la complejidad del diseño curricular mediante la aplicación de restricciones formales que previenen configuraciones inválidas (Rodríguez; Vizcaíno y Piattini, 2019). Por ejemplo, el sistema puede asegurar automáticamente que:

- Se cumplan todos los prerrequisitos entre asignaturas y temáticas de las mismas.
- No existan asignaturas y/o temáticas que no esten relacionadas con otras o que sí lo estén, pero debido a las restricciones y reglas, es imposible seleccionar o incluir en cualquier configuración válida del producto.
- Los límites de créditos por semestre se respeten.
- La coherencia entre las competencias a desarrollar y las asignaturas seleccionadas se mantengan.
- Se eviten conflictos de horario y sobrecarga académica.
- No existan asignaturas o temáticas incompatibles en el mismo plan de estudio.
- Se garantice la inclusión de asignaturas obligatorias y la correcta selección de asignaturas optativas según las reglas definidas en el modelo curricular.
- Se cumplan las restricciones de diversidad temática y metodológica para asegurar un plan de estudio equilibrado y completo.

1.1.5. Gestión y flexibilidad curricular. Línea de Productos Educativos

La gestión curricular, como se expuso al inicio de este capítulo, constituye el conjunto de procesos mediante los cuales las instituciones educativas diseñan, implementan, evalúan y actualizan sus planes de estudio. Tradicionalmente, esta gestión se ha apoyado en modelos rígidos y documentos estáticos que dificultan la adaptación a contextos cambiantes (Gómez, 2023). Sin embargo, las demandas del siglo XXI —caracterizadas por la aceleración tecnológica, la diversidad de perfiles estudiantiles y la necesidad de formación continua— exigen un enfoque radicalmente distinto: la flexibilización curricular.

La flexibilidad curricular puede entenderse como la capacidad de un plan de estudios para adaptarse a las necesidades, intereses y contextos de los estudiantes o bien por necesidades circunstanciales, así como a las demandas del entorno social y productivo, sin perder coherencia ni calidad académica (Díaz, 2002). Esta flexibilidad opera en múltiples dimensiones:

- Flexibilidad en el tiempo: Posibilidad de cursar asignaturas en diferentes momentos, ritmos y secuencias.
- Flexibilidad en el espacio: Modalidades presencial, híbrida o virtual.
- Flexibilidad en los contenidos: Selección de asignaturas optativas, énfasis, menciones o itinerarios formativos.
- Flexibilidad metodológica: Diversidad de enfoques pedagógicos (aprendizaje basado en proyectos, aula invertida, gamificación).
- Flexibilidad en la evaluación: Adaptación de los mecanismos de acreditación a las características del estudiante y del contexto.

Lograr esta flexibilidad de manera sistemática y sostenible requiere superar las prácticas artesanales basadas en documentos y hojas de cálculo. Es aquí donde los paradigmas de la ingeniería de software ofrecen un camino prometedor.

Hacia las Líneas de Productos Educativos

La aplicación del paradigma de SPL al dominio educativo da lugar al concepto de Líneas de Productos Educativos. Esta puede definirse como una familia de artefactos educativos (planes de estudio, cursos, materiales didácticos, actividades de aprendizaje) que comparten un núcleo común y que pueden ser configurados sistemáticamente para generar productos adaptados a contextos, perfiles o necesidades específicas (Chimalakonda y Nori, 2012; Okada; Hisazumi; Nakanishi y Fukuda, 2009).

La idea fundamental consiste en trasladar los principios de la SPLE —gestión de variabilidad, reutilización planificada, activos centrales, puntos de variación— al diseño y gestión curricular. Así, un tronco curricular común (asignaturas obligatorias, competencias básicas, estructura general) constituye los activos centrales de la línea, mientras que los itinerarios formativos, las asignaturas optativas, las menciones o las modalidades de impartición se modelan como puntos de variabilidad (Silva Marcolino y Francine Barbosa, 2017).

Investigaciones pioneras en este campo han demostrado la viabilidad y los beneficios de este enfoque:

- Okada; Hisazumi; Nakanishi y Fukuda (2009) propusieron una metodología para mejorar la reutilización de materiales educativos mediante **SPLE**. Su trabajo demostró que la separación de partes comunes y variables en los materiales docentes permite generar variantes adaptadas a diferentes escenarios de uso, resolviendo los problemas de consistencia que surgen cuando los educadores gestionan múltiples variantes de forma artesanal.
- Chimalakonda y Nori (2012) exploraron la aplicación de **SPL** en tecnologías educativas para abordar la diversidad lingüística y contextual de la India. Su investigación evidenció que el desarrollo de soluciones educativas para 22 idiomas y múltiples contextos socioculturales puede beneficiarse enormemente de un enfoque sistemático de reutilización, reduciendo el esfuerzo de desarrollo de años-persona a procesos de configuración guiados.
- Silva Marcolino y Francine Barbosa (2017) diseñaron una arquitectura de línea de productos para construir aplicaciones de aprendizaje móvil (m-learning) orientadas a la enseñanza de programación. Su trabajo identificó las características comunes y variables en este dominio y demostró cómo el enfoque **SPL** puede facilitar la creación de herramientas educativas adaptadas a diferentes estilos de aprendizaje y contextos institucionales.
- Solis Pino; Ruiz y Hurtado Alegria (2022) aplicaron **SPL** como herramienta educativa para el aprendizaje de programación de robots industriales con Arduino. Su estudio con estudiantes de primer año universitario mostró que el enfoque permitió reutilizar entre el 38 % y el 43 % del código total necesario para programar los robots, mejorando significativamente el tiempo de desarrollo y facilitando el aprendizaje de conceptos de abstracción y modularidad.
- Galán y otros (2014) refactorizaron una aplicación de e-learning (E-Learning Web Miner) como una línea de productos, demostrando que el paradigma **SPL** puede aliviar los costosos y propensos a errores procesos de adaptación manual que caracterizan a muchas plataformas educativas cuando deben ajustarse a diferentes contextos de enseñanza-aprendizaje.

1.2. Fortalezas de los Modelos de Característica aplicado en la gestión curricular

La verdadera potencia del **FM** radica en que, una vez formalizado, se convierte en un modelo lógico-matemático que define explícitamente el espacio de diseño o el conjunto de configuraciones válidas (2^n) que se pueden generar (donde n es el número de características). Esta formalidad permite realizar análisis automatizados esenciales mediante herramientas conocidas como solucionadores de satisfacibilidad (*SAT Solvers*) o restricciones, permitiendo la validación de consistencia, que se encarga de determinar si un **FM** es consistente, es decir, si existe al menos una configuración válida; la Identificación de características muertas, lo cual permite encontrar las características que nunca pueden ser seleccionadas debido a la severidad de las

restricciones y posibilitan una configuración guiada, garantizándose que el producto final (ej. el plan de estudios) sea siempre válido.

Aunque existen otros modelos de variabilidad (como las tablas de decisión o los diagramas UML de variabilidad), el FM es el formalismo preferido por su simplicidad visual y su potencia analítica. A diferencia de modelos como las tablas, que pueden volverse inmanejables al aumentar el número de características, el FM provee una visión de alto nivel y, simultáneamente, una base lógica binaria robusta para el análisis automatizado de la consistencia.

En los próximos epígrafes se profundiza en esta línea argumental desde dos perspectivas complementarias. En primer lugar, se establece una analogía formal curricular que traduce los principales conceptos de la SPLE al dominio educativo, demostrando mediante un ejemplo (figuras 1.3 y 1.4) concreto —como el modelado de la asignatura Estructura de Datos I— cómo los FMs pueden capturar la variabilidad intrínseca de los planes de estudio. Posteriormente, se sintetizan los beneficios de las SPL, destacando las ventajas que este enfoque aporta a la gestión curricular en términos de reutilización, consistencia, trazabilidad y exploración del espacio de configuraciones.

1.2.1. Analogía formal curricular

La aplicación de los FMs a la gestión y diseño curricular constituye el principal aporte metodológico de esta investigación. Esta extensión se fundamenta en la premisa de que la variabilidad, concepto central de la SPLE es intrínseca y necesaria en los planes de estudio modernos de la enseñanza superior.

Para fundamentar la propuesta, es imprescindible establecer una analogía (tabla 1.2) formal y coherente que permita sustituir los constructos de SPL por sus equivalentes en el dominio educativo, elevando el plan de estudios de un simple documento descriptivo a un activo configurable y analizable.

Cuadro 1.2. Analogía de conceptos de SPL en el Dominio Educativo. Fuente: Elaboración propia.

Concepto de SPL	Concepto análogo en planificación curricular	Justificación
Línea de Productos	Carrera o área de estudio	El tronco común de conocimientos que define una familia de perfiles profesionales
Producto Específico	Plan de estudio configuracional	La ruta académica única y válida para un estudiante o perfil de egreso definido
Característica	Unidad curricular(asignatura, módulo, tema de estudio)	La unidad funcional mínima que puede ser seleccionada u omitida
Característica Raíz	El currículo base (ej. la carrera Ingeniería en Ciencias Informáticas o en caso que se esté modelando una asignatura, el nombre de la misma)	La entidad que engloba todos los módulos y asignaturas posibles de la carrera

Continúa en la siguiente página

Cuadro 1.2 – Continuación de la página anterior.

Concepto de SPL	Concepto análogo en planificación curricular	Justificación
Variabilidad	Opciones académicas	El conjunto de decisiones disponibles (optatividad, electividad, elección de menciones) que permiten personalizar la ruta del estudiante

Yendo un paso más adelante para evidenciar la aplicabilidad del FM al dominio curricular, las figuras 1.3 y 1.4 muestran un ejemplo concreto de modelado de la variabilidad curricular basado en la asignatura Estructura de Datos I. Se expone el código 1.2 UVL y su esquema jerárquico conceptual con el fin ilustrativo y suficiente para comprender las decisiones de variabilidad del FM.

Código fuente 1.2. Ejemplo de modelo de variabilidad en UVL para la asignatura Estructura de Datos I.

```
namespace EstructuraDatos
include
  Boolean.group-cardinality
  Arithmetic.aggregate-function
  Arithmetic.feature-cardinality
  Type
features
  Estructura de Datos I
    mandatory
      Fundamentos y Análisis {Horas 6}

      Estructuras Lineales
        cardinality [1..*]
        mandatory
          Arreglos
          Listas Enlazadas
        optional
          Listas Circulares

      Pilas {Horas 6}
        mandatory
          Implementacion
            alternative
              Con Arreglos {Horas 4, Dif 1}
              Con Listas {Horas 6, Dif 2}

      Colas {Horas 8}
        mandatory
          FIFO {Dif 2}
          Prioridad {Dif 3}
        optional
```

```

Deque {Dif 2}

Árboles Binarios {Horas 12, Dif 4}

Algoritmos Ordenamiento {Horas 10}

Lenguaje
  alternative
    C/C++ {Carga 9}
    Java {Carga 6}
    Python {Carga 4}

Técnicas de Programación
  mandatory
    Recursión Avanzada {Carga 8}
    Generica {Carga 10}
    Manejo de Memoria {Carga 8}

Herramientas
  or
    IDE {Carga 5}
    Debugger {Carga 2}
    Git {Carga 4}

Análisis de Complejidad
  mandatory
    Notación Big-0
    Análisis Temporal
    Análisis Espacial

constraints
  // R1: Prerrequisito pedagógico
  Árboles Binarios => Recursión Avanzada

  // R2: Dependencia técnica de bajo nivel
  C/C++ <=> Manejo de Memoria

  // R3: Restricción por abstracción del lenguaje
  Python => !Manejo de Memoria

  // R4: Restricción de recursos (Suma de atributos)
  sum(Horas) <= 64

  // R5: Restricción de viabilidad (Cálculo entre atributos)
  (avg(Dif) * Lenguaje.Carga) < 50

```

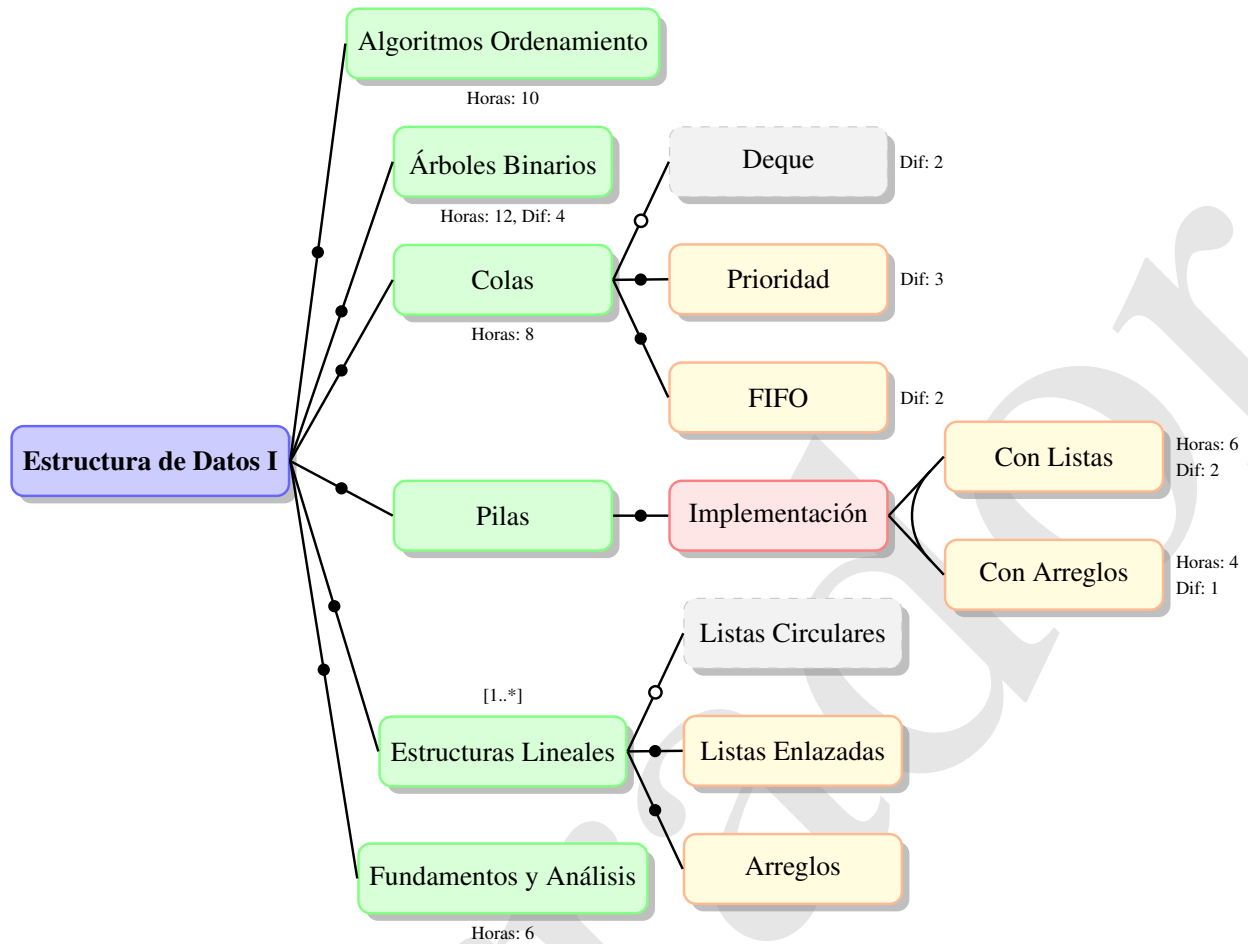
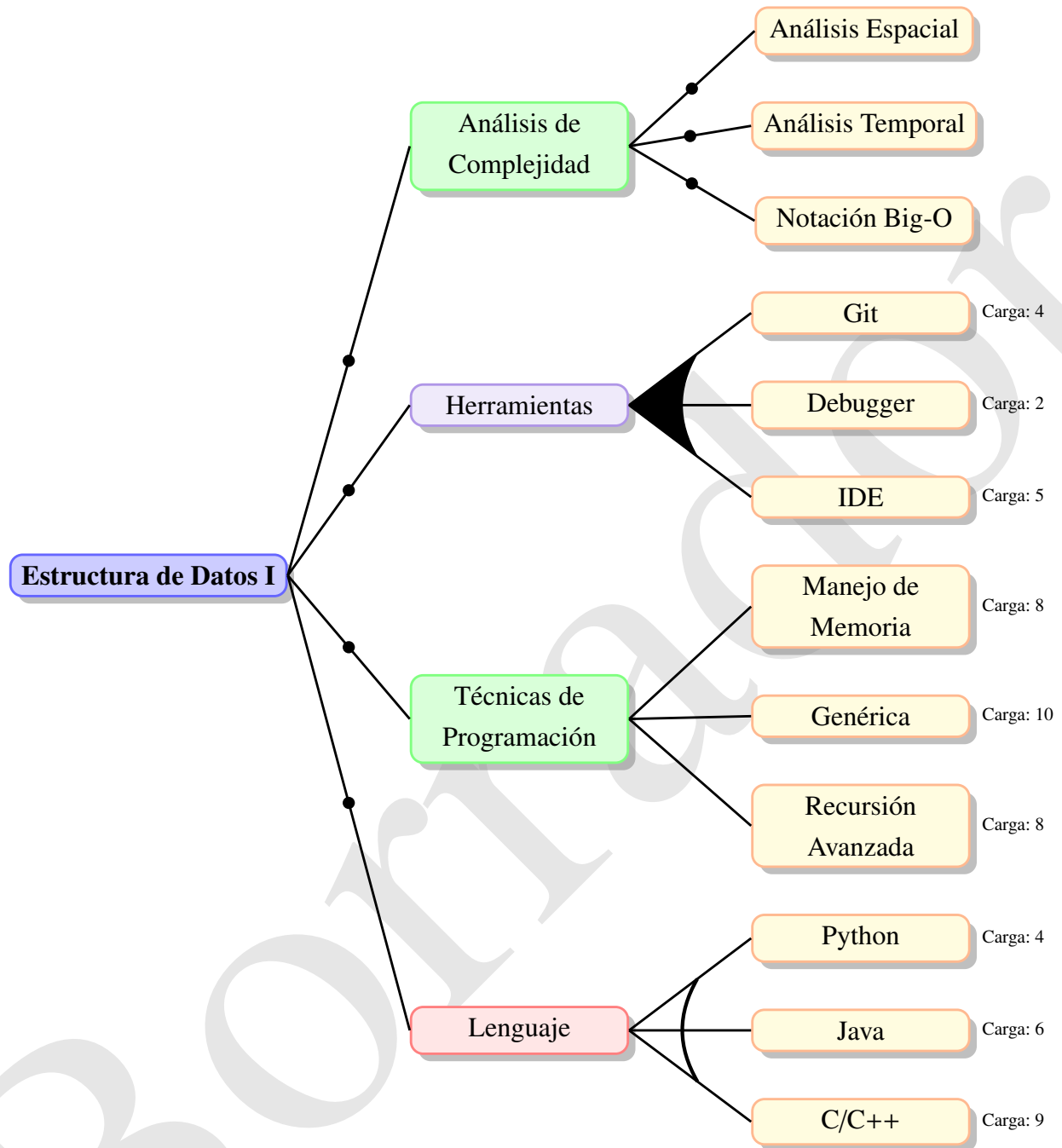


Figura 1.3. Representación simplificada #1 del FM de la asignatura Estructura de Datos I. Fuente: elaboración propia.



ID	Restricciones Cruzadas (UVL / Lógica Proposicional)
R1	Árboles Binarios $\Rightarrow$ Recursión Avanzada
R2	C/C++ $\Leftrightarrow$ Manejo de Memoria
R3	Python $\Rightarrow \neg$ Manejo de Memoria
R4	$\text{sum}(\text{Horas}) \leq 64$ (Límite semestral)
R5	$\text{avg}(\text{Dificultad}) * \text{Lenguaje.Carga} < 50$ (Umbral de viabilidad)

Figura 1.4. Representación simplificada #2 del FM de la asignatura Estructura de Datos I. Fuente: elaboración propia.

Este caso permite demostrar cómo una asignatura, lejos de ser un listado estático de contenidos, posee una estructura jerárquica, elementos obligatorios y opcionales, variabilidad controlada, dependencias de prerrequisitos y restricciones internas; todas ellas propiedades capturables dentro de un FM formal. El modelo mostrado (figuras 1.3 y 1.4) constituye una representación declarativa de la asignatura, estructurada como un árbol donde la característica raíz corresponde al curso completo y las características hijas representan módulos, temas o decisiones didácticas. Esta representación no solo captura los contenidos, sino que los organiza con semántica formal, permitiendo análisis automatizable. Los ejemplos ilustra distintos tipos de variabilidad presentes en currículos reales: contenidos obligatorios (módulos troncales), opcionales (tópicos avanzados), alternativas mutuamente excluyentes (implementación de pilas), configuraciones tipo OR (técnicas complementarias) y variantes dependientes (manejo de memoria solo si la implementación se realiza en C/C++).

La estructura del FM muestra claramente cómo una asignatura puede contener subniveles jerárquicos complejos, donde cada nivel constituye un subconjunto configurable. Esto permite modelar asignaturas que se imparten según distintas líneas, enfoques o lenguajes de implementación (C/C++, Java, Python), conservando la validez curricular y evitando inconsistencias.

La variabilidad no solo define qué temas se abordan, sino cómo se implementan, qué carga cognitiva poseen y qué prerrequisitos deben cumplirse. La presencia de relaciones cruzadas de árboles, restricciones lógicas e implicaciones capturan dependencias reales, tales como: para comprender árboles es necesario estudiar recursión (relación R1), o bien optar por C/C++ implica la posibilidad de manejo manual de memoria (relación R2). Esta lógica curricular, que en los planes tradicionales queda implícita o depende de interpretación docente, aquí se vuelve explícita, formal y verificable. Además, la inclusión de atributos numéricos (Horas, Dificultad, Carga) permite que el FM funcione como un sistema de soporte a la decisión. Restricciones como R4 y R5 demuestran cómo el formalismo puede garantizar que una configuración curricular no solo sea lógicamente válida, sino también factible en términos de carga cognitiva para el estudiante y disponibilidad horaria del programa.

De este modo, la asignatura deja de ser un documento textual estático para convertirse en un componente configurable. El FM presenta la asignatura como un micro-espacio de variabilidad curricular, análogo a un “producto configurable” dentro de una línea de productos académicos. Diferentes configuraciones del FM representan distintas adaptaciones válidas de la asignatura según perfiles de estudiante, competencias, niveles de profundidad o enfoques pedagógicos. Este modelo puede procesarse mediante un motor lógico para verificar su consistencia, detectar características inaccesibles y generar configuraciones válidas automáticamente.

Por tanto, los FMs no solo son adecuados para representar planes de estudios completos, sino también para modelar unidades académicas individuales con variabilidad interna. Esto abre la puerta a una flexibilidad curricular controlada, verificable y sustentada en formalismos computacionales, alineándose con los principios de SPLE aplicados a la planificación educativa.

1.2.2. Beneficios de las Líneas de Productos Educativos

En una Línea de Productos Educativos, la variabilidad curricular se convierte en el objeto central de modelado y gestión. Retomando los conceptos de [FM](#) y [UVL](#) desarrollados en secciones anteriores, es posible representar explícitamente:

- Características obligatorias: Asignaturas troncales, competencias básicas, prácticas profesionales obligatorias.
- Características opcionales: Asignaturas optativas, seminarios complementarios, actividades extracurriculares.
- Relaciones de alternancia: Menciones o itinerarios excluyentes entre sí (ej., mención en Ingeniería de Software vs. mención en Inteligencia Artificial).
- Agrupaciones OR: Conjuntos de asignaturas entre las cuales debe elegirse un mínimo (ej., elegir al menos 3 de 5 asignaturas de un bloque).
- Restricciones (*cross-tree*): Prerrequisitos entre asignaturas, incompatibilidades, correlatividades, límites de créditos.
- Atributos: Créditos, horas lectivas, semestre recomendado, área de conocimiento, modalidad de impartición.

La adopción del enfoque [SPL](#) en la gestión curricular reporta beneficios análogos a los observados en el desarrollo de software (Apel; Batory; Kästner y Saake, [2013](#)). De los cuales se tiene:

- Reutilización sistemática: Los diseños curriculares, una vez validados, pueden ser reutilizados para generar nuevos programas académicos sin partir desde cero.
- Consistencia y trazabilidad: La representación formal de la variabilidad garantiza que todos los planes de estudio derivados respeten las mismas reglas y restricciones, facilitando la trazabilidad de los cambios.
- Reducción de errores: Al automatizar la validación de configuraciones, se eliminan los errores humanos derivados de la gestión manual (inconsistencias, omisiones, conflictos).
- Agilidad en la actualización: Las modificaciones al modelo central se propagan automáticamente a todos los productos derivados, manteniendo la coherencia institucional.
- Exploración del espacio de configuraciones: Herramientas de análisis automatizado pueden responder preguntas como “¿cuántos planes de estudio válidos pueden generarse?” o “¿qué asignaturas quedan sin posibilidad de ser cursadas?”.

1.3. Análisis del mercado

El presente epígrafe desarrolla un análisis sistemático del mercado relacionado con los sistemas de gestión curricular y herramientas de modelado de variabilidad. Se examinan tanto las soluciones tradicionales

empleadas por instituciones académicas como aquellas provenientes del ámbito de la [SPLE](#), identificando sus fortalezas, limitaciones y brechas funcionales. A partir de esta revisión, se fundamenta la oportunidad tecnológica que sustenta la propuesta investigativa, destacando el vacío existente entre las plataformas de administración académica y los mecanismos de planificación curricular basados en variabilidad, lo que justifica la necesidad y pertinencia de la solución desarrollada.

1.3.1. Panorama General del Mercado

El mercado de sistemas de gestión curricular presenta una segmentación clara entre soluciones genéricas de administración académica y herramientas especializadas en modelado de variabilidad. Mientras plataformas como Moodle, Blackboard y Canvas dominan el segmento de los [Sistema de Gestión del Aprendizaje \(LMS\)](#) (Anthology Inc., [2025](#); Instructure, [2025](#); Moodle Project, [2025](#)), existen vacíos significativos en soluciones específicas para la planificación curricular basada en Líneas de Productos Educativos (Sánchez; Gutiérrez y Cabrera, [2020](#)).

1.3.2. Análisis de Soluciones Existentes

El análisis de soluciones existentes se organiza en dos grupos claramente diferenciados, cada uno representativo de perspectivas y alcances funcionales opuestos dentro del dominio investigado. Por un lado, se examinan los sistemas institucionales de gestión curricular tradicional, ampliamente implementados en universidades, cuyo propósito principal es administrar información académica pero que presentan limitaciones para modelar y validar variabilidad curricular. Por otro lado, se estudian herramientas avanzadas de [SPLE](#), diseñadas para gestionar características y restricciones complejas, pero que requieren conocimientos técnicos elevados y carecen de adaptación al contexto educativo. Esta comparación permite identificar brechas tecnológicas y conceptuales que fundamentan la necesidad de una solución especializada que integre fortalezas de ambos mundos mientras atiende las demandas propias de la planificación curricular.

Sistemas de Gestión Curricular Tradicionales

- **SIGA y Banner:** Los sistemas como Banner son fundamentalmente sistemas de información transaccionales diseñados para la gestión operativa de una institución. Su arquitectura está optimizada para manejar grandes volúmenes de datos de estudiantes, matrículas, registros de cursos y pagos. La evidencia de su funcionamiento, como se observa en la documentación de "Banner Student", muestra una estructura basada en bloques, ventanas y secciones para la gestión de admisiones, currículos y campos de estudio (Ellucian Company, [2015](#)). Esta lógica está orientada a capturar y recuperar información de forma consistente, no a modelar relaciones complejas y variables entre componentes curriculares. Carecen de un motor de reglas nativo que permite definir y verificar restricciones complejas como las que se modelan con "requiere" o "excluye" en un [FM](#). Su limitación principal es que operan sobre una estructura de datos fija y predefinida, lo que los hace rígidos para experimentar con diferentes

configuraciones curriculares o para gestionar una línea de productos educativos donde las relaciones entre asignaturas y competencias son dinámicas y variables.

- Moodle: como [LMS](#), es una plataforma excelente para la ejecución y gestión de un plan de estudios ya definido. Su fortaleza reside en organizar cursos, desglosarlos en unidades y lecciones, gestionar la entrega de contenidos, las actividades de evaluación y la interacción entre estudiantes y profesores (Moodle Project, [2025](#)). Sin embargo, su funcionalidad no se extiende al diseño instruccional ni a la planificación curricular basada en características. Moodle asume que el diseño del curso (los objetivos, la secuencia de contenidos, las actividades de evaluación) ya ha sido realizado externamente. No proporciona herramientas para que los gestores académicos modelen visualmente un plan de estudios, definan características opcionales u obligatorias, o establezcan restricciones entre ellas. En el contexto de la investigación, Moodle sería un consumidor potencial de la configuración curricular final generada por la propuesta de solución, pudiéndose integrar como plataforma de entrega y gestión de los cursos derivados, pero no como herramienta para diseñar o validar la estructura curricular en sí.

Herramientas de Modelado de Variabilidad

- Pure::variants: es una solución comercial robusta para [SPLE](#). Proporciona un entorno completo para definir características, crear modelos, gestionar restricciones y, crucialmente, derivar productos concretos a partir de configuraciones válidas. Su potencia es innegable en el dominio técnico para el que fue diseñado (pure-systems GmbH, [2024](#)). El problema central para el contexto educativo es su alta barrera de entrada para usuarios no técnicos. Los gestores académicos y diseñadores curriculares, que son expertos en pedagogía pero no necesariamente en [SPLE](#), se verían enfrentados a una terminología y flujos de trabajo propios de la ingeniería de software. Adaptar pure::variants requeriría una personalización significativa de la interfaz y una capacitación intensiva, lo que dificulta su adopción ágil en un entorno académico, validando la necesidad de una solución más especializada y accesible.
- FeatureIDE: es un entorno de código abierto construido como un complemento (*plugin*) para Eclipse, lo que lo hace muy popular en entornos de investigación y enseñanza de la ingeniería de software (T. Thüm; Kästner; Benduhn; Meinicke; Saake y Leich, [2014](#)). Al igual que pure::variants, es poderoso para el análisis y la configuración de [FMs](#). Sin embargo, su dependencia de Eclipse y su orientación hacia desarrolladores lo convierten en una herramienta que requiere especialización técnica. Para un coordinador de carrera o un vicerrector académico, interactuar con FeatureIDE supondría una curva de aprendizaje muy pronunciada. Su arquitectura no está diseñada para integrarse de manera natural con los sistemas de información universitarios existentes (como los propios SIGA o Banner), lo que limita su utilidad práctica para la gestión curricular del día a día. Su uso en educación se limita predominantemente a la enseñanza de conceptos de [SPL](#), no a la gestión operativa de currículos.
- Flamapy / FlamapyIDE: es un ecosistema de código abierto para el análisis automatizado de [FMs](#), desarrollado activamente por el grupo de investigación Diverso Lab de la Universidad de Sevilla, liderado por el catedrático David Benavides (Galindo; Horcas; Felferning; Fernandez-Amoros y Da-

vid Benavides, 2023). A diferencia de pure::variants y FeatureIDE, Flamapy se concibe desde su origen como un “*framework* Python” modular y extensible, diseñado para facilitar la integración de capacidades de análisis de variabilidad en otras aplicaciones, lo que lo emparenta directamente con el objetivo de la presente investigación. El *framework* soporta múltiples formatos de entrada, incluyendo UVL, FeatureIDE (.fide) y otros (Team, 2025a), y proporciona transformaciones a representaciones intermedias como Diagrama de Decisión Binaria (BDD) para análisis eficientes (Team, 2025c). Flamapy ofrece tres interfaces principales que lo convierten en una propuesta particularmente relevante (Team, 2024):

- FlamapyIDE: un entorno de edición y análisis basado en web, accesible en <https://ide.flamapy.org/editor>, que permite cargar modelos UVL o FeatureIDE, visualizar su estructura, y ejecutar operaciones de análisis (validación de consistencia, detección de características muertas, conteo de configuraciones, etc.) sin necesidad de instalar software local. Esta interfaz gráfica reduce significativamente la barrera de entrada para usuarios no técnicos.
- Interfaz de Programación de Aplicaciones (API) de Transferencia de Estado Representacional (REST): Flamapy proporciona una API RESTful que expone sus capacidades de análisis como servicios web, permitiendo que otras aplicaciones (como el *backend* propuesto en esta investigación) integren funcionalidades avanzadas de validación y configuración de modelos de características mediante llamadas de Protocolo de Transferencia de Hipertexto (HTTP) (ibíd.). La API puede desplegarse localmente mediante Docker o accederse a versiones alojadas, facilitando su adopción en arquitecturas de microservicios.
- Línea de comandos y biblioteca Python: para integraciones más profundas, Flamapy ofrece una interfaz de línea de comandos y una biblioteca Python que permite incorporar directamente sus capacidades en aplicaciones desarrolladas en este lenguaje (Team, 2025b).

1.3.3. Oportunidades de Mercado Identificadas

Del análisis realizado se evidencia que el mercado asociado a la temática padece de un vacío crítico en herramientas capaces de integrar la gestión de variabilidad, propia de las SPL, con la planificación curricular educativa, creando una oportunidad para sistemas especializados (Sánchez; Gutiérrez y Cabrera, 2020). Las herramientas existentes de SPL están diseñadas para ingenieros de software, no para profesionales de la educación ni mucho menos para la gestión curricular, limitando su adopción en instituciones educativas y obligando a comprender los conceptos fundamentales de SPLE (Chen; Babar y Ali, 2020). En el caso de Flamapy, destacar que no permite gestionar y operar el FM desde la vista gráfica (visualización del árbol), sino mediante la escritura de UVL. En este escenario, la necesidad de proponer una solución propia no surge de la ausencia total de tecnologías, sino de la falta de una alternativa que articule las fortalezas identificadas en ambos tipos de herramientas y las adapte al dominio educativo.

De los sistemas analizados se reconoce como indispensable la capacidad de representar explícitamente características con relaciones jerárquicas y restricciones, la validación automática de configuraciones, la

posibilidad de derivar múltiples variantes a partir de un modelo común y la interoperabilidad con ecosistemas académicos existentes. Flamapy, en particular, se destaca su funcionamiento en capturar y modelar los FMs a partir de escribir código en UVL, su amplio abanico de exportación a diferentes formatos de archivos y la capacidad de diagnosticar los FMs (funcionalidad no ausente en los otros 2 sistemas analizados) en búsquedas de estadísticas, como cantidad de configuraciones, características muertas, etc. La relevancia de Flamapy para la presente investigación es múltiple:

- Su naturaleza como *framework* Python facilita la integración con la propuesta solución, permitiendo reutilizar sus capacidades de análisis en lugar de implementarlas desde cero.
- La existencia de una API REST validada y documentada proporciona un referente arquitectónico para el diseño del servicio.
- Su enfoque en accesibilidad (interfaz web, API) lo alinea con la necesidad de acercar las técnicas de SPL a gestores académicos no especializados.
- El equipo de desarrollo (Diverso Lab) es el mismo que lidera el estándar UVL, garantizando coherencia con las notaciones adoptadas en esta investigación.

La solución propuesta se posiciona estratégicamente en el mercado educativo como un “Motor de Variabilidad Curricular” especializado, que capitaliza las tendencias actuales de crecimiento en educación híbrida y multimodal, así como las demandas crecientes de personalización educativa y eficiencia en la gestión académica post-pandemia. Su propuesta de valor se sustenta en fortalezas técnicas diferenciadoras, ofreciendo un servidor de aplicaciones (*backend*) que permita diseñar y gestionar modelos de características aplicados a planes de estudio, y derivar itinerarios formativos dinámicos y personalizables. Su valor diferencial radica en su especialización en gestión de variabilidad curricular, incorporar motores de reglas pedagógicas que comprenden prerrequisitos, competencias y secuencias, una arquitectura de API para integración con sistemas existentes y un modelo de datos optimizado para planificación académica. Estas capacidades se traducen en ventajas competitivas tangibles como la automatización de validaciones que reduce errores en diseño curricular, la generación de múltiples variantes de planes de estudio desde un tronco común y su adaptabilidad a contextos institucionales diversos, posicionándose como complemento esencial -no competencia- de los sistemas LMS existentes, al enfocarse específicamente en el diseño y planificación académica más que en la ejecución curricular. En consecuencia, la propuesta investigativa no se limita a replicar herramientas existentes, sino que se posiciona como un puente necesario entre el rigor formal de las SPL y la flexibilidad curricular demandada por la educación contemporánea, constituyendo así una innovación con pertinencia tecnológica y académico-formativa.

1.4. Herramientas y tecnologías

La implementación de la solución propuesta requiere la selección cuidadosa de tecnologías, herramientas y *frameworks* que permitan materializar de manera eficiente los modelos curriculares, soportar reglas de

variabilidad y garantizar una experiencia de desarrollo fluida. En este epígrafe se describen los componentes tecnológicos que conformarán la plataforma, justificando su elección. Asimismo, se analizan los entornos de desarrollo, lenguajes de programación, sistemas de persistencia y bibliotecas especializadas que facilitan la representación formal de restricciones curriculares y la validación automática de configuraciones. Esta caracterización permite comprender la base tecnológica que sustenta la solución y cómo cada herramienta contribuye al logro de sus objetivos funcionales y no funcionales.

1.4.1. Lenguaje de Programación

Los lenguajes de programación son herramientas formales que permiten expresar instrucciones de manera estructurada para que un sistema computacional ejecute tareas determinadas (Sebesta, 2016). A través de ellos se implementan algoritmos, modelos de datos, reglas de negocio y mecanismos de interacción con el usuario u otros sistemas. La elección de un lenguaje influye directamente en la escalabilidad, el rendimiento, la mantenibilidad y la capacidad de expresar soluciones complejas de manera eficiente. En el contexto del sistema propuesto, el lenguaje seleccionado debe ofrecer flexibilidad, capacidad de representación estructural y soporte robusto para lógica y concurrencia (I. Sommerville, 2016).

Entre los lenguajes de programación más utilizados para el desarrollo de aplicaciones del lado del servidor se encuentran Java, Python, C#, JavaScript\TypeScript (Node.js), Go, PHP, entre otros. Python será el lenguaje principal utilizado para el desarrollo del *backend* de la solución propuesta. Su elección se fundamenta en su madurez tecnológica, su amplio ecosistema de bibliotecas y la claridad sintáctica que facilita la implementación de modelos complejos manteniendo niveles altos de legibilidad y mantenibilidad del código (Lutz, 2013; Van Rossum y Drake, 1995). Python es ampliamente utilizado en el ámbito académico y profesional, lo que garantiza disponibilidad de documentación, ejemplos de buenas prácticas y una sólida comunidad de soporte (Ray; Posnett; Filkov y Devanbu, 2014).

Una de las principales fortalezas de Python radica en su capacidad de integración con múltiples paradigmas de desarrollo, permitiendo tanto programación estructurada como orientada a objetos y funcional (Lutz, 2013). Esto resulta particularmente útil para el diseño modular del sistema, la definición formal de FM y la implementación de validaciones de reglas en motores lógicos.

La representación estructurada de FMs requiere un lenguaje capaz de expresar jerarquías, relaciones lógicas, restricciones transversales y cardinalidades con claridad y sin ambigüedad. Python ofrece capacidades idóneas para este propósito debido a su flexibilidad en estructuras de datos y su riqueza semántica para definir invariantes y reglas de negocio. La cantidad de líneas de código que se requiere para operar los FMs en Python es drásticamente menor que en comparación con otros lenguajes como pueden ser Java y C#, lo que mejora la experiencia de desarrollo.

En primer lugar, Python permite modelar FMs de forma natural mediante diccionarios, listas y clases, manteniendo la semántica jerárquica del dominio sin necesidad de sintaxis complejas ni compilación intermedia. Esto facilita representar nodos, ramas, grupos OR/XOR, atributos y relaciones *cross-tree* como estructuras anidadas. Gracias a esta expresividad, resulta posible describir distintos niveles del modelo

—desde características obligatorias y opcionales hasta restricciones de cardinalidad— de manera directa y legible, lo que favorece tanto su mantenimiento como su inspección. Adicionalmente, esta flexibilidad estructural de Python facilita la implementación de mecanismos de serialización y deserialización que permiten la interoperabilidad con formatos estándar de la comunidad de [SPL](#), como [UVL](#) (Chico Sundermann et al., 2021). [UVL](#) se ha propuesto como un formato independiente del lenguaje para representar modelos de variabilidad, y Python, gracias a su soporte para procesamiento de texto, análisis sintáctico y generación de estructuras de datos, permite construir traductores bidireccionales que convierten modelos [UVL](#) en representaciones internas del sistema y viceversa, asegurando la portabilidad de los modelos entre distintas herramientas y entornos de desarrollo.

Además, Python ofrece un ecosistema amplio de herramientas para la validación programática de configuraciones. El sistema puede implementar algoritmos de selección y verificación que comprueben automáticamente consistencia lógica, satisfacibilidad de prerequisites, exclusiones y cumplimiento de restricciones. La sintaxis de Python resulta especialmente adecuada para traducir expresiones lógicas como requerimientos, exclusiones o implicaciones, ya sea mediante evaluaciones directas o a través de módulos que permiten construir árboles lógicos, satisfacibilidad booleana o incluso resolución por *backtracking*.

Adicionalmente, la flexibilidad del lenguaje, junto con el respaldo de su amplio ecosistema de bibliotecas, facilita la construcción de algoritmos que no solo derivan configuraciones de forma eficiente, sino que también las verifican mediante evaluaciones lógicas, análisis de dependencias y resolución de conflictos. Este mismo soporte permite implementar mecanismos de versionado y *snapshots* del modelo, conservando estados históricos y garantizando que la evolución del [FM](#) no invalide configuraciones previas, preservando así la consistencia y trazabilidad del sistema en el tiempo.

En conjunto, estas características hacen de Python un lenguaje altamente apropiado tanto para la representación formal de [FMs](#) como para la generación y validación automatizada de configuraciones dentro del sistema propuesto (Ray; Posnett; Filkov y Devanbu, 2014). Por su equilibrio entre simplicidad expresiva y potencia funcional, Python constituye una base tecnológica idónea para el *backend* del sistema, permitiendo acelerar el desarrollo sin sacrificar calidad, extensibilidad ni alineación con los requisitos funcionales de la solución.

1.4.2. *Framework* de desarrollo

Los *frameworks* de desarrollo son estructuras software que proporcionan un conjunto de herramientas, patrones, módulos y convenciones que simplifican la construcción de aplicaciones (I. Sommerville, 2016). Permiten acelerar la implementación de funcionalidades complejas, garantizando buenas prácticas de diseño y facilitando la mantenibilidad del software (Pressman, 2014). Un *framework* adecuado reduce la repetición de código, mejora la seguridad, permite la modularización y posibilita el desarrollo escalable. En este proyecto, el *framework* seleccionado debe soportar concurrencia, validación, comunicación con bases de datos y un desarrollo claro basado en [APIs](#) (Gamma; Helm; Johnson y Vlissides, 1995).

Algunos de los *frameworks backend* con los que cuenta Python son Django\ [Django REST Framework](#)

(DRF), FastAPI, AIOHTTP, Flask, Tornado, entre otros no tanto conocidos o aclamados por la comunidad. Se evaluaron (tabla 1.3) estas opciones considerando criterios como rendimiento, facilidad de desarrollo, soporte para tipado, validación automática y capacidades asíncronas.

Cuadro 1.3. Comparación entre *frameworks* de Python. Fuente: elaboración propia.

<i>Framework</i>	Tipo	Rendimiento	Curva de aprendizaje	Ideal para
Django	<i>Full-stack</i>	Medio	Media	Sistemas completos
DRF	<i>Full-stack API</i>	Medio	Media	APIs sobre Django
FastAPI	Asíncrono	Alto	Baja	APIs modernas
Flask	Micro	Medio	Muy baja	Prototipos, servicios
Tornado	Asíncrono	Alto	Alta	Tiempo real
AIOHTTP	Asíncrono	Alto	Media	Clientes/servidores asíncronos

De la tabla 1.3 se puede concluir que, si bien DRF ofrece una solución robusta y madura para construir APIs sobre el ecosistema Django, su naturaleza síncrona y su curva de configuración pueden representar limitaciones para escenarios que requieren alta concurrencia y validación estricta de FMs. Por su parte, Tornado y AIOHTTP proporcionan excelente rendimiento asíncrono, pero demandan un desarrollo más manual y complejo. Flask, aunque sencillo y flexible, carece de tipado fuerte y validación automática, aspectos cruciales para garantizar la consistencia de los FMs y las configuraciones derivadas. En contraste, FastAPI fue seleccionado como el *framework* principal para el desarrollo del *backend* debido a su combinación equilibrada de alto rendimiento asíncrono, sintaxis intuitiva, simplicidad de desarrollo y capacidades avanzadas de validación mediante Pydantic (Ramírez, 2021).

Un factor determinante en la selección de FastAPI es su rendimiento comparable a implementaciones escritas en Go y Node.js, gracias a su construcción sobre Starlette (para la capa asíncrona) y Pydantic (para validación y serialización) (Pesce, 2021). Esta combinación permite manejar múltiples peticiones concurrentes, realizar verificaciones de reglas e interacciones dinámicas con el motor de configuraciones sin degradación perceptible del servicio, incluso bajo cargas elevadas.

Un argumento especialmente relevante para seleccionar FastAPI es la naturaleza intensiva en validaciones y operaciones **Entrada/Salida (I/O)** que caracteriza al sistema propuesto. La plataforma no solo debe recibir configuraciones y verificar reglas, sino también consultar persistencia, aplicar evaluaciones lógicas, generar respuestas alternativas y registrar versiones de modelo, todo ello de manera concurrente y potencialmente bajo múltiples usuarios. Debido a su arquitectura asíncrona nativa basada en `async/await`, FastAPI permite manejar este patrón de carga sin bloqueo, maximizando la eficiencia en tareas I/O y reduciendo latencias (Eugene y Anderson, 2023; Pesce, 2021). Esto se traduce en una capacidad real de escalar, sin incurrir en cuellos de botella característicos de *frameworks* síncronos, lo que resulta esencial en un sistema donde las consultas y validaciones son frecuentes, complejas y potencialmente concurrentes.

FastAPI aprovecha las características más recientes de Python (*type hints*) para ofrecer validación automática de datos, documentación interactiva generada dinámicamente (OpenAPI/Swagger) y soporte nativo para concurrencia, lo que resulta particularmente ventajoso para implementar operaciones de validación y derivación de configuraciones en FM (Ramírez, 2023). Esta capacidad de tipado fuerte permite mapear directamente las estructuras de los FMs —como características, restricciones y relaciones— a modelos Pydantic, asegurando que los datos procesados cumplan consistentemente con las reglas del dominio. Además, su enfoque modular y su integración fluida con bases de datos relacionales y no relacionales mediante bibliotecas como SQLAlchemy y Motor lo convierten en una opción versátil y preparada para escalar junto con los requisitos del sistema (Lubanovic, 2019). Por estas razones, FastAPI constituye la base tecnológica idónea para implementar la propuesta de solución, alineado con los objetivos funcionales y no funcionales del sistema.

1.4.3. Base de Datos y Gestor de Base de Datos

Las bases de datos son sistemas diseñados para almacenar, organizar y proteger información de manera eficiente y estructurada, garantizando su disponibilidad a lo largo del tiempo (*Database Management System - an overview* 2024). La persistencia de los modelos curriculares, configuraciones generadas, versiones históricas y metadatos asociados requiere de un sistema robusto, seguro y con garantías de consistencia transaccional.

Entre los sistemas de bases de datos más utilizados se encuentran MySQL, PostgreSQL, MongoDB y SQL Server. Se selecciona PostgreSQL como gestor de base de datos relacional, debido a su madurez, fiabilidad y su capacidad de manejar estructuras complejas con fuertes requisitos de integridad referencial (Momjian, 2001; PostgreSQL Global Development Group, 2024). PostgreSQL destaca por su cumplimiento estricto del estándar *Lenguaje de Consulta Estructurado (SQL)*, su modelo *Atomicidad, Consistencia, Aislamiento, Durabilidad (ACID)*, y su eficiente gestión de transacciones, elementos imprescindibles para representar relaciones jerárquicas entre características, restricciones *cross-tree*, y *snapshots* versionados del modelo sin comprometer la coherencia global del sistema (Momjian, 2001).

Además, PostgreSQL ofrece capacidades avanzadas como índices compuestos, consultas optimizadas, vistas materializadas, *triggers* y procedimientos almacenados, que resultan útiles para procesar datos curriculares con alto grado de estructuración (PostgreSQL Global Development Group, 2024). El motor también soporta tipos de datos *Notación de Objetos JavaScript (JSON)* y *JSON Binario (JSONB)*, permitiendo almacenar representaciones semiestructuradas de configuraciones y estados transitorios, por lo que proporciona flexibilidad para almacenar tanto esquemas estáticos como *snapshots* dinámicos relacionados con la evolución del FM (ibíd.).

Adicionalmente, PostgreSQL se integra de forma natural con SQLAlchemy como *Mapeo Objeto - Relacional (ORM)* dentro del *backend* desarrollado en FastAPI, facilitando el diseño declarativo de entidades, la gestión del ciclo de persistencia, la validación de relaciones y la migración segura del esquema. Esta integración favorece una arquitectura limpia y escalable, desacoplando las capas de datos, servicios y lógica

de negocio.

Para la administración visual del motor se empleará pgAdmin, interfaz ampliamente utilizada para la gestión de bases de datos PostgreSQL. Mediante esta herramienta es posible inspeccionar tablas, ejecutar consultas SQL, visualizar índices y restricciones, supervisar cargas de trabajo y realizar respaldos de forma controlada. En el contexto de la solución propuesta, pgAdmin permite auditar el estado de los modelos almacenados, verificar reglas impuestas, revisar configuraciones generadas y garantizar la trazabilidad del sistema.

En conjunto, la elección de PostgreSQL como sistema de persistencia y pgAdmin como herramienta administrativa proporciona una base confiable y alineada con las necesidades de integridad, consistencia y trazabilidad del sistema a desarrollar.

1.4.4. Contenerización y Orquestación

La contenerización es una tecnología que permite empaquetar aplicaciones junto con todas sus dependencias en unidades aisladas, reproducibles y portables llamadas contenedores. Esto garantiza que una aplicación se ejecute de la misma manera en cualquier entorno, eliminando problemas de incompatibilidad. Para ello se emplean herramientas especializadas, donde Docker se ha consolidado como estándar industrial. Por su parte, la orquestación permite administrar múltiples contenedores, asegurando coordinación, escalabilidad y resiliencia; funciones que normalmente se alcanzan mediante tecnologías como Docker Compose o Kubernetes. Este enfoque es crucial para despliegues confiables, mantenibles y escalables.

Para garantizar la portabilidad, reproducibilidad y facilidad de despliegue del sistema propuesto, se adoptará Docker como tecnología base de contenedorización (Boettiger, 2015; D. Inc., 2024). Docker permite encapsular los servicios de la aplicación —incluyendo el *backend* FastAPI, la base de datos PostgreSQL, la herramienta administrativa pgAdmin y otros servicios de utilidad o futuros módulos asociados— dentro de entornos aislados y autocontenidos. Esto elimina problemas de dependencias, divergencias de entorno y configuraciones inconsistentes entre despliegues locales y productivos (Boettiger, 2015). La capacidad de reproducir una imagen de forma determinística contribuye directamente a la robustez y mantenibilidad de la solución.

La contenedorización resulta especialmente útil para aplicaciones multicapa como la propuesta, donde distintos servicios deben colaborar manteniendo canales de comunicación estables. Docker permite garantizar uniformidad tanto en las dependencias del *backend*, como versiones de bibliotecas, configuración del servidor de aplicación y parámetros del motor de base de datos, disminuyendo errores derivados de diferencias de entorno y facilitando diagnósticos (D. Inc., 2024).

Sobre Docker se construye Docker Compose, herramienta de orquestación ligera que permite describir, mediante un único archivo declarativo, toda la topología de servicios del sistema (*ibíd.*). Esto incluye la configuración de redes internas, gestión de persistencia en volúmenes, variables de entorno, reglas de dependencias entre contenedores, puntos de exposición de puertos, así como políticas de reinicio. En el contexto del sistema de modelado curricular, Docker Compose facilita levantar el entorno completo de de-

sarrollo o producción con un solo comando, garantizando escalabilidad y reduciendo tiempos de despliegue o recuperación ante fallos (Boettiger, 2015).

Un beneficio adicional de esta estrategia es su contribución a las prácticas de [Integración Continua y Despliegue Continuo \(CI/CD\)](#). Gracias a la naturaleza declarativa y reproducible de los contenedores, el sistema puede desplegarse automáticamente mediante [pipelines] controlados sin intervención manual, asegurando que las versiones del modelo curricular, las configuraciones generadas y los *snapshots* almacenados sean accesibles en entornos iguales o equivalentes entre sí (D. Inc., 2024).

1.4.5. Almacenamiento en Memoria, *Caching* y Mensajería

El almacenamiento en memoria es una técnica que permite guardar temporalmente datos en la [Memoria de Acceso Aleatorio \(RAM\)](#) para acelerar el acceso a información que se consulta de forma recurrente, evitando operaciones de disco o consultas de base de datos innecesarias. El *caching* se basa precisamente en optimizar velocidad, reduciendo latencias y mejorando el rendimiento global del sistema. Existen herramientas especializadas que permiten gestionar este almacenamiento temporal, soportar estructuras clave-valor y facilitar comunicación entre servicios. Redis destaca entre ellas por su eficiencia, simplicidad y capacidades adicionales como mensajería, *pub/sub* y persistencia configurable, lo que la convierte en una pieza clave para arquitecturas modernas.

Redis será incorporado como componente clave para el almacenamiento temporal y procesamiento eficiente de información crítica durante las fases de generación y validación de configuraciones curriculares (Carlson, 2013). Su naturaleza de base de datos en memoria permite alcanzar tiempos de respuesta extremadamente bajos, lo cual resulta particularmente valioso en un sistema donde las operaciones de verificación de restricciones, derivación de configuraciones válidas y resolución de incompatibilidades pueden ejecutarse de forma intensiva y repetitiva.

A diferencia de un motor relacional tradicional, Redis está optimizado para operaciones clave-valor y estructuras avanzadas como listas, conjuntos, mapas *hash* y *sorted sets* (*ibíd.*). Estas estructuras permiten modelar transitoriamente elementos como:

- Configuraciones candidatas generadas por el usuario.
- Listas de incompatibilidades detectadas.
- Descriptores internos del [FM](#).
- Estados de sesión durante la exploración de alternativas.
- Cachés de resultados de validaciones repetitivas.

Este almacenamiento en memoria reduce significativamente la latencia, lo que contribuye a experiencias interactivas y a la capacidad del sistema de responder en tiempo real a decisiones del usuario (por ejemplo, activar o desactivar características, consultar prerrequisitos, etc.) (*ibíd.*).

Otro aspecto clave es que Redis permite persistencia opcional mediante *snapshots*, lo que lo convierte en un aliado directo del soporte de versionado de modelos y configuraciones (*ibíd.*). Es decir, puede alma-

cenar estados intermedios del proceso de modelado curricular sin interferir con la base de datos principal, permitiendo restaurar fácilmente versiones anteriores o explorar ramas alternativas del mismo modelo.

Además, Redis implementa un sistema de Time-To-Live ([Tiempo de Vida \(TTL\)](#)), lo que permite gestionar la caducidad automática de configuraciones temporales, sesiones y cálculos ya procesados. Esta funcionalidad contribuye a la integridad global del sistema y evita acumulación de datos residual (Carlson, 2013).

Finalmente, una característica determinante para su elección es su excelente integración con FastAPI, permitiendo aprovechar plenamente la naturaleza asíncrona del *backend* (Ramírez, 2019). Gracias a bibliotecas como `aioredis`, las operaciones de lectura/escritura se realizan sin bloquear el `event loop`, lo que maximiza la concurrencia cuando múltiples usuarios exploran configuraciones simultáneamente.

1.4.6. Almacenamiento de Archivos y Objetos

El almacenamiento persistente de archivos, versiones históricas de modelos y resultados derivados de las evaluaciones requiere una tecnología que ofrezca acceso distribuido, tolerancia a fallos, escalabilidad y compatibilidad con estándares industriales. Contar con una herramienta que permita aislar objetos de la base de datos es sumamente importante para evitar sobrecargar el sistema de gestión de datos relacional con información que no requiere consultas estructuradas, sino almacenamiento eficiente y recuperación rápida, por lo que PostgreSQL se encargaría de almacenar la dirección [Localizador Uniforme de Recursos \(URL\)](#) del objeto y sus metadatos; y en otro sistema se almacenaría el objeto como tal. Para satisfacer esta necesidad se incorporará MinIO, un sistema de almacenamiento de objetos compatible con la [API](#) de Amazon [Simple Storage Service \(S3\)](#), ampliamente adoptado en entornos de producción y servicios en la nube ([MinIO Documentation 2024](#)).

MinIO ofrece una interfaz uniforme para almacenar objetos binarios, documentos, *snapshots* de modelos y resultados asociados al proceso de validación. Su compatibilidad con [S3](#) permite integrar la plataforma con herramientas estándar sin requerir dependencias propietarias (Adler y Zhang, 2023). Esta característica asegura interoperabilidad con sistemas externos, posibilitando que módulos de análisis, auditoría o visualización operen sobre un *backend* de almacenamiento robusto y escalable.

En el contexto de la solución propuesta, MinIO será empleado para almacenar:

- *Snapshots* persistentes de versiones del [FM](#).
- Configuraciones válidas generadas por los motores [SAT/Satisfacibilidad Módulo Teorías \(SMT\)](#).
- Artefactos derivados del análisis estructural.
- Archivos de auditoría e informes.
- Registros de pruebas y diagnósticos.

A diferencia del almacenamiento en memoria provisto por Redis, MinIO garantiza persistencia a largo plazo, redundancia y recuperación confiable de información. Su arquitectura distribuida permite configurar réplicas, dispersión de datos y mecanismos de resiliencia sin sacrificar rendimiento (*ibíd.*). Además, sus

políticas de control de acceso simplifican la gestión de permisos, evitando inconsistencias o pérdida de trazabilidad a lo largo del ciclo de vida del currículo.

Desde el punto de vista operativo, MinIO permite integrarse con Docker y Docker Compose de manera directa, facilitando el despliegue, escalado y administración sin requerir configuración adicional. Esto habilita una solución completa de almacenamiento distribuido que puede operar tanto en entornos locales (*on-premise*) como en infraestructuras institucionales o nube privada.

1.4.7. Procesamiento Asíncrono

El procesamiento asíncrono permite ejecutar tareas en segundo plano sin bloquear la operación principal del sistema, lo cual es esencial cuando existen tareas costosas, como validaciones complejas, consultas externas, cálculos intensivos o envío de notificaciones. En el ecosistema de Python existen herramientas especializadas para administrar colas de trabajo, distribuir tareas y realizar ejecución diferida. Entre ellas, Celery se ha posicionado como la solución estándar para orquestar tareas asíncronas, ofreciendo escalabilidad, distribución y tolerancia a fallos. Su integración con brokers como Redis permite construir sistemas altamente eficientes, modulares y no bloqueantes.

En la arquitectura propuesta, Celery desempeñará un rol esencial como orquestador de tareas asíncronas, permitiendo que operaciones complejas, lentas o intensivas en cómputo se ejecuten fuera del ciclo de solicitud-respuesta del [API](#), mejorando sustancialmente la experiencia del usuario y la disponibilidad del sistema (Celery Project, 2025a).

Dado que el proceso de generación y validación de configuraciones curriculares incluye fases donde se deben:

- Recorrer árboles completos del [FM](#).
- Validar múltiples restricciones cruzadas.
- Propagar efectos lógicos por dependencias.
- Explorar configuraciones alternativas válidas.
- Verificar versiones y retrocompatibilidad.

Resulta natural que estas tareas no se ejecuten directamente en una petición [HTTP](#), ya que podrían bloquear el servidor o provocar tiempos de respuesta superiores a lo aceptable. Celery permite delegar estos procesos a “workers” paralelos que pueden ejecutarse en segundo plano, liberando a FastAPI para seguir atendiendo solicitudes sin congestión.

La integración de Celery con Redis (como *message broker*) permite que las tareas se publiquen, distribuyan, encolen, ejecuten y documenten de forma confiable. Asimismo, Redis funge como *backend* de resultados, permitiendo al sistema:

- Consultar el estado de una ejecución.
- Recuperar resultados parciales o finales.
- Cancelar o reprogramar tareas.

- Mostrar progresos al usuario.

Otra ventaja crítica es que Celery soporta escalamiento horizontal, lo cual es relevante para el sistema, en el cual múltiples usuarios pueden realizar evaluaciones simultáneamente sobre modelos distintos o versiones diferentes. Con más carga, simplemente se añaden más *workers*, sin necesidad de cambios estructurales (Celery Project, 2025a). Desde el punto de vista de robustez, Celery ofrece herramientas para:

- Reintentos automáticos ante fallos.
- Límite de concurrencia.
- Priorización de colas.
- Periodicidad (con Celery Beat) para ejecutar tareas programadas.

Esto permite automatizar procesos como:

- Generación periódica de *snapshots* de FM.
- Validaciones nocturnas de consistencia.
- Limpieza de configuraciones expiradas.
- Copias de seguridad de estados intermedios.

Celery es altamente compatible con FastAPI y Python, manteniendo coherencia tecnológica con la decisión de usar un *stack* asíncrono y un modelo de validación compleja (Celery Project, 2025a; Ramírez, 2019). Su adopción garantiza que la capa lógica pesada no degrade la utilidad interactiva del sistema, sino que se ejecute de forma paralela, programada y resistente. La tabla 1.4 respalda la justificación de por qué adicionar celery al sistema.

Cuadro 1.4. Tabla Comparativa: FastAPI vs FastAPI + Celery. Fuente: Elaboración propia.

Criterio	Solo FastAPI	FastAPI + Celery
Manejo de operaciones pesadas	Bloquea el servidor si la tarea tarda mucho	Delegación a <i>workers</i> externos sin bloqueo
Experiencia del usuario	Tiempos de respuesta largos para validaciones	Respuestas inmediatas + seguimiento del progreso
Escalabilidad	Limitada al número de <i>workers</i> del servidor	Escalamiento horizontal agregando <i>workers</i> Celery
Naturaleza de carga	Orientada a I/O y sincronización	Orientada a cómputo intensivo y tareas diferidas
Validación de modelos complejos	Puede agotar recursos del servidor	Optimización mediante <i>workers</i> paralelos
Generación de <i>snapshots</i> o versiones	Bajo rendimiento si se ejecuta en el <i>request</i>	Tareas programadas y automáticas (Celery Beat)

Continúa en la siguiente página

Cuadro 1.4 – Continuación de la página anterior.

Criterio	Solo FastAPI	FastAPI + Celery
Robustez ante fallos	Fallos interrumpen solicitudes activas	Reintentos automáticos e independientes
Concurrencia	Limitada	Altamente concurrente
Distribución de carga	Manual, difícil de gestionar	Balance natural mediante colas y <i>brokers</i>
Estado de procesos largos	No persistente, no rastreable	Persistente, rastreable, recuperable
Integración con Redis	Opcional	Ideal, Redis como <i>broker</i> y <i>backend</i>
Mantenimiento	Monolítico	Arquitectura desacoplada y modular

Esta comparación evidencia que mientras FastAPI permite construir servicios web altamente eficientes para operaciones sin bloqueo y validaciones ligeras, la incorporación de Celery habilita un procesamiento distribuido ideal para la naturaleza intensiva de la manipulación y validación de **FM**s, generando configuraciones, *snapshots* y cálculos alternativos sin afectar la disponibilidad del sistema.

1.4.8. Algoritmos, Motor de Reglas y Validación

Para garantizar la consistencia, completitud y validez de las configuraciones generadas a partir del **FM**, la plataforma integrará un conjunto de algoritmos especializados, combinando enfoques deterministas y heurísticos según la naturaleza del problema a resolver.

En primer lugar, se utilizará SymPy como motor base para la representación simbólica y evaluación lógica de restricciones asociadas al **FM** (Meurer, 2017). SymPy permite manipular expresiones booleanas, aplicar simplificaciones, detectar contradicciones y verificar satisfacibilidad, lo que resulta esencial para la evaluación de relaciones *cross-tree* como *requires*, *excludes*, *implies* y restricciones de cardinalidad. Su uso asegura una verificación formal, basada en lógica proposicional y álgebra simbólica, manteniendo precisión y trazabilidad en los análisis (*ibíd.*).

De manera complementaria, la plataforma empleará NetworkX para modelar y procesar las dependencias entre características como grafos dirigidos, permitiendo ejecutar algoritmos de recorrido, detección de ciclos, análisis de conectividad, propagación de condiciones y determinación de componentes fuertemente conexas (Hagberg; Schult y Swart, 2008). Este enfoque orientado a grafos habilita la inspección estructural del **FM**, la detección temprana de inconsistencias y la aplicación eficiente de validaciones transversales.

En escenarios donde la validación determinista no sea suficiente para explorar el espacio de configuraciones posibles —especialmente cuando existan alternativas múltiples y restricciones complejas— se utilizarán técnicas heurísticas como algoritmos genéticos. Estos permiten buscar soluciones aproximadas optimizando criterios como completitud, compatibilidad y minimización de violaciones. Su empleo resulta valioso para sugerir configuraciones válidas, proponer versiones corregidas y explorar variantes del **FM** bajo metas específicas.

Si bien el sistema incorpora herramientas como SymPy para análisis simbólico, NetworkX para la evaluación estructural del modelo y algoritmos genéticos para la exploración heurística del espacio de soluciones, estas tecnologías no sustituyen un motor formal de satisfacibilidad. Por su naturaleza, SymPy resulta adecuado para verificar restricciones locales o detectar inconsistencias obvias, pero no está diseñado para resolver problemas de variabilidad a gran escala. De igual forma, los algoritmos evolutivos permiten sugerir configuraciones o completar decisiones, aunque sus resultados son aproximados. Por ello, la plataforma complementa este enfoque híbrido con motores lógicos especializados basados en SAT/SMT para garantizar corrección formal y escalabilidad en los análisis.

Para validar y generar configuraciones curriculares consistentes, se adoptó por un motor lógico basado en técnicas SAT y resolución de restricciones (CSP). Estas técnicas son exactas, determinísticas y ampliamente empleadas en entornos SPL industriales. Librerías como PySAT (python-sat) y Z3 Solver permiten representar cardinalidades, dependencias, exclusiones y restricciones complejas entre características, garantizando un análisis consistente independientemente del tamaño del modelo.

Para la validación formal del FM se hará uso de PySAT, un *toolkit* industrial que integra múltiples solucionadores SAT de alto rendimiento (MiniSAT, Glucose, Lingeling, Maple, entre otros). PySAT permite formular restricciones curriculares como cláusulas booleanas y delegar su resolución a técnicas probadas a nivel de investigación y producción (H. Zhao; Liffiton; Eijk; Malikov y Previti, 2018). Su uso mejora la escalabilidad del sistema, ya que permite manejar modelos con miles de dependencias de forma eficiente, además de habilitar análisis avanzados como detección de dependencias redundantes, pruebas de satisfacibilidad incremental y extracción de explicaciones (UNSAT cores), funcionalidad útil para asistir al usuario en la corrección de configuraciones inválidas.

Los solucionadores SAT-based han demostrado ser una técnica altamente efectiva en SPL para la verificación automática de configuraciones (D. Benavides; S. Segura y A. Ruiz-Cortés, 2010). Su aplicación consiste en reducir el FM a una fórmula booleana y comprobar su satisfacibilidad mediante algoritmos especializados. Este enfoque es determinista, completo y garantiza la detección de inconsistencias globales, no sólo locales. A diferencia de técnicas heurísticas, los solucionadores SAT permiten decisiones exactas, prueba de imposibilidad, generación de configuraciones alternativas, evaluación de cardinalidades y análisis de impacto ante cambios, aportando rigor matemático y soporte computacional sólido para la variabilidad curricular.

Mientras los solucionadores SAT determinan si una configuración es válida o no, el uso de técnicas Max-SAT permite optimizar soluciones, seleccionar variantes preferidas y resolver conflictos minimizando el número de violaciones. El solucionador Z3, desarrollado por Microsoft Research, soporta Max-SAT, SMT y razonamiento híbrido, lo que lo convierte en una herramienta idónea para el análisis curricular (De Moura y Bjørner, 2008). Esto permite no sólo verificar configuraciones, sino sugerir alternativas cercanas a las decisiones originales del usuario, resolver conflictos automáticamente y explorar escenarios de “mejor esfuerzo” cuando no existe una solución exacta. En términos prácticos, Max-SAT habilita un sistema inteligente que auxilia al gestor curricular en decisiones complejas, proporcionando soluciones justificadas y transparentes.

1.4.9. Gestor de dependencias

Para la gestión eficiente de dependencias, aislamiento de entornos y versionado reproducible de bibliotecas, la plataforma adoptará Poetry y UV como gestores complementarios (authors, 2024). La combinación de Poetry y UV permite una gestión reproducible y eficiente de dependencias, alineada con las necesidades de mantenibilidad, escalabilidad y automatización del sistema (authors, 2024; Marsh, 2024).

Poetry permite mantener un control declarativo de las dependencias del proyecto mediante archivos `pyproject.toml`, garantizando la trazabilidad del *stack* tecnológico y la compatibilidad de versiones. Su mecanismo de resolución de dependencias asegura consistencia en los entornos de desarrollo y producción, reduciendo incidencias asociadas a incompatibilidades entre bibliotecas (authors, 2024). Además, Poetry facilita la generación de entornos virtuales aislados, el bloqueo de versiones exactas (`poetry.lock`), la publicación del proyecto como paquete y la automatización de *scripts* relacionados con el ciclo de vida del sistema.

En conjunto con Poetry, se utilizará UV como herramienta moderna para instalación ultrarrápida de dependencias y creación de entornos (Marsh, 2024). UV destaca por su rendimiento de orden de magnitud superior a herramientas tradicionales, lo cual mejora tiempos de despliegue, entornos CI/CD y operaciones de reconstrucción.

1.4.10. Herramienta para el control de versiones

Los sistemas de control de versiones más populares son Git, Subversion y Mercurial. Se eligió Git debido a que es un sistema de control de versiones distribuido utilizado en el desarrollo de software que rastrea las modificaciones realizadas en el código fuente. Git, a diferencia de los sistemas de control de versiones centralizados, permite que varios desarrolladores colaboren de forma independiente en un solo proyecto al permitirles mantener repositorios locales separados que se sincronizan con un repositorio remoto. Los repositorios, que contienen los archivos del proyecto y el historial de cambios, son esenciales para el funcionamiento de Git (Ponuthorai y J. Loeliger, 2022). Una confirmación en un repositorio es una instantánea del proyecto en ese momento específico; cada confirmación se distingue por un *hash* distinto y va acompañada de un mensaje de explicación. Las ramas permiten a los desarrolladores trabajar de forma independiente en varias funciones o correcciones (Jon Loeliger y McCullough, 2012).

Mientras que otras ramas se utilizan para el desarrollo, el código estable generalmente se encuentra en la rama principal, también conocida como `main`. La reorganización y la combinación son dos formas en que los cambios de las ramas se pueden combinar en la rama principal, cada una con una función distinta para preservar el historial del proyecto. Se crea una copia local de un repositorio clonándolo, y los cambios locales se sincronizan con un repositorio remoto mediante la inserción y la extracción (Ghodke y Chavan, 2024). El área de preparación sirve como zona de influencia donde se realizan modificaciones antes de la confirmación. Los submódulos, `cherry-pick` y `Git hooks` son ejemplos de funciones avanzadas que proporcionan más personalización y control. En resumen, Git permite llevar un historial exhaustivo de cambios, ramificación para desarrollo de nuevas características, fusiones controladas, auditoría de modificaciones y recuperación

de estados previos del sistema. Los conocidos servicios de alojamiento de Git, como GitHub, GitLab y Bitbucket, ofrecen plataformas para la revisión de código, la gestión de repositorios y la integración de herramientas, que promueven el desarrollo colaborativo. El uso de Git mejora los procesos de desarrollo al facilitar el trabajo en equipo y la gestión de código efectivos (Ponuthorai y J. Loeliger, 2022).

GitHub es una plataforma de alojamiento de código basada en la web que utiliza Git como tecnología subyacente para el control de versiones. Actúa como un servicio de repositorio remoto centralizado, ampliando las capacidades de Git con herramientas de colaboración, gestión de proyectos y características sociales. Como una de las plataformas más populares para el desarrollo de software colaborativo, GitHub permite a los equipos alojar repositorios públicos y privados, facilitando la revisión de código mediante *pull requests*, el seguimiento de problemas con *issues* y la automatización de flujos de trabajo mediante GitHub Actions. Su interfaz gráfica y sus herramientas de gestión hacen que las operaciones de Git, como la fusión de ramas y la resolución de conflictos, sean más accesibles para los desarrolladores, al tiempo que proporcionan integraciones robustas con herramientas de *CI/CD* y servicios de terceros. GitHub también fomenta la comunidad *open-source* mediante funciones como la bifurcación (*forking*) de repositorios y las contribuciones externas, consolidándose como un entorno integral para el ciclo de vida del desarrollo de software (Ghodke y Chavan, 2024; G. Inc., 2024).

1.4.11. Lenguaje de modelado

El lenguaje de modelado se refiere a un lenguaje formal utilizado para crear modelos que representan sistemas, procesos o estructuras de datos. Estos lenguajes proporcionan un conjunto de reglas sintácticas y semánticas que facilitan la comprensión de fenómenos complejos de manera estructurada (Rumbaugh; Jacobson y Booch, 2004).

El lenguaje de modelado a utilizar es el **Lenguaje de Modelado Unificado (UML)**, este lenguaje ofrece un estándar para describir un plano del sistema, incluyendo aspectos conceptuales tales como procesos de negocios, funciones del sistema, expresiones de lenguajes de programación, esquemas de bases de datos y componentes reutilizables. Este lenguaje de modelo es el seleccionado para especificar o describir métodos o procesos, definir el sistema y sus artefactos para la documentación y construcción de la aplicación web a desarrollar (*Unified Modeling Language (UML) Specification* 2017).

1.4.12. Herramienta CASE

Una herramienta de **Ingeniería de Software Asistida por Computadora (CASE)** es un programa informático destinado a aumentar la productividad durante todo el proceso de desarrollo de software reduciendo el coste de este en términos de tiempo y de dinero (Pressman, 2014). Según Sommerville, estas herramientas nos pueden ayudar en todos los aspectos del ciclo de vida de desarrollo del software en tareas como el diseño de proyectos, cálculo de costes, implementación de parte del código automáticamente con el diseño dado, compilación automática, documentación o detección de errores, automatizar actividades de análisis, mejorando así la productividad y calidad del software. Para el diseño de la solución se empleará la herramienta

CASE Visual Paradigm en su versión 16.2 (I. Sommerville, 2016).

La herramienta Visual Paradigm es reconocida mundialmente por sus soluciones de software para la transformación de negocio. Propicia un conjunto de ayudas para el desarrollo de programas informáticos, desde la planificación, pasando por el análisis y el diseño, hasta la generación de código. Soporta los principales estándares de la industria tales como el UML, la notación Notación de Modelado de Procesos de Negocio (BPMN) y un sinfín de diagramas. Por lo que Visual Paradigm es una herramienta CASE integral y flexible para el modelo y gestión en el desarrollo de software, procesos de negocio y arquitectura empresarial (Ltd., 2024).

1.4.13. Herramientas para diseñar y ejecutar pruebas

Para garantizar la calidad y correcto funcionamiento del sistema propuesto, se emplearán herramientas especializadas tanto para pruebas de API como para pruebas unitarias del código.

Postman será utilizado como plataforma de desarrollo y pruebas de APIs (P. Inc., 2024). Esta herramienta permite diseñar, documentar y probar *endpoints* RESTful de forma interactiva, facilitando la verificación de contratos de datos, validación de respuestas, gestión de variables de entorno y automatización de colecciones de pruebas. Postman resulta esencial para validar el comportamiento del *backend* FastAPI, asegurando que las operaciones de generación y validación de configuraciones curriculares respondan correctamente bajo distintos escenarios (*ibíd.*).

Complementariamente, se utilizará Pytest como *framework* para la ejecución de pruebas unitarias en Python (Krekel y contributors, 2024). Pytest destaca por su sintaxis clara, capacidad de parametrización, gestión de *fixtures* y generación de reportes detallados, lo que facilita la escritura y mantenimiento de pruebas automatizadas. Su integración con el ecosistema Python permite validar lógica de negocio, algoritmos de validación, operaciones sobre modelos de características y comportamiento de componentes aislados del sistema (*ibíd.*).

La combinación de Postman para pruebas de integración a nivel de API y Pytest para pruebas unitarias proporciona una cobertura completa del sistema, garantizando robustez tanto en la interfaz externa como en la lógica interna.

1.4.14. Entorno Integrado de Desarrollo

Los Entorno de Desarrollo Integrado (IDE) son las herramientas que todo desarrollador debe tener a mano. Permiten editar y gestionar código fuente en diversos lenguajes de programación y ofrecen múltiples herramientas para facilitar el trabajo y aumentar la productividad, pues permiten depurar, resaltar sintaxis, autocompletado, integración con control de versiones y la refactorización. Por lo general cada IDE está asociado a un único lenguaje de programación o a un conjunto pequeño de lenguajes, cuyo factor provoca que la gran mayoría de desarrolladores opten por otras alternativas, ya que en proyectos donde el uso de múltiples lenguajes es evidente, puede ser un obstáculo (IBM Corporation, 2023).

Se escogió como entorno de desarrollo Visual Studio Code (Corporation, 2025a), cuya herramienta es no convencional pero muy usada por muchos programadores, siendo señalado como el editor de código más popular entre proyectos de código abierto (*open source*) (Ramel, 2021). El creador del editor de código multiplataforma, VSCode, es Microsoft. Este software es gratuito y de código abierto. Incluye soporte para la depuración, control integrado de Git, resaltado de sintaxis, finalización inteligente de código, fragmentos y refactorización de código. Además, su nivel de personalización permite adaptar el entorno a las preferencias de cada programador mediante temas, atajos y configuraciones avanzadas (Corporation, 2025b).

VSCode presenta extensiones importantísimas que permite adaptar el entorno de desarrollo para una enorme infinidad de tareas, tecnologías y lenguajes. Este conjunto, prácticamente ilimitado, es totalmente accesible dentro del propio editor. Tiene soporte integrado para aplicaciones web; por lo tanto, se puede llevar a cabo en este entorno de trabajo la codificación de la aplicación a desarrollar con las tecnologías FastAPI y Python. VSCode es considerado por brindar eficiencia y agilidad en la programación, pues funciona muy bien en incluso equipos con recursos limitados; además, los desarrolladores lo aprecian porque su interfaz de usuario es muy intuitiva y permite comenzar, prácticamente, a trabajar sin necesidad de alguna experiencia (Corporation, 2025a; Corporation, 2025b).

1.5. Metodología de desarrollo del software

Las metodologías de desarrollo de software son enfoques estructurados que guían las distintas fases del ciclo de vida de un proyecto, desde la planificación y análisis hasta la implementación y mantenimiento. Estas metodologías establecen principios, prácticas y procedimientos para gestionar el desarrollo de manera organizada y eficiente, promoviendo la colaboración en equipo y la calidad del producto final (Pacienza, 2015).

A lo largo de esta sección se realiza un análisis de las metodologías de software disponibles y, finalmente, se detallan los principios, fases y prácticas de la metodología seleccionada para guiar el diseño e implementación del sistema propuesto. La elección de la metodología es fundamental para asegurar un proceso de desarrollo ágil, eficiente y adaptable a posibles cambios en los requisitos del proyecto. Estas metodologías facilitan además la detección temprana de errores y el ajuste progresivo de funcionalidades, garantizando la calidad final del sistema.

En la actualidad se pueden diferenciar dos grandes grupos de metodologías de desarrollo de software: las ágiles y las tradicionales. A continuación, se explican las características de cada una de ellas.

1.5.1. Metodologías de desarrollo de software tradicionales

Las metodologías de desarrollo de software tradicionales se caracterizan por definir total y rígidamente los requisitos al inicio de los proyectos de ingeniería de software. Los ciclos de desarrollo son poco flexibles y no permiten realizar cambios, al contrario que las metodologías ágiles; lo que ha propiciado el incremento en el uso de las segundas.

La organización del trabajo de las metodologías tradicionales es lineal, es decir, las etapas se suceden una tras otra y no se puede empezar la siguiente sin terminar la anterior. Tampoco se puede volver hacia atrás una vez se ha cambiado de etapa. Estas metodologías, no se adaptan nada bien a los cambios, y el mundo actual cambia constantemente (Awad, 2005). Las principales metodologías tradicionales o clásicas son:

- **Modelo en Cascada:** también conocido como el ciclo de vida clásico del software, es una de las metodologías de desarrollo de software más antiguas y ampliamente utilizadas. Este enfoque se basa en la idea de que el desarrollo de software debe seguir una secuencia lineal y predefinida de etapas de riguroso orden, cada una de las cuales debe completarse antes de pasar a la siguiente. Los requisitos y especificaciones iniciales no están predispuestos para cambiarse.
- **Incremental:** en esta metodología de desarrollo de software se va construyendo el producto final de manera progresiva. En cada etapa incremental se agrega una nueva funcionalidad, lo que permite ver resultados de una forma más rápida en comparación con el modelo en cascada. El software se puede empezar a utilizar incluso antes de que se complete totalmente y, en general, es mucho más flexible que las demás metodologías.
- **Espiral:** es una combinación de los dos modelos anteriores, que añade el concepto de análisis de riesgo. Se divide en cuatro etapas: planificación, análisis de riesgo, desarrollo de prototipo y evaluación del cliente. El nombre de esta metodología da nombre a su funcionamiento, ya que se van procesando las etapas en forma de espiral. Cuanto más cerca del centro se está, más avanzado está el proyecto.

1.5.2. Metodologías de desarrollo de software ágiles

Hoy en día las metodologías ágiles de desarrollo de software son las más utilizadas debido a su alta flexibilidad y agilidad. Los equipos de trabajo que las utilizan son mucho más productivos y eficientes, ya que saben lo que tienen que hacer en cada momento. Además, la metodología permite adaptar el software a las necesidades que van surgiendo por el camino, lo que facilita construir aplicaciones más funcionales.

Las metodologías ágiles se basan en la metodología incremental, en la que en cada ciclo de desarrollo se van agregando nuevas funcionalidades a la aplicación final. Sin embargo, los ciclos son mucho más cortos y rápidos, por lo que se van agregando pequeñas funcionalidades en lugar de grandes cambios.

Este tipo de metodologías permite construir equipos de trabajo autosuficientes e independientes que se reúnen cada poco tiempo para poner en común las novedades. Poco a poco, se va construyendo y puliendo el producto final, a la vez que el cliente puede ir aportando nuevos requerimientos o correcciones, ya que puede comprobar cómo avanza el proyecto en tiempo real (Pacienza, 2015). Las principales metodologías ágiles son:

- **Kanban:** metodología de trabajo inventada por la empresa de automóviles Toyota. Consiste en dividir las tareas en porciones mínimas y organizarlas en un tablero de trabajo dividido en tareas pendientes, en curso y finalizadas. De esta forma, se crea un flujo de trabajo muy visual basado en tareas prioritarias e incrementando el valor del producto.

- Scrum: es también una metodología incremental que divide los requisitos y tareas de forma similar a Kanban. Se itera sobre bloques de tiempos cortos y fijos (entre dos y cuatro semanas) para conseguir un resultado completo en cada iteración. Las etapas son: planificación de la iteración (*planning sprint*), ejecución (*sprint*), reunión diaria (*daily meeting*) y demostración de resultados (*sprint review*). Cada iteración por estas etapas se denomina también *sprint*.
- **Extreme Programming (XP)**: es una metodología de desarrollo de software basada en las relaciones interpersonales, que se consideran la clave del éxito. Su principal objetivo es crear un buen ambiente de trabajo en equipo y que haya un *feedback* constante del cliente. El trabajo se basa en 12 conceptos: diseño sencillo, *testing*, refactorización y codificación con estándares, propiedad colectiva del código, programación en parejas, integración continua, entregas semanales e integridad con el cliente, *cliente in situ*, entregas frecuentes y planificación.

1.5.3. Elección de la metodología

En las metodologías tradicionales el principal problema es que nunca se logra planificar bien el esfuerzo requerido para seguir la metodología. Pero entonces, si se logran definir métricas que apoyen la estimación de las actividades de desarrollo, muchas prácticas de metodologías tradicionales podrían ser apropiadas. El no poder predecir siempre los resultados de cada proceso no significa que se esté frente a una disciplina de azar. Lo que significa es que se está frente a la necesidad de adaptación de los procesos de desarrollo que son llevados por parte de los equipos que desarrollan software (Awad, 2005).

Tener metodologías diferentes para aplicar de acuerdo con el proyecto que se desarrolle resulta una idea interesante. Estas metodologías pueden involucrar prácticas tanto de metodologías ágiles como de metodologías tradicionales. De esta manera se podría tener una metodología por cada proyecto, la problemática sería definir cada una de las prácticas, y en el momento preciso definir parámetros para saber cuál usar. Es importante tener en cuenta que el uso de un método ágil no vale para cualquier proyecto. Sin embargo, una de las principales ventajas de los métodos ágiles es su peso inicialmente ligero y por eso las personas que no estén acostumbradas a seguir procesos encuentran estas metodologías bastante agradables. La tabla 1.5 explica de forma resumida las diferencias que existen entre las metodologías ágiles y las tradicionales.

Cuadro 1.5. Comparación, Metodología Ágil VS Metodología Tradicional (ibíd.).

Parámetros	Metodologías Ágiles	Metodologías Tradicionales
Acercamiento	Adaptivo	Predictivo
Medición del objetivo	Valor para el negocio	Conformación del plan
Tamaño del proyecto	Pequeño	Largo
Tipo de administración	Descentralizado	Autocrático
Perspectiva de cambios	Adaptable a cambio	Sostenible al cambio
Cultura del equipo	Liderazgo\Colaboración	Comandada\Controlada
Documentación	Baja	Pesada

Parámetros	Metodologías Ágiles	Metodologías Tradicionales
Orientada	Personas	Procesos
Ciclos de desarrollo	Numerosos	Limitados
Dominio de desarrollo	Impredecibles	Predecibles
Planificación inicial	Mínima	Comprehensiva
Retorno de la inversión	Temprano en el proyecto	Al final del proyecto
Tamaño del equipo	Pequeño	Grande

Para la selección de la metodología a utilizar se toma como referencia la tabla 1.5. Para ello se analizan las características del proyecto atendiendo a los diferentes aspectos y se selecciona la metodología con la que haya más coincidencias, dando como resultado de este análisis la tabla 1.6.

Cuadro 1.6. Comparación de la Metodología Ágil y la Metodología Tradicional con respecto al proyecto. Fuente: Elaboración propia.

Parámetros	Ágiles	Pesadas
Acercamiento	X	
Medición del objetivo	X	
Tamaño del proyecto	X	
Tipo de administración	X	
Perspectiva de cambios	X	
Cultura del equipo	X	
Documentación	X	
Orientada	X	
Ciclos de desarrollo	X	
Dominio de desarrollo	X	
Planificación inicial	X	
Retorno de la inversión		X
Tamaño del equipo	X	
Total:	12	1

Como resultado se obtiene que lo mejor es utilizar una metodología ágil. Ya que, según el análisis realizado, se acomoda mejor a las necesidades y características del proceso a llevar a cabo. Para decidir qué metodología ágil se utilizará, se realizó una comparación entre algunas de las metodologías ágiles más comúnmente utilizadas, la cual se refleja en la tabla 1.7.

Cuadro 1.7. Comparación de Metodología Ágiles. Fuente: Elaboración propia.

Parámetros	XP	Scrum
Duración	1 a 2 semanas	2 semanas a 1 mes
Cambios	Sensibles al cambio	No permite cambios

Parámetros	XP	Scrum
Prioridad	Estricta, priorizada por el cliente	Determinada por el equipo
Prácticas de ingeniería	Prescribe prácticas (pruebas, programación en parejas)	No prescribe prácticas
Tamaño de los proyectos	Pequeños y medianos	Pequeños, medianos y grandes
Tamaño de equipo	Menor que 10	Múltiples equipos menores que 10

Teniendo en cuenta la tabla 1.7 se decide utilizar la metodología XP. Esto se debe a que es muy eficiente durante el proceso de pruebas y planificación, su tasa de error es muy pequeña, facilita los cambios, origina una programación muy organizada, además de fomentar la comunicación entre los desarrolladores y los clientes. También se puede aplicar a cualquier lenguaje de programación, el cliente tiene control sobre las prioridades, durante todo el proyecto se pueden realizar diversas pruebas y, sobre todo, permite ahorrar mucho tiempo y dinero.

1.5.4. Programación Extrema

XP es una metodología ágil de desarrollo de software con bases en la comunicación constante y la retroalimentación. Uno de sus fines principales es el de construir un producto que vaya en línea con los requerimientos del cliente. En ese sentido es adaptable a los cambios, generando una rápida respuesta frente a cualquier inconveniente. Por otro lado, el equipo de trabajo tiene la ventaja de potenciar sus relaciones, ya que el proceso que de este se desprende es abierto, conjunto y de aprendizaje continuo (Pressman, 2014).

Las 4 etapas que sigue XP son:

- Planificación: toma como referencia la identificación de las Historias de Usuario (HU) con pequeñas versiones que se irán revisando en periodos cortos con el fin de obtener un software funcional.
- Diseño: trabaja el código orientado a objetivos y, sobre todo, usando los recursos necesarios para que funcione.
- Codificación: se refiere al proceso de programación organizada en parejas, estandarizada y que resulte en un código universal entendible.
- Pruebas: consiste en un testeo automático y continuo en el que el cliente tiene voz para validar y proponer.

Los principios de XP comprenden diez buenas prácticas que involucran al equipo de trabajo, los procesos y el cliente (ibíd.); los cuales son:

- Planificación incremental: Se toman los requerimientos en HU, las cuales son negociadas progresivamente con el cliente.
- Entregas pequeñas: Se desarrolla primero la más mínima parte útil que le proporcione funcionalidad al sistema, y poco a poco se efectúan incrementos que añaden funcionalidad a la primera entrega, cada ciclo termina con una entrega del sistema.

- Diseño sencillo: Solo se efectúa el diseño necesario para cumplir con los requerimientos actuales, es decir, no se abordan requerimientos futuros.
- Desarrollo previamente aprobado: Una de las características relevantes y propias de XP es que primero se escriben las pruebas y luego se da la codificación, esto con la finalidad de asegurar la satisfacción del requerimiento.
- Limpieza del código o refactorización: Consiste en simplificar y optimizar el programa sin perder funcionalidad, es decir, alterar su estructura interna sin afectar su comportamiento externo.
- Programación en parejas: Es otra de las características de esta metodología, que propone que los desarrolladores trabajen en parejas en una terminal, verificando cada uno el trabajo del otro y ayudándose para buscar las mejores soluciones. Se entiende que de esta forma el trabajo será más eficiente y de mayor calidad.
- Propiedad colectiva: El conocimiento y la información deben ser de todos, por lo tanto no se desarrollan islas de conocimiento, todos los programadores poseen todo el código y cualquiera puede sugerir y realizar mejoras.
- Integración continua: Al terminar una tarea, esta se integra al sistema entero y se realizan pruebas de unidad a todo el sistema, esta práctica permite que la aplicación sea más funcional en cada iteración y garantiza su funcionamiento con los demás módulos del sistema.
- Ritmo sostenible: No es aceptable trabajar durante grandes cantidades de horas ya que se considera que puede reducir la calidad del código y la productividad del equipo a mediano plazo, se sugieren 40 horas semanales.
- Cliente presente: Se debe tener un representante (cliente o usuario final) a tiempo completo, ya que en XP este hace parte del equipo de desarrollo y es responsable de formular los requerimientos para el desarrollo del sistema.

Artefactos

Las metodologías utilizan artefactos para describir las actividades o procedimientos que se deben seguir durante su desarrollo. A continuación, se describen los artefactos de XP:

- HU: Representan una breve descripción del comportamiento del sistema. Emplea terminología del cliente sin lenguaje técnico. Se realiza una por cada característica principal del sistema. Se emplean para hacer estimaciones de tiempo y para el plan de lanzamientos. Reemplazan un gran documento de requisitos y presiden la creación de las pruebas de aceptación. Estas deben proporcionar sólo el detalle suficiente como para poder hacer razonable la estimación de cuánto tiempo requiere la implementación de la HU, difiere de los casos de uso porque son escritos por el cliente, no por los programadores, empleando terminología del cliente (Pressman, 2014).
- Tareas de Ingeniería: Una HU se descompone en varias tareas de ingeniería, que describen las actividades que se realizarán en cada una de ellas. Así mismo, las tareas de ingeniería se vinculan más al desarrollador, ya que permite tener un acercamiento con el código (ibíd.).

- Tarjetas **Clase, Responsabilidad y Colaboración (CRC)**: Estas tarjetas se dividen en tres secciones que contienen la información del nombre de la clase, sus responsabilidades y sus colaboradores. Una clase es cualquier persona, cosa, evento, concepto, pantalla o reporte. Las responsabilidades de una clase son las cosas que conoce y las que realiza, sus atributos y métodos. Los colaboradores son las demás clases con las que trabaja en conjunto para llevar a cabo sus responsabilidades (Pressman, 2014).

En síntesis, actualmente **XP** es una de las metodologías de mayor aceptación en la industria del software. Su enfoque basado en los métodos ágiles, su énfasis en la gestión del recurso humano, el cual es uno de los puntos más críticos en todo proyecto, y sus principios de previsibilidad y adaptabilidad hacen de esta metodología una buena opción a seguir.

1.6. Conclusiones del capítulo

En el desarrollo del capítulo se obtuvo una mejor visión acerca del problema planteado y se dieron cumplimiento a las primeras tareas de investigación llegando a las siguientes conclusiones:

- El análisis de los conceptos asociados a la solución permitió un acercamiento a los temas relacionados con la **SPLE**, **FM** y el **UVL**, estableciendo una base teórica sólida que fundamenta la propuesta. La analogía formal desarrollada entre estos conceptos y el dominio educativo demostró la viabilidad de aplicar técnicas de variabilidad a la gestión curricular, abriendo nuevas posibilidades para la flexibilización controlada de planes de estudio.
- El estudio de los sistemas homólogos relacionados con el marco de la investigación, permitió identificar una brecha significativa en el mercado: mientras existen herramientas maduras para la gestión administrativa académica (SIGA, Banner, Moodle) y potentes *frameworks* de modelado de variabilidad (pure::variants, FeatureIDE, Flamapy), ninguna de ellas integra de manera natural la capacidad de modelar, validar y configurar currículos educativos con un enfoque accesible para gestores académicos no especializados en ingeniería de software. Esta constatación justifica la necesidad y pertinencia de la solución propuesta.
- La selección tecnológica resultante, respaldada por un análisis comparativo riguroso, define las bases para el diseño e implementación satisfactoria de la aplicación a desarrollar, garantizando rendimiento, escalabilidad, mantenibilidad y alineación con los requisitos funcionales del sistema.
- El proyecto tiene las características de tener un tiempo de desarrollo corto y la necesidad de realizar entregas constantes en ciclos cortos, por lo que es necesario el uso de una metodología ágil. Luego de un análisis comparativo entre metodologías tradicionales y ágiles, se decide utilizar la metodología **XP**, permitiendo esto cierta flexibilidad a lo largo del desarrollo de la aplicación, una interacción constante con el usuario, la incorporación de cambios durante el proceso, y la garantía de calidad mediante prácticas como programación en parejas, desarrollo dirigido por pruebas e integración continua.

Análisis y diseño de la propuesta de solución

El presente capítulo constituye el núcleo de la fase de análisis y diseño de la investigación, traduciendo los fundamentos teóricos establecidos en el capítulo anterior en una propuesta técnica concreta y viable. Partiendo del diagnóstico de la problemática y del estudio de sistemas homólogos, se procede a definir de manera formal y estructurada la solución de software que dará respuesta al objetivo general de la investigación.

El capítulo se inicia con la propuesta de solución, que describe a alto nivel el sistema a desarrollar. A continuación, se realiza un levantamiento detallado de requisitos, clasificados en funcionales y no funcionales, para precisar el comportamiento y las cualidades del sistema. Estas especificaciones se refinan a través de [HU](#), las cuales, junto con su estimación de esfuerzo, sirven de base para el desarrollo del plan de iteraciones que guiará la implementación. Posteriormente, el diseño orientado a objetos se plasma mediante tarjetas [CRC](#), sentando las bases para la definición de la arquitectura de software basada en el patrón arquitectónico por capas. Finalmente, se especifican los [Patrones de Software Generales para la Asignación de Responsabilidades \(GRASP\)](#) y los patrones de diseño [GoF](#), que se aplicarán para resolver problemas comunes en el desarrollo de software y otros ámbitos referentes al diseño.

2.1. Descripción de la propuesta de solución

En función de dar cumplimiento al objetivo planteado y ajustándose a las necesidades presentes, se propone desarrollar un servicio *backend* que, basado en los principios de [SPLE](#) y la técnica de modelado [FM](#), permita a los profesionales de la educación representar y gestionar la variabilidad curricular de forma sistemática utilizando [UML](#) como lenguaje de modelado u operando directamente sobre la representación gráfica. Esta solución permitirá representar formalmente planes de estudio, definir variabilidad entre módulos, establecer dependencias, restricciones y rutas formativas alternativas, así como derivar configuraciones válidas que representen itinerarios personalizados. En esencia, el sistema habilita una ingeniería curricular basada en la lógica [SPLE](#), donde los planes de estudio se conciben como familias de productos y cada configuración curricular corresponde a un producto derivado.

El servicio abordará todas las fases necesarias para este proceso: diseño del modelo curricular (equivalente al análisis de dominio), definición de restricciones y cardinalidades (gestión de variabilidad), validación automatizada de configuraciones (ensamblaje de productos) y versionamiento del modelo (gestión de activos). Para ello se implementará un motor central que permita representar jerarquías de características, aplicar reglas transversales, procesar requerimientos, exclusiones e implicaciones, y generar configuraciones consistentes, detectando violaciones y proponiendo alternativas.

El sistema incorporará funcionalidades que responden directamente a los retos del dominio educativo: evitar inconsistencias curriculares, detectar prerrequisitos no satisfechos, prevenir combinaciones inviables, explorar rutas posibles, permitir adaptaciones por niveles, perfiles o competencias, y almacenar *snapshots* versionados que faciliten la evolución del plan de estudios sin invalidar configuraciones previas. Este enfoque no se limita a documentar un currículo, sino que lo convierte en un modelo computable y verificable, habilitando su reutilización y variación controlada.

2.1.1. Sistemas de roles estáticos

Un sistema de roles estáticos constituye un mecanismo de control de acceso y organización funcional, en el cual los permisos, responsabilidades y alcances de cada usuario se definen de manera fija durante la fase de diseño del sistema y permanecen invariantes a lo largo del tiempo, salvo intervención explícita de un administrador (Ferraiolo; Kuhn y Chandramouli, 2007; Sandhu; Coyne; Feinstein y Youman, 1996). Este paradigma, ampliamente adoptado en sistemas empresariales y académicos, asigna a cada usuario uno o varios roles predefinidos, y cada rol posee un conjunto determinado de privilegios sobre los recursos y operaciones del sistema.

La principal característica de los sistemas de roles estáticos es su predictibilidad y estabilidad: las acciones que un usuario puede realizar están claramente delimitadas por el rol que se le ha asignado, sin que exista una lógica dinámica que modifique estos permisos en tiempo de ejecución en función del contexto, el estado del sistema o el historial de acciones del usuario (V. C. Hu; Ferraiolo y Kuhn, 2006). Esta rigidez, que podría considerarse una limitación en entornos altamente dinámicos, se convierte en una ventaja en dominios donde la separación de responsabilidades debe ser clara, auditable y fácil de comprender por los propios usuarios.

En el contexto de los sistemas de gestión, los roles estáticos más comunes suelen incluir:

- Administrador: Usuario con privilegios máximos, responsable de la configuración del sistema, la gestión de usuarios y la asignación de roles.
- Usuario estándar: Rol con permisos básicos para operar dentro del sistema según su función específica (lectura, escritura, modificación de ciertos recursos).
- Invitado o visitante: Rol con acceso limitado a información pública o de solo lectura, sin capacidad de modificación.

La adopción de un modelo de roles estáticos en una aplicación software implica, desde el punto de vista

arquitectónico, la necesidad de implementar un subsistema de autenticación y autorización que valide cada solicitud del usuario contra la matriz de permisos definida para su rol. Este enfoque simplifica el desarrollo al eliminar la complejidad asociada a los sistemas de roles dinámicos (donde los permisos pueden cambiar según reglas de negocio, tiempo, ubicación o estado del sistema), pero requiere una planificación cuidadosa de la jerarquía y granularidad de los roles para cubrir todos los casos de uso previstos (Colantonio; Di Pietro; Ocello y Verde, 2010).

En el servicio *backend* objeto de esta investigación, se han definido los roles estáticos mostrados en la tabla 2.1, cada uno con una misión y función principal.

Cuadro 2.1. Funciones y misión de los roles del sistema. Fuente: elaboración propia.

Rol	Función principal	Misión
Administrador	Gestión y control total de plataforma y datos.	Posee el control total de la plataforma y por tanto es el responsable de su salud, configuración y gestión de usuarios.
Diseñador de modelo	Propietario funcional del ciclo de diseño del modelo.	El arquitecto creativo encargado de idear, diseñar y liderar la construcción de los FM.
Editor de modelo	Colaborador con capacidad de edición acotada al ámbito invitado.	Aporta su conocimiento editando modelos a los que ha sido invitado.
Revisor	Rol de control de calidad y publicación, sin edición estructural.	Vela por la calidad del modelo, valida y aprueba los modelos antes de que puedan ser utilizados.
Configurador	Consumidor experto de modelos publicados para derivar configuraciones.	Constructor práctico que utiliza los FM aprobados para ensamblar soluciones finales.
Observador	Acceso de solo lectura a artefactos finales/publicados.	Consulta la información pública y final sin poder alterarla.

Esta estratificación de roles garantiza una clara separación de responsabilidades y sigue el principio de privilegio mínimo, según el cual cada usuario debe tener únicamente los permisos necesarios para cumplir con sus funciones (Saltzer y Schroeder, 1975). En la tabla 2.2 se muestra los permisos estáticos de escritura y lectura que posee cada rol.

Cuadro 2.2. Jerarquía de roles estáticos del sistema. Fuente: elaboración propia.

Tabla	Administrador	Diseñador de modelo	Editor de modelo	Revisor	Configurador	Observador
User	R=Sí / W=Sí (global)	R=Sí / W=No	R=Sí / W=No	R=No / W=No	R=No / W=No	R=No / W=No
Domain	R=Sí / W=Sí (global)	R=Sí / W=No	R=Sí / W=No	R=Sí / W=No	R=Sí / W=No	R=Sí / W=No
Continúa en la siguiente página						

Cuadro 2.2 – Continuación de la página anterior.

Tabla / Entidad	Administrador	Diseñador de modelo	Editor de modelo	Revisor	Configurador	Observador
FeatureModel	R=Sí / W=Sí (global)	R=Sí / W=Sí (solo propios)	R=Sí / W=Sí (solo donde colabora; sin crear ni borrar)	R=Sí / W=No	R=Sí (publicados) / W=No	R=Sí (publicados) / W=No
FeatureModel Version	R=Sí / W=Sí (global)	R=Sí / W=Parcial	R=Sí / W=Parcial	R=Sí / W=Parcial	R=Sí / W=No	R=No / W=No
Feature	R=Sí / W=Sí (global)	R=Sí / W=Sí (en sus modelos)	R=Sí / W=Sí (en modelos permitidos)	R=Sí / W=No	R=Sí / W=No	R=No / W=No
FeatureRelation	R=Sí / W=Sí (global)	R=Sí / W=Sí (en sus modelos)	R=Sí / W=Sí (en modelos permitidos)	R=Sí / W=No	R=Sí / W=No	R=No / W=No
FeatureGroup	R=Sí / W=Sí (global)	R=Sí / W=Sí (en sus modelos)	R=Sí / W=Sí (en modelos permitidos)	R=Sí / W=No	R=Sí / W=No	R=No / W=No
restricción	R=Sí / W=Sí (global)	R=Sí / W=Sí (en sus modelos)	R=Sí / W=Sí (en modelos permitidos)	R=Sí / W=No	R=Sí / W=No	R=No / W=No
FeatureModel Collaborator	R=Sí / W=Sí (global)	R=Sí / W=Sí (gestiona colaboradores de sus modelos)	R=Sí / W=No	R=No / W=No	R=No / W=No	R=No / W=No
Resource / Tag	R=Sí / W=Sí (global)	R=Sí / W=Sí	R=Sí / W=Sí	R=Sí / W=Sí	R=Sí / W=Sí (acotado según configuración propia)	R=No / W=No
Configuration	R=Sí / W=Sí (global)	R=Sí / W=No	R=Sí / W=No	R=No / W=No	R=Sí / W=Sí (propias)	R=Sí (públicas) / W=No

Convención utilizada en la tabla: R = lectura, W = escritura.

2.1.2. Componentes funcionales de la lógica de negocio del sistema

Dentro de la arquitectura general del sistema existe una separación funcional de componentes fundamentales que encapsulan en requisitos de alto nivel las funcionalidades principales del sistema. Cada uno respaldado por estrategias tecnológicas y buenas prácticas del desarrollo de software que optimizan su ejecución y conexión entre los mismos.

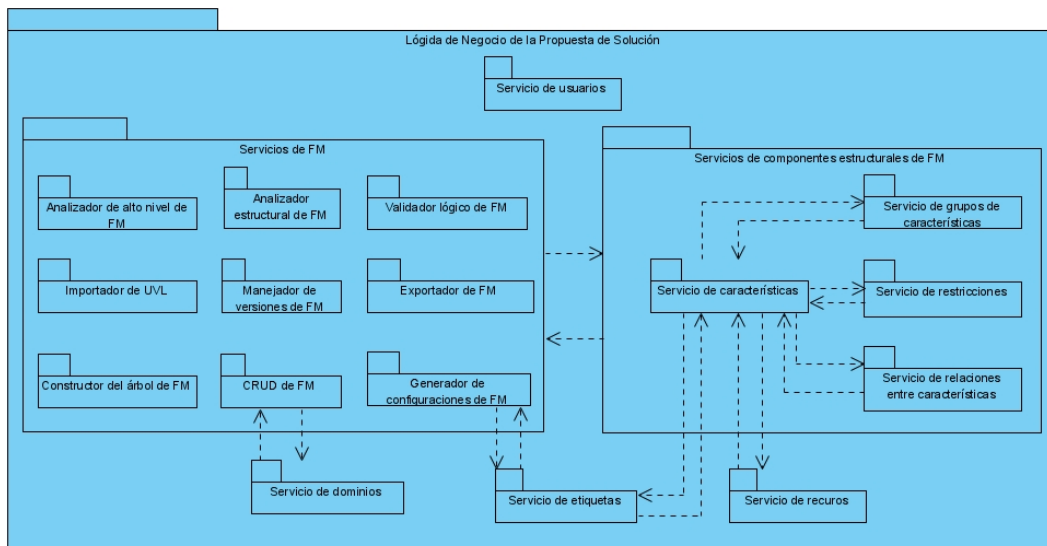


Figura 2.1. Componentes de la lógica del negocio del sistema. Fuente: elaboración propia.

La figura 2.1 ilustra los elementos que componen la lógica del negocio del sistema. En ella se ve claramente dividido el núcleo funcional en dos principales responsabilidades: los servicios de componentes estructurales de FM y los servicios que operan directamente sobre el FM en su totalidad. Ambos son dependientes del otro para su correcto funcionamiento.

Los servicios de componentes estructurales de FM proveen las funcionalidades básicas (Creación, Lectura, Actualización y Eliminación (CRUD) y otras) que permiten construir y modelar los FM. Por otro lado, los servicios de FM, están descompuestos estratégicamente como parte del principio de alta cohesión. A continuación, se describen los servicios de FM:

CRUD de FM

Tiene como objetivos exponer operaciones completas de gestión de FM y centralizar control de permisos, validaciones y respuesta de datos para los clientes. Tiene como funcionalidades:

- Proveer *endpoints* para administrar FM.
- Aplicar control de permisos por rol y propiedad del modelo.

Validador lógico de FM

Tiene como objetivos verificar la consistencia lógica global de un FM y asegurar satisficibilidad de restricciones y relaciones. Entre sus funcionalidades se encuentran:

- Construir variables booleanas para cada característica.

- Codificar jerarquías, grupos y restricciones.
- Verificar satisfacibilidad completa del modelo y de configuraciones específicas.
- Soportar validación de selecciones parciales con caché.
- Permitir enumerar configuraciones válidas (cuando se utiliza en conjunto con el generador).
- Permitir validaciones en tres niveles:
 - Nivel 1 (SymPy): validación simbólica para modelos pequeños.
 - Nivel 2 (PySAT): solucionador [SAT](#) escalable para modelos medianos/grandes.
 - Nivel 3 (Z3): [SMT](#)/optimización para análisis avanzados.

Analizador estructural de [FM](#)

Se encarga de analizar propiedades topológicas del modelo (más allá de la lógica pura) y detectar problemas estructurales y métricas de complejidad. Cuenta con las funcionalidades de:

- Construir grafos de dependencias y jerarquía.
- Analizar alcanzabilidad, redundancias y relaciones implícitas.
- Calcular métricas de impacto y complejidad.
- Proveer los siguientes tipos de análisis:
 - Características muertas: Características inaccesibles desde la raíz.
 - Redundancias: relaciones o restricciones duplicadas.
 - Relaciones implícitas: dependencias no explícitas en la jerarquía.
 - Dependencias transitivas: efectos indirectos entre características.
 - Componentes fuertemente conexas: ciclos y conectividad.
 - Métricas: centralidad, complejidad, profundidad, impacto.

Analizador de alto nivel de [FM](#)

Tiene el objetivo principal de orquestar análisis de alto nivel combinando validación lógica, análisis estructural y métricas derivadas. Entre sus funcionalidades se encuentran:

- Construir *payloads* desde la versión.
- Ejecutar validación lógica y análisis estructural.
- Calcular métricas derivadas: núcleo / comunalidad, conjuntos atómicos y estimación de configuraciones.
- Integrar validaciones [UVL](#) y validación adicional con Flamapy.

Constructor del árbol de [FM](#)

Encargado únicamente de construir la representación completa del [FM](#) para consumo de los clientes, por lo que tiene como funcionalidades:

- Construir el árbol jerárquico anidado con metadatos.
- Generar lista de relaciones y restricciones con descripciones legibles.
- Calcular estadísticas agregadas (total de nodos, profundidad del árbol, grupos, etc.).
- Incluir recursos y etiquetas asociados a características.

Manejador de versiones de FM

Tiene como objetivos gestionar el ciclo de vida de versiones del FM y garantizar trazabilidad, inmutabilidad y reproducibilidad. Cuenta con las siguientes funcionalidades:

- Crear nuevas versiones (vacías o clonadas).
- Publicar, archivar y restaurar versiones.
- Genera *snapshots* completos con mapeos *UUID* $\leftrightarrow$ *Integer* y estadísticas.
- Controla estados (borrador, en revisión, publicado, archivado).

Exportador de FM

Su objetivo es exportar el FM a formatos estándar para herramientas externas. Tiene las funciones de:

- Mapea Identificador Único Universal (UUID)s a identificadores numéricos para formatos SAT/externos.
- Generar representaciones en múltiples estándares, tales como:
 - UVL.
 - Lenguaje de Marcado Extensible (XML).
 - Lenguaje Textual de Variabilidad (TVL).
 - Formateo DIMACS (DIMACS).
 - JSON.
 - Lenguaje DOT de Graphviz (DOT).
 - Mermaid.

Importador de UVL

Tiene como objetivos importar especificaciones UVL y materializarlas como estructura de FM; y validar UVL en fases: análisis sintáctico, estructura, restricciones simples y diferencia. Tiene como funcionalidades:

- Analizar sintácticamente UVL en una estructura intermedia (nodos, grupos, restricciones).
- Validar estructura: raíz única, ciclos, cardinalidad mínima de grupos.
- Convertir UVL a entidades del modelo (características, grupos, relaciones, restricciones).
- Permitir comparar un UVL contra una versión existente.

Generador de configuraciones de FM

Encargado de generar configuraciones válidas completas o completar decisiones parciales, y proponer soluciones diversas o bien optimizadas según estrategia. Cuenta con las funcionalidades de:

- Usar validación lógica para asegurar que las configuraciones cumplan todas las restricciones.
- Generar una configuración única o múltiples configuraciones diversas.
- Permitir diferentes estrategias para generar las configuraciones:
 - GREEDY: determinista, rápida y con prioridad en relaciones obligatorias; especializa en rapidez y consistencia sobre diversidad.
 - RANDOM: estocástica; prioriza diversidad y exploración, útil para muestras variadas.

- BEAM_SEARCH: explora varios caminos en paralelo con poda; balancea exploración/eficiencia y prioriza soluciones prometedoras.
- GENETIC ([Algoritmos Evolutivos Distribuidos en Python \(DEAP\)](#)): búsqueda evolutiva multi-objetivo; prioriza optimización de criterios definidos (p. ej., minimizar costos o maximizar cobertura).
- SAT_ENUM: enumeración por SAT; prioriza exhaustividad controlada, ideal para listar configuraciones válidas.
- PAIRWISE / UNIFORM / STRATIFIED: muestreo para cobertura; prioriza representatividad (cobertura de pares, distribución uniforme o estratos definidos).
- CP_SAT ([Herramientas de Investigación Operativa \(OR-Tools\)](#)): programación con restricciones; prioriza factibilidad bajo restricciones complejas y objetivos declarativos.
- BDD: muestreo con diagramas de decisión binaria; prioriza eficiencia en conteo/muestreo cuando el modelo es grande.
- [Algoritmo Genético de Ordenamiento No-Dominado II \(NSGA2\)](#): optimización multi-objetivo con Pareto; prioriza soluciones no dominadas y *trade-offs* entre objetivos.

2.2. Levantamiento de requisitos

El levantamiento de requisitos constituye una fase crítica dentro del ciclo de vida del desarrollo de software, al permitir identificar de forma estructurada las necesidades, restricciones y expectativas de los usuarios en relación con el sistema a construir. Esta etapa no solo define el alcance funcional del producto, sino que también condiciona su calidad, adaptabilidad y éxito posterior. (Pressman, 2014)

2.2.1. Requisitos funcionales

Los [Requisitos Funcionales \(RF\)](#) son especificaciones detalladas que describen las funciones y comportamientos que un sistema, software o producto debe realizar para satisfacer las necesidades y expectativas de los usuarios. Su principal función es definir qué debe hacer el sistema para cumplir con los objetivos del negocio y las demandas del usuario, estableciendo claramente las tareas, procesos y respuestas que el sistema debe ejecutar ante diferentes situaciones. Estos requisitos son esenciales para asegurar que el sistema cumpla su propósito y funcione de acuerdo a los objetivos especificados del proyecto. (Maciaszek y Liong, 2005). La tabla 2.3 muestra los RF que debe cumplir la aplicación propuesta para dar solución a la problemática planteada.

Cuadro 2.3. Requisitos funcionales del sistema. Fuente: Elaboración propia.

No.	Nombre	Descripción	Prioridad	Complejidad
RF#1	Autenticar usuario	Permite acceder al sistema introduciendo los siguientes datos: • correo electrónico • contraseña	Alta	Baja
Continúa en la siguiente página				

Cuadro 2.3 – Continuación de la página anterior.

No.	Nombre	Descripción	Prioridad	Complejidad
RF#2	Cerrar sesión	Permite cerrar la sesión de un usuario anteriormente autenticado	Baja	Baja
RF#3	Registrar usuario	Permite registrar un usuario en el sistema con los siguientes datos: • correo electrónico • contraseña	Alta	Baja
RF#4	Solicitar recuperación de contraseña	Genera un token de recuperación y lo envía por correo para restablecer el acceso de forma controlada.	Baja	Media
RF#5	Restablecer contraseña	Permite definir una nueva contraseña válida de forma segura a partir de uno de los siguientes escenarios: • Escenario con token: token de recuperación y nueva contraseña. • Escenario autenticado: contraseña actual y nueva contraseña.	Baja	Media
RF#6	Listar usuarios	Proporciona el listado paginado de usuarios para perfiles con permisos autorizados.	Alta	Baja
RF#7	Filtrar usuarios	Permite filtrar usuarios por rol o mediante un término de búsqueda.	Media	Baja
RF#8	Actualizar perfil propio	Permite al usuario editar su información personal sin depender de administración externa. Campo de entrada: • correo electrónico (opcional)	Alta	Baja

Continúa en la siguiente página

Cuadro 2.3 – Continuación de la página anterior.

No.	Nombre	Descripción	Prioridad	Complejidad
RF#9	Asignar rol de usuario	Permite al administrador del sistema modificar el rol asignado a un usuario. Parámetros de entrada: • identificador de usuario (ruta) • rol (consulta)	Alta	Baja
RF#10	Desactivar usuario	El sistema debe permitir inhabilitar cuentas activas.	Media	Baja
RF#11	Activar usuario	Reactiva cuentas inactivas para restablecer su acceso al sistema.	Media	Baja
RF#12	Listar dominios	El sistema debe permitir listar los dominios activos en el sistema.	Alta	Baja
RF#13	Obtener dominio	Obtiene la información detallada de un dominio específico por su identificador.	Media	Baja
RF#14	Crear dominio	El sistema debe permitir crear nuevos dominios desde perfiles administrativos. Campos de entrada: • nombre • descripción (opcional)	Alta	Baja
RF#15	Actualizar dominio	Permite editar la información de un dominio ya existente a un administrador. Parámetros y campos de entrada: • identificador de dominio (ruta) • nombre (opcional) • descripción (opcional)	Alta	Baja
RF#16	Buscar dominios	Permite localizar dominios por nombre o por descripción.	Media	Baja
RF#17	Obtener dominio con FM	Devuelve un dominio junto a sus FM relacionados.	Media	Baja
RF#18	Activar dominio	Habilita un dominio previamente desactivado.	Media	Baja

Continúa en la siguiente página

Cuadro 2.3 – Continuación de la página anterior.

No.	Nombre	Descripción	Prioridad	Complejidad
RF#19	Desactivar dominio	Inhabilita un dominio para evitar su uso temporal o definitivo.	Media	Baja
RF#20	Listar FM	Permite listar FM con paginación y filtros por dominio para facilitar su navegación.	Alta	Media
RF#21	Crear FM	Crea un FM asociado a un dominio válido y consistente. Campos de entrada: • nombre • descripción (opcional) • identificador de dominio	Alta	Media
RF#22	Consultar FM	Muestra el detalle del FM incluyendo sus versiones.	Alta	Media
RF#23	Actualizar FM	El sistema debe permitir actualizar metadatos del FM manteniendo integridad de la información. Parámetros y campos de entrada: • identificador de modelo (ruta) • nombre (opcional) • descripción (opcional) • estado activo (opcional)	Alta	Media
RF#24	Activar FM	Activa un FM para habilitar su uso operativo.	Alta	Baja
RF#25	Desactivar FM	Desactiva un FM cuando se requiere restringir su uso.	Alta	Baja
RF#26	Listar versiones de modelo	Lista las versiones disponibles de un FM específico.	Alta	Baja
RF#27	Consultar versión específica de un modelo	Permite recuperar el detalle de una versión concreta de FM.	Alta	Baja

Continúa en la siguiente página

Cuadro 2.3 – Continuación de la página anterior.

No.	Nombre	Descripción	Prioridad	Complejidad
RF#28	Crear versión de un modelo	Crea una nueva versión, con soporte de <i>copy-on-write</i> desde una versión origen. Parámetros y campos de entrada: • identificador de modelo (ruta) • identificador de versión origen (opcional) • descripción (opcional)	Alta	Media
RF#29	Publicar versión de un modelo	El sistema debe permitir publicar una versión y ejecuta validaciones necesarias antes de su disponibilidad oficial.	Alta	Alta
RF#30	Achivar versión de un modelo	Cambia el estado de una versión a archivada para su preservación histórica.	Media	Baja
RF#31	Restaurar versión de un modelo	EL sistema debe permitir recuperar una versión archivada y la devuelve al flujo de trabajo activo.	Media	Baja
RF#32	Obtener estructura completa de un FM	El sistema debe entregar el árbol completo para consulta integral de una versión concreta ya sea por identificar o por la versión publicada más reciente.	Alta	Alta
RF#33	Obtener Estadísticas de un FM	Obtiene métricas principales de la versión publicada más reciente o de una versión específica.	Alta	Media

Continúa en la siguiente página

Cuadro 2.3 – Continuación de la página anterior.

No.	Nombre	Descripción	Prioridad	Complejidad
RF#34	Exportar un FM	El sistema debe permitir exportar a la versión publicada más reciente o a una versión concreta en los siguientes formatos: • UVL. • XML. • TVL. • DIMACS. • JSON. • DOT. • mermaid.	Alta	Media
RF#35	Obtener UVL efectivo de un FM	Permite devolver el UVL final, ya sea almacenado o generado desde la estructura vigente.	Alta	Media
RF#36	Guardar UVL	El sistema debe permitir guardar un UVL analizado y validado. Parámetros y campos de entrada: • identificador de modelo (ruta) • identificador de versión (ruta) • contenido UVL	Alta	Alta
RF#37	Sincronizar UVL desde estructura	Regenera y persiste UVL a partir del modelo visual para mantener coherencia.	Alta	Alta
RF#38	Aplicar UVL a estructura	El sistema debe permitir construir una nueva versión estructural tomando como entrada una definición UVL. Parámetros y campos de entrada: • identificador de modelo (ruta) • identificador de versión (ruta) • contenido UVL	Alta	Alta

Continúa en la siguiente página

Cuadro 2.3 – Continuación de la página anterior.

No.	Nombre	Descripción	Prioridad	Complejidad
RF#39	Validar lógica del modelo	El sistema debe permitir evaluar la satisfacibilidad y consistencia lógica global del modelo.	Alta	Alta
RF#40	Validar estructura del modelo	Verifica integridad estructural, incluyendo raíz, ciclos y nodos huérfanos de un FM.	Alta	Media
RF#41	Validar configuración seleccionada	Comprueba que una selección concreta de características sea válida y consistente.	Alta	Alta
RF#42	Análisis estructural de validación	El sistema debe permitir ejecutar análisis de características muertas, redundancias y componentes fuertemente conectados.	Alta	Alta
RF#43	Crear configuración persistente	Crea y almacena configuraciones asociadas a una versión tras validar consistencia. Campos de entrada: • nombre • descripción (opcional) • identificador de versión del modelo de características • lista de identificadores de características	Alta	Alta
RF#44	Obtener configuración por identificador	Recupera una configuración específica junto con sus características relacionadas.	Media	Baja
RF#45	Listar configuraciones	Lista configuraciones disponibles con soporte de paginación.	Media	Baja

Continúa en la siguiente página

Cuadro 2.3 – Continuación de la página anterior.

No.	Nombre	Descripción	Prioridad	Complejidad
RF#46	Actualizar configuración	Modifica metadatos y características de una configuración existente. Parámetros y campos de entrada: • identificador de configuración (ruta) • nombre (opcional) • descripción (opcional) • lista de identificadores de características (opcional)	Media	Baja
RF#47	Activar configuración	Marca una configuración como activa para su uso.	Media	Baja
RF#48	Desactivar configuración	Permite al sistema eliminar lógicamente una configuración.	Media	Baja
RF#49	Validar configuración propuesta	El sistema debe permitir evaluar una selección de características sin necesidad de persistirla.	Alta	Alta
RF#50	Generar configuraciones válidas	Genera una o varias configuraciones válidas según estrategia definida.	Alta	Alta
RF#51	Optimizar configuraciones	Ejecuta algoritmos de optimización para obtener configuraciones de mejor calidad.	Alta	Alta
RF#52	Listar características	Lista características mediante filtros y representación de árbol por versión.	Alta	Media

Continúa en la siguiente página

Cuadro 2.3 – Continuación de la página anterior.

No.	Nombre	Descripción	Prioridad	Complejidad
RF#53	Crear característica	El sistema debe permitir crear nuevas características aplicando validaciones y permisos sobre la versión objetivo del modelo. Campos de entrada: • nombre • tipo • identificador de versión del modelo de características • identificador de padre (opcional) • identificador de grupo (opcional) • identificador de recurso (opcional) • propiedades (opcional)	Alta	Alta
RF#54	Obtener característica	Recupera el detalle completo de una característica específica.	Media	Baja
RF#55	Obtener subárbol de característica	Devuelve una característica junto con toda su descendencia.	Media	Media
RF#56	Consultar relaciones de característica	Lista relaciones en las que participa una característica determinada.	Media	Baja
RF#57	Listar etiquetas de característica	Recupera las etiquetas asociadas a una característica.	Media	Baja
RF#58	Asociar etiqueta con característica	El sistema debe permitir vincular una etiqueta a una característica.	Media	Baja
RF#59	Desasociar etiqueta de una característica	Permite eliminar el vínculo entre una etiqueta y una característica.	Media	Baja

Continúa en la siguiente página

Cuadro 2.3 – Continuación de la página anterior.

No.	Nombre	Descripción	Prioridad	Complejidad
RF#60	Actualizar característica	El sistema debe permitir actualizar campos puntuales de una característica. Parámetros y campos de entrada: • identificador de característica (ruta) • nombre (opcional) • tipo (opcional) • identificador de padre (opcional) • identificador de grupo (opcional)	Alta	Alta
RF#61	Mover característica	El sistema debe permitir reubicar una característica cambiando su nodo padre sin perder trazabilidad.	Alta	Alta
RF#62	Desactivar característica	El sistema debe permitir eliminar lógicamente una característica y generar una nueva versión consistente del modelo.	Alta	Alta
RF#63	Activar característica	El sistema debe permitir recuperar una característica anteriormente desactivada.	Alta	Alta
RF#64	Listar grupos de características	El sistema debe permitir listar grupos de características con filtros por versión, padre y tipo de agrupación.	Media	Baja
RF#65	Obtener grupo de características	Recupera la información de un grupo de características por su identificador.	Media	Baja

Continúa en la siguiente página

Cuadro 2.3 – Continuación de la página anterior.

No.	Nombre	Descripción	Prioridad	Complejidad
RF#66	Crear grupo de características	Crea grupos de características con validaciones para un modelo. Campos de entrada: • tipo de grupo • identificador de característica padre • cardinalidad mínima (opcional, valor por defecto 1) • cardinalidad máxima (opcional)	Alta	Alta
RF#67	Actualizar grupo de características	Actualiza grupos de características conservando consistencia y versionado en el modelo. Parámetros y campos de entrada: • identificador de grupo (ruta) • tipo de grupo (opcional) • identificador de característica padre (opcional) • cardinalidad mínima (opcional) • cardinalidad máxima (opcional)	Alta	Alta
RF#68	Desactivar grupo de características	Elimina lógicamente grupos de características aplicando reglas de integridad y versionado.	Alta	Alta
RF#69	Activar grupo de características	Permitir recuperar grupos de características desactivados.	Alta	Alta
RF#70	Listar relaciones de características	Lista relaciones entre características con criterios de filtrado.	Media	Baja
RF#71	Obtener relación de características	Obtiene el detalle de una relación de características por identificador.	Media	Baja

Continúa en la siguiente página

Cuadro 2.3 – Continuación de la página anterior.

No.	Nombre	Descripción	Prioridad	Complejidad
RF#72	Crear relación de características	El sistema debe permitir crear relaciones entre características con validaciones semánticas y <i>copy-on-write</i> . Campos de entrada: • tipo • identificador de característica origen • identificador de característica destino	Alta	Alta
RF#73	Actualizar relación de características	El sistema debe permitir actualizar relaciones de características existentes. Parámetros y campos de entrada: • identificador de relación (ruta) • tipo (opcional) • identificador de característica origen (opcional) • identificador de característica destino (opcional)	Alta	Alta
RF#74	Activar relaciones de características	Recupera una relación de características anteriormente desactivada.	Alta	Alta
RF#75	Desactivar relaciones de características	Elimina lógicamente una relación de características manteniendo la consistencia global del modelo.	Alta	Alta
RF#76	Listar de restricciones	Lista restricciones aplicables según filtros definidos.	Media	Baja
RF#77	Obtener restricción	Recupera el detalle de una restricción por identificador.	Media	Baja

Continúa en la siguiente página

Cuadro 2.3 – Continuación de la página anterior.

No.	Nombre	Descripción	Prioridad	Complejidad
RF#78	Crear restricción	El sistema debe permitir crear restricciones complejas con validaciones y control de versiones. Campos de entrada: • identificador de versión del modelo de características • expresión lógica • descripción (opcional)	Alta	Alta
RF#79	Actualizar restricción	Actualiza restricciones existentes preservando coherencia lógica. Parámetros y campos de entrada: • identificador de restricción (ruta) • expresión lógica (opcional) • descripción (opcional)	Alta	Alta
RF#80	Desactivar restricción	Elimina restricciones aplicando <i>copy-on-write</i> para mantener trazabilidad.	Alta	Alta
RF#81	Activar restricción	Elimina restricciones aplicando <i>copy-on-write</i> para mantener trazabilidad.	Alta	Alta
RF#82	Listar etiquetas	Lista etiquetas activas disponibles para clasificación funcional.	Media	Baja
RF#83	Obtener etiqueta	Recupera una etiqueta específica mediante su identificador.	Media	Baja
RF#84	Crear etiqueta	Crea nuevas etiquetas asegurando unicidad de nombre. Campos de entrada: • nombre • descripción (opcional)	Media	Baja
RF#85	Listar recursos	Lista recursos registrados con soporte de paginación.	Media	Baja

Continúa en la siguiente página

Cuadro 2.3 – Continuación de la página anterior.

No.	Nombre	Descripción	Prioridad	Complejidad
RF#86	Consultar recurso	Obtiene el detalle de un recurso específico por su identificador.	Media	Baja
RF#87	Crear recurso	El sistema debe permitir registrar recursos y asigna propietario. Campos de entrada: • título • tipo • URL o contenido del recurso • descripción (opcional) • idioma (opcional) • duración en minutos (opcional) • versión (opcional) • estado (opcional) • fecha de publicación (opcional) • nombre del autor (opcional) • identificador de propietario (opcional) • licencia (opcional) • válido hasta (opcional) • etiquetas (opcional) • notas de accesibilidad (opcional)	Media	Baja

Continúa en la siguiente página

Cuadro 2.3 – Continuación de la página anterior.

No.	Nombre	Descripción	Prioridad	Complejidad
RF#88	Actualizar recurso	El sistema debe permitir actualizar parcial o totalmente la información de un recurso y mantener el último documento guardado para el mismo recurso. Parámetros y campos de entrada: • identificador de recurso (ruta) • título (opcional) • tipo (opcional) • URL o contenido del recurso (opcional) • descripción (opcional) • idioma (opcional) • duración en minutos (opcional) • versión (opcional) • estado (opcional) • fecha de publicación (opcional) • nombre del autor (opcional) • identificador de propietario (opcional, solo superusuario) • licencia (opcional) • válido hasta (opcional) • etiquetas (opcional) • notas de accesibilidad (opcional)	Media	Media

2.2.2. Requisitos no funcionales

Los **Requisitos no Funcionales (RnF)** especifican criterios que permiten juzgar la operación de un sistema, en lugar de sus comportamientos específicos. Estos requisitos, clasificados en categorías como rendimiento, seguridad, fiabilidad, usabilidad, etc; actúan como restricciones o atributos de calidad que el software debe cumplir. A diferencia de los funcionales, que definen qué hace el sistema, los no funcionales establecen cómo debe hacerlo, garantizando propiedades medibles como tiempos de respuesta, capacidad de carga o estándares de protección de datos. Su correcta definición es crítica, pues impactan directamente en la arquitectura del sistema y en la experiencia del usuario final. (Serna-Montoya, 2012). De forma alternativa definen las restricciones del sistema (Ian Sommerville, 2005).

A continuación se definen los requisitos no funcionales del sistema:

Seguridad

- **RnF#1:** El sistema debe implementar autenticación basada en *tokens* **JSON Web Token (JWT)** para garantizar la seguridad de las sesiones de usuario.
- **RnF#2:** El sistema debe implementar un ciclo de vida completo para los *tokens* **JWT** que incluya operaciones de emisión, renovación e invalidación, garantizando que los *tokens* tengan un tiempo de expiración limitado y puedan ser revocados explícitamente por el usuario.
- **RnF#3:** El sistema debe implementar un mecanismo de eliminación lógica mediante un cambio de estado (activo/inactivo) para conservar la integridad referencial de la **Base de datos (BD)**.
- **RnF#4:** Uso exclusivo de **UUID** como tipo de dato para llaves primarias en las tablas de la **BD**, no valores numéricos. Su generación se realizará automáticamente a nivel de aplicación o **BD**. Esto garantiza unicidad global y mejora la seguridad al no exponer patrones secuenciales.
- **RnF#5:** Solo usuarios autorizados deben poder crear, modificar y eliminar registros del sistema.
- **RnF#6:** El acceso al sistema a través de cualquier cliente debe ser mediante una comunicación segura **TLS¹/SSL²** (**Protocolo de Transferencia de Hipertexto Seguro (HTTPS)**) para proteger la confidencialidad e integridad de los datos transmitidos.
- **RnF#7:** Toda configuración variable (**BD**, credenciales, **URLs** externas) debe inyectarse vía variables de entorno, no **hardcodeada**.

Rendimiento

- **RnF#8:** El 95 % de las peticiones a *endpoints* críticos deben responderse en ≤ 200 ms, y el 99 % en ≤ 500 ms.
- **RnF#9:** El *backend* debe manejar al menos 500 usuarios simultáneos sin degradación superior al 20 %.
- **RnF#10:** La latencia p95 entre capas (*backend* $\rightarrow$ **BD**) no debe superar los 50 ms en la misma región.
- **RnF#11:** El sistema debe soportar al menos 1000 peticiones por segundo en condiciones normales de operación.

Integración

- **RnF#12:** Todas las **APIs** públicas deben seguir **RESTful** con códigos **HTTP** estandarizados y formato **JSON** consistente.
- **RnF#13:** Todas las **APIs** públicas deben incluir la versión en la ruta (*/api/v1*) para cambios rompientes y mantener compatibilidad hacia atrás para parches/pequeñas mejoras.

Mantenibilidad y evolución

- **RnF#14:** Todo *endpoint* debe estar documentado con OpenAPI/Swagger; el código debe seguir una guía de estilo definida.
- **RnF#15:** El código debe estar modularizado para poder actualizarse o reemplazarse sin afectar al resto.
- **RnF#16:** El sistema debe soportar **CI/CD**.

¹Seguridad de la Capa de Transporte (TLS)

²Capa de Sockets Seguros (predecesor de TLS) (SSL)

Disponibilidad y confiabilidad

- **RnF#17:** Ante la caída de una instancia del *backend*, el tráfico debe redirigirse automáticamente a otra sin pérdida de peticiones activas.
- **RnF#18:** El *lifespan* de la aplicación debe garantizar arranque y apagado ordenados: inicializar dependencias, exponer *readiness/liveness*, permitir drenado de peticiones activas, limpiar recursos y permitir reanudación segura de trabajos en otra instancia.
- **RnF#19:** Todos los trabajos en curso (tareas asíncronas, procesos en memoria, sesiones transaccionales) deben poder persistir su estado o reencolarse para permitir reanudación en otra instancia sin pérdida.

Fiabilidad

- **RnF#20:** Los resultados de exportación de los **FM** deben ser consistentes en consultas repetidas.
- **RnF#21:** Las estructuras completas de un **FM** deben ser consistentes en consultas repetidas.

Monitoreo y observabilidad

- **RnF#22:** El sistema debe exponer una ruta pública (chequeo de vida) para comprobar de forma rápida que está en ejecución y respondiendo correctamente.
- **RnF#23:** El sistema debe mostrar el estado detallado de los servicios principales (PostgreSQL, Redis, Celery y MinIO) para saber la operatividad de los mismos, así como las métricas clave del *host* para facilitar el monitoreo.
- **RnF#24:** Todos los *logs* deben ser estructurados en formato **JSON**, con nivel (*debug/info/warn/error*) y *timestamp* ISO.
- **RnF#25:** Se debe alertar automáticamente ante cualquier error del sistema.

Escalabilidad

- **RnF#26:** El sistema debe permitir agregar o quitar instancias del *backend* sin detener el servicio ni perder sesiones activas (escalado horizontal).
- **RnF#27:** En horas pico, el sistema debe auto-escalar de 2 a 10 réplicas en menos de 3 minutos.
- **RnF#28:** La capa de datos debe soportar particionamiento o réplicas de lectura sin cambios en la lógica de negocio.

Portabilidad

- **RnF#29:** El *backend* debe poder ejecutarse idénticamente en desarrollo, *staging* y producción usando Docker/-Kubernetes bajo **Infraestructura como Código (IaC)** reproducible.

2.3. Historias de usuario

Las **HU**, dentro del enfoque **XP**, constituyen una técnica ágil empleada para la recolección y especificación de requisitos funcionales desde la perspectiva del usuario final. Estas se expresan como descripciones breves, usualmente en una o dos frases, utilizando un lenguaje natural no técnico que refleja directamente las necesidades, expectativas o problemas del usuario con relación al sistema. Su propósito es capturar de forma rápida, comprensible y accionable

qué funcionalidad requiere el usuario y por qué, sin profundizar en detalles de implementación. (MenZinsky; López; Palacio; Sobrino; Álvarez y Rivas, 2018)

Las **HU** son escritas para ser implementadas en un tiempo de 1 a 3 semanas como máximo. Cuando una **HU** esté compuesta por diferentes funcionalidades, será dividida en varias **HU**. Algunas no tienen un alto grado de complejidad para implementarlas. Entonces, se determinará que su período de desarrollo sea de 1, 2 o 4 días en un equivalente de 0.2, 0.4 y 0.8 en semanas, respectivamente, para así calcular el tiempo estimado del proyecto. (Beck, 2000)

Las **HU** se agrupan y organizan en épicas, las cuales son etiquetas que representan conjuntos de **HU** relacionadas entre sí por un objetivo funcional o temático de mayor envergadura. Una épica no es más que una **HU** de gran tamaño que aún no ha sido desglosada en historias más pequeñas y manejables (Cohn, 2004). Las épicas sirven como mecanismo de organización jerárquica dentro del *backlog* de producto, permitiendo a los equipos ágiles planificar y priorizar a niveles estratégicos sin perder de vista la trazabilidad entre las historias individuales y los grandes bloques de funcionalidad. En el contexto de esta investigación, las épicas constituyen la capa superior de organización de los requisitos funcionales, agrupando las **HU** que contribuyen a un mismo propósito dentro del sistema.

Las épicas definidas para esta investigación son las siguientes:

- **Épica 1: Gestión de Identidad y Acceso:** Agrupa las historias relacionadas con la autenticación, gestión de usuarios y control de acceso al sistema.
- **Épica 2: Gobernanza de Dominios Curriculares:** Incluye las historias que abordan la administración de dominios como contenedores de características.
- **Épica 3: Ciclo de Vida del FM:** Contiene las historias que tratan sobre la creación, gestión, versionado, métricas, exportación y publicación de los **FM** dentro del sistema.
- **Épica 4: Modelado de Variabilidad Estructural:** Reúne las historias enfocadas en la construcción del árbol (características), grupos y relaciones.
- **Épica 5: Restricciones y Lógica Avanzada:** Agrupa las historias relacionadas con la gestión de restricciones transversales y sincronización con el lenguaje **UVL**.
- **Épica 6: Validación y Análisis de Modelos:** Análisis de consistencia, detección de errores estructurales y características muertas.
- **Épica 7: Configuración, Optimización y Recursos:** Contiene las historias relacionadas con la derivación de productos, validación de selecciones, algoritmos de optimización, etiquetas y recursos pedagógicos.

A continuación, se presenta un ejemplo de las **HU** confeccionadas, el resto se puede encontrar en el anexo A.

Cuadro 2.4. Historia de usuario # 1

Historia de usuario	
Número: 1	Nombre: Validación Lógica y Estructural del Modelo
Épica: Validación y Análisis Lógica Avanzada	
Prioridad en negocio: Alta	Riesgo en desarrollo: Alta
Puntos estimados: 1.0	
RFs cubiertos: 40, 41, 43	
Programador responsable: Jose Luis Echemendia López	

Continúa en la próxima página

Cuadro 2.4. Continuación de la página anterior.

Descripción: El revisador y el diseñador deben poder ejecutar análisis de consistencia, detectar características muertas y redundancias para asegurar que el plan de estudios sea lógicamente posible y no presente errores.
Criterios de aceptación: • Evaluar la satisfacibilidad global del modelo mediante un solucionador SAT. • Detectar ciclos y nodos huérfanos. • Identificar características muertas y redundancias.
Observaciones: El análisis debe advertir problemas estructurales y semánticos antes de publicar o derivar configuraciones.

2.3.1. Estimación de esfuerzo por historia de usuarios

A lo largo de los años se han llevado a cabo numerosos estudios relacionados con la estimación de esfuerzo en el desarrollo de software ágil, los cuales permiten comprender los avances que han tenido y la necesidad de su uso en las empresas (Prieto, 2021). La metodología XP propone que la estimación de esfuerzo se realice de acuerdo a las HU. Para la solución informática a desarrollar se asignó una duración aproximada a cada una de las HU las cuales dieron como resultado una duración total aproximada de 13 semanas, como se muestra en la tabla 2.5.

Cuadro 2.5. Estimación de esfuerzo por historia de usuario

Iteración	Historias de usuario		Puntos estimados (semanas)
1	1	Administración de Usuarios y Permisos	0.3
	2	Autenticación y Autogestión de cuenta	0.4
2	3	Gestión de Dominios de Conocimiento	0.4
3	4	Análisis de Métricas y Exportación	0.8
	5	Gestión de Versiones y Publicación	2.5
	6	Gestión Operativa de FM	1.0
4	7	Modelado de Grupos y Relaciones Estructurales	1.0
	8	Edición de la Jerarquía de Características	1.0
5	9	Integración y Sincronización UVL	1.0
	10	Gestión de Restricciones Transversales	0.8
6	11	Validación Lógica y Estructural del Modelo	1.0
7	12	Enriquecimiento con Recursos y Taxonomías	0.8
	13	Validación y Generación Automática de Configuraciones	1.0
	14	Gestión de Configuraciones	1.0
Total			13.0

2.3.2. Desarrollo del plan de iteraciones

Luego de estimar el tiempo de desarrollo de las HU se realizará el plan de iteraciones, mediante este se seleccionan las HU que se desarrollarán en cada una de las etapas de entregas pendientes. La tabla 2.6 muestra el plan de iteraciones con el tiempo empleado en cada una de ellas y las HU que las componen.

Cuadro 2.6. Plan de duración de las iteraciones

Iteración	Historias de usuario		Duración (semanas)
1	1	Administración de Usuarios y Permisos	3
	2	Autenticación y Autogestión de cuenta	
2	3	Gestión de Dominios de Conocimiento	0.4
3	4	Análisis de Métricas y Exportación	4.3
	5	Gestión de Versiones y Publicación	
	6	Gestión Operativa de FM	
4	7	Modelado de Grupos y Relaciones Estructurales	2.0
	8	Edición de la Jerarquía de Características	
5	9	Integración y Sincronización UVL	1.8
	10	Gestión de Restricciones Transversales	
6	11	Validación Lógica y Estructural del Modelo	1.0
7	12	Enriquecimiento con Recursos y Taxonomías	2.8
	13	Validación y Generación Automática de Configuraciones	
	14	Gestión de Configuraciones	
Total			15.3

2.3.3. Desarrollo del plan de entregas

Una vez definido el plan de iteraciones, que organiza el desarrollo en ciclos temporales consecutivos, es necesario establecer el plan de entregas, el cual agrupa las iteraciones en hitos de mayor significación para el cliente o los *stakeholders* del proyecto (Beck y Andres, 2004). Mientras que las iteraciones representan ciclos internos de desarrollo orientados a la generación incremental de funcionalidad, las entregas constituyen versiones del producto que son puestas a disposición del usuario final, ya sea para su validación, para su uso en entornos de prueba o para su despliegue en producción. En la metodología XP, se recomienda que las entregas sean frecuentes y de corto ciclo, permitiendo así una retroalimentación temprana por parte del cliente y una adaptación continua de los requisitos (Wells, 2009). En la tabla 2.7 se presenta el plan de entregas definido para el desarrollo del servicio *backend*, especificando las iteraciones que componen cada entrega, la versión del producto obtenida y las fechas estimadas.

Cuadro 2.7. Plan de entregas.

Versión	Iteración	Fecha de inicio	Fecha de fin
Versión 0.1.1	Iteración 1	20 de diciembre del 2025	9 de enero del 2026
Versión 0.1.2	Iteración 2	10 de enero del 2026	30 de enero del 2026
Versión 0.1.3	Iteración 3	31 de enero del 2026	20 de febrero del 2026
Versión 0.1.4	Iteración 4	21 de febrero del 2026	13 de marzo del 2026
Versión 0.1.5	Iteración 5	14 de marzo del 2026	3 de abril del 2026
Versión 0.1.6	Iteración 6	4 de abril del 2026	24 de abril del 2026
Versión 0.1.7	Iteración 7	25 de abril del 2026	15 de mayo del 2026

2.4. Tarjetas Clase-Responsabilidad-Colaboración (CRC)

Las tarjetas **CRC** son una técnica utilizada en el diseño orientado a objetos para definir clases, asignar responsabilidades y establecer las colaboraciones necesarias entre ellas. Esta herramienta, de naturaleza visual y participativa, involucra activamente a los diseñadores en la exploración de comportamientos e interacciones del sistema. Cada tarjeta representa una clase y resume su nombre, funciones principales y otras clases con las que debe interactuar. Este enfoque promueve el pensamiento en términos de comportamiento más que de estructura de datos, y facilita la comprensión global del sistema desde etapas tempranas del desarrollo. (Pacienza, 2015)

A continuación, se presenta un ejemplo de las tarjetas **CRC** correspondientes a clases del sistema desarrollado, el resto se puede encontrar en el anexo B.

Cuadro 2.8. Tarjeta CRC # 1

Tarjeta CRC	
Clase: FeatureModel	
Responsabilidad	Colaboración
• Representar un FM (nombre, descripción, dominio). • Enlazar propietario y versiones. • Mantener colaboradores.	Domain User FeatureModelVersion FeatureModelCollaborator

2.5. Arquitectura de software

La arquitectura de software constituye el conjunto de decisiones de diseño fundamentales que definen la estructura global de un sistema informático. Comprende la identificación y organización de los componentes que lo conforman, las relaciones de comunicación y dependencia entre ellos, así como los principios y patrones que guían su evolución a lo largo del tiempo (Bass; Clements y Kazman, 2012). Más allá de un mero diagrama de bloques, la arquitectura representa el puente entre los requisitos funcionales y no funcionales del sistema y su implementación concreta,

estableciendo las bases para alcanzar atributos de calidad como la escalabilidad, el mantenimiento, la seguridad, el rendimiento y la interoperabilidad (Clements; Bachmann; Bass; Garlan; Ivers; Little; Nord y Stafford, 2010).

La importancia de definir adecuadamente la arquitectura de software radica en que esta determina, en gran medida, el éxito o fracaso del proyecto a largo plazo. Una arquitectura bien concebida permite gestionar la complejidad del sistema, facilita la comunicación entre los miembros del equipo de desarrollo, reduce los riesgos asociados a decisiones técnicas críticas y posibilita la reutilización de componentes en futuros proyectos (Rojas y Pérez, 2021). Por el contrario, una arquitectura deficiente o implícita puede derivar en sistemas difíciles de mantener, propensos a errores y costosos de extender. En el contexto del desarrollo ágil, la arquitectura no se define completamente al inicio, sino que emerge y se refina iterativamente a lo largo del proyecto, manteniendo un equilibrio entre la planificación anticipada y la adaptabilidad frente a requisitos cambiantes (Fowler, 2003).

En esta sección se justifica la elección del patrón específico adoptado para este proyecto. Además, se explica cómo su estructura se alinea con la naturaleza de la aplicación y facilita el cumplimiento de los objetivos de diseño planteados.

2.5.1. Representación arquitectónica de la solución

La arquitectura en capas, también conocida como arquitectura n-capas o arquitectura multitier, es un patrón arquitectónico que organiza los componentes de un sistema en niveles horizontales, donde cada capa cumple una función específica y se comunica únicamente con las capas adyacentes (Bass; Clements y Kazman, 2012). Este enfoque sigue el principio de separación de preocupaciones, promoviendo la cohesión interna de cada capa y reduciendo el acoplamiento entre ellas (Gamma; Helm; Johnson y Vlissides, 1995). Como resultado, las modificaciones en una capa tienen un impacto limitado sobre las demás, lo que facilita el mantenimiento, las pruebas y la evolución del sistema.

La arquitectura en capas ofrece múltiples beneficios que la convierten en una elección adecuada para la solución propuesta (Rojas y Pérez, 2021):

- Separación de responsabilidades: Cada capa se especializa en una función específica (presentación, lógica de negocio, acceso a datos), lo que facilita la comprensión del sistema y la asignación de tareas de desarrollo.
- Mantenibilidad: Los cambios en una capa (por ejemplo, migrar de BD) no afectan necesariamente a las demás, siempre que se respeten las interfaces de comunicación.
- Reutilizabilidad: Las capas inferiores (como la de persistencia) pueden ser reutilizadas por múltiples aplicaciones que requieran las mismas funcionalidades.
- Testabilidad: Cada capa puede ser probada de forma independiente mediante pruebas unitarias o de integración, facilitando la detección temprana de errores.
- Escalabilidad: Es posible escalar capas específicas según la demanda (por ejemplo, replicar la capa de servicios sin replicar la BD).

Capas definidas para la propuesta de solución

La solución se implementa como un monolito modular estructurado en capas, que combina principios de la arquitectura en capas con una organización orientada al dominio y mecanismos de procesamiento asíncrono. En cuanto al código, hay tres capas bien diferenciadas, cada una con responsabilidades claramente definidas. Además, se cuenta con una cuarta capa que encapsula los servicios externos de infraestructura que necesita el sistema para su correcto funcionamiento. La figura 2.2 ilustra la organización general de la arquitectura, el flujo de comunicación entre las capas y algunos componentes clave dentro de cada una de ellas.

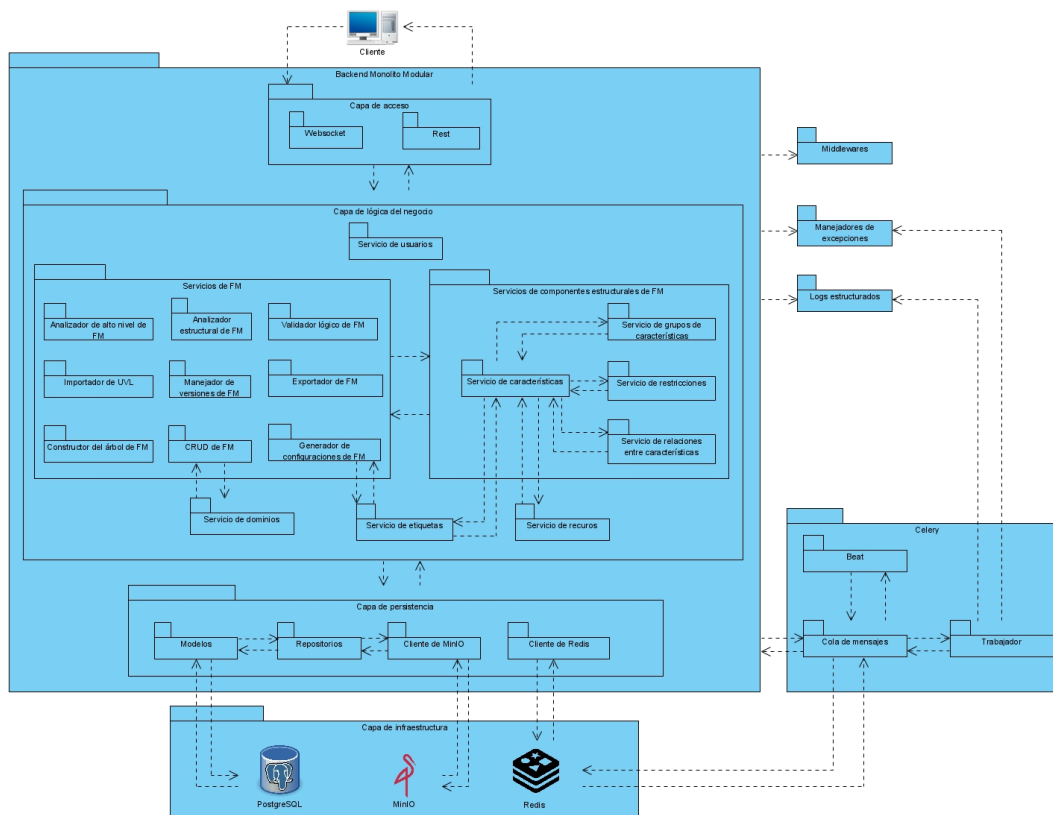


Figura 2.2. Arquitectura en capas de la propuesta de solución. Fuente: elaboración propia.

A continuación, se describen cada una de las capas que componen la arquitectura:

- **Capa de acceso (controladores):** Constituye el punto de entrada al sistema. Expone los *endpoints* de la [API](#) que reciben las peticiones de los clientes. Es responsable de:
 - Recibir y validar las solicitudes [HTTP](#).
 - Autenticar y autorizar a los usuarios mediante los mecanismos de control de acceso.
 - Enrutar las peticiones hacia los servicios correspondientes en la capa de lógica de negocio.
 - Formatear las respuestas (generalmente en [JSON](#)) y manejar los códigos de estado [HTTP](#).
 - Brindar una comunicación en tiempo real para obtener información estadística de los [FM](#).
- **Capa de lógica de negocio (servicios):** Constituye el núcleo funcional del sistema. Contiene toda la lógica relacionada con la gestión de [FM](#), la variabilidad curricular y elementos complementarios. Esta capa está estructurada siguiendo principios de [Diseño Orientado al Dominio \(DDD\)](#), organizando la lógica en servicios especializados que encapsulan reglas complejas del sistema de [FM](#) y de los componentes estructurales del mismo. Sus responsabilidades incluyen:
 - Implementar las operaciones [CRUD](#) de [FM](#).
 - Gestionar las relaciones estructurales y las restricciones de los [FM](#).
 - Gestionar el versionamiento de los modelos.
 - Ejecutar los análisis automatizados sobre los modelos (validación de consistencia, detección de características muertas, conteo de configuraciones, etc).

- Aplicar las reglas de negocio específicas del dominio educativo (prerrequisitos, correlatividades, límites de créditos, cantidad de horas clases, etc).
 - Orquestrar la generación de configuraciones válidas a partir de los modelos.
 - Exportación de [FM](#) e importación de [UVL](#).
 - Gestión de recursos, etiquetas, dominios y usuarios.
- **Capa de persistencia (repositorios y clientes):** Se encarga de gestionar todo el almacenamiento de datos del sistema, ya sea de larga duración ([BD](#), almacenamiento de archivos) o de corta duración (caché, sesiones). Su objetivo es abstraer a las capas superiores (lógica de negocio y acceso) de los detalles técnicos de dónde y cómo se guarda cada tipo de información al proporcionar una interfaz uniforme para las operaciones de acceso a datos. Sus funciones son:
 - Gestionar el almacenamiento de los [FM](#) en su representación interna.
 - Almacenar las configuraciones generadas y los metadatos asociados (usuario propietario, fechas, dominios).
 - Gestionar las entidades referentes a etiquetas, recursos, dominios, usuarios y las correspondientes al núcleo del sistema.
 - Almacenar archivos de exportación de [FM](#).
 - Almacenar recursos educativos (conferencias, clases prácticas, etc) asociados a las características.
 - Gestionar la memoria caché.
 - **Capa de infraestructura ([BD](#), [MinIO](#), [Redis](#)):** Constituye el nivel más bajo de la arquitectura del sistema. Es la responsable de proporcionar todos los componentes tecnológicos subyacentes que hacen posible el almacenamiento, la recuperación y el procesamiento eficiente de los datos. Esta capa es completamente transparente para las capas superiores (acceso, lógica de negocio y persistencia), las cuales interactúan con ella únicamente a través de los repositorios y clientes definidos en la capa de persistencia. Sus responsabilidades son:
 - Almacenar físicamente los datos en estructuras optimizadas (tablas, índices).
 - Garantizar la integridad referencial y la consistencia de los datos.
 - Ejecutar consultas eficientes sobre grandes volúmenes de información.
 - Gestionar transacciones y asegurar la durabilidad de los datos.
 - Almacenar físicamente los archivos de exportación de [FM](#) y recursos educativos.
 - Descargar objetos mediante [URLs](#) prefirmadas.
 - Organizar los objetos en *buckets*.
 - Guardar respaldos significativos de la [BD](#).
 - Cachear consultas frecuentes y resultados de operaciones de alto costo computacional para optimizar el rendimiento y reducir la latencia del sistema.

Flujo de comunicación entre capas

La comunicación entre capas sigue una dirección descendente: la capa de acceso recibe la petición del cliente y la delega a la capa de lógica de negocio; esta, a su vez, solicita los datos necesarios a la capa de persistencia; finalmente, la capa de persistencia interactúa con la infraestructura. La respuesta sigue el camino inverso, ascendiendo desde el nivel más bajo hasta el cliente.

Este flujo unidireccional garantiza que no existan dependencias circulares, lo que simplifica el mantenimiento y facilita la sustitución de implementaciones en cada nivel. Por ejemplo, es posible cambiar el [Sistema de Gestión de](#)

Bases de Datos (SGBD) (de PostgreSQL a MySQL) sin modificar la capa de lógica de negocio, siempre que la capa de persistencia mantenga la misma interfaz.

Arquitectura de Celery

Para atender los requisitos de procesamiento asíncrono y ejecución de tareas de larga duración (como la validación de **FM** complejos, el conteo de configuraciones mediante solucionadores **SAT** o la generación masiva de planes de estudio, entre otras tareas), la solución incorpora un sistema de colas de mensajes basado en Celery. Celery es un sistema de procesamiento de tareas distribuidas y asíncronas, ampliamente utilizado en aplicaciones Python, que permite descargar operaciones costosas del flujo principal de la aplicación, mejorando así la capacidad de respuesta del servicio y la experiencia del usuario (Celery Project, 2025a). La arquitectura de Celery presenta cuatro componentes fundamentales que interactúan entre sí para garantizar la ejecución confiable y distribuida de las tareas, tal como se ilustra en la figura 2.3.

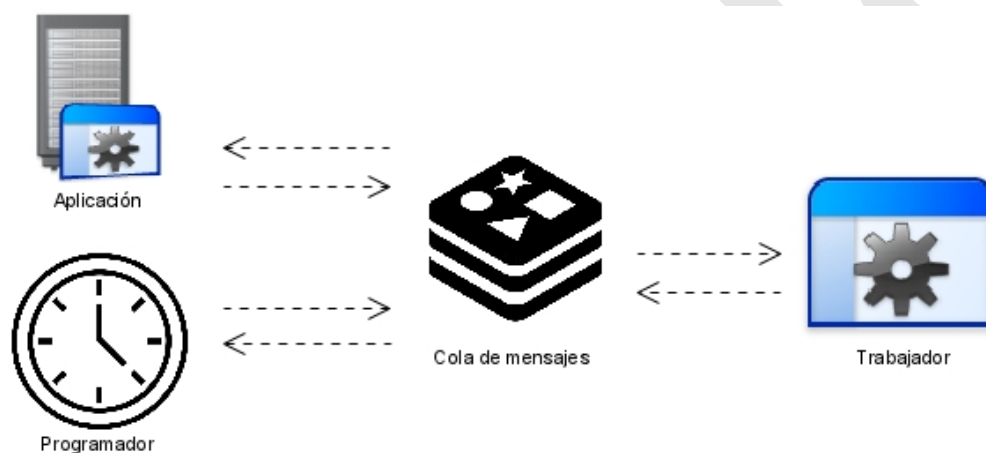


Figura 2.3. Arquitectura de Celery. Fuente: elaboración propia.

A continuación, se describen cada uno de estos componentes:

- **Aplicación:** Constituye el punto de entrada principal para la definición y configuración de las tareas. Es el componente que se integra directamente con el código del servicio *backend*, permitiendo invocar funciones como tareas asíncronas. La aplicación define qué tareas están disponibles, cómo deben serializarse los argumentos y qué cola de mensajes utilizar para la comunicación (Celery Project, 2025b).
- **Cola de mensajes:** Actúa como intermediario entre la aplicación que publica tareas y los trabajadores que las ejecutan. Cuando la aplicación invoca una tarea asíncrona, esta se empaqueta como un mensaje y se publica en la cola. Es responsable de almacenar temporalmente estos mensajes y distribuirlos a los trabajadores disponibles. En esta solución se emplea Redis como cola de mensajes, aprovechando su alto rendimiento y su capacidad para gestionar grandes volúmenes de mensajes (Celery Project, 2025d).
- **Trabajador:** Es el proceso que consume los mensajes de la cola y ejecuta las tareas definidas en la aplicación. Los trabajadores operan de forma independiente y pueden ejecutarse en uno o múltiples nodos, permitiendo escalar horizontalmente el procesamiento de tareas según la demanda del sistema. Cada trabajador puede gestionar múltiples tareas en paralelo mediante el uso de procesos hijos (Celery Project, 2025e).

- **Programador:** Es un componente especializado que actúa como planificador de tareas periódicas. Genera tareas de forma automática en intervalos de tiempo predefinidos (por ejemplo, cada hora, cada día) y las publica en la cola de mensajes para que sean ejecutadas por los trabajadores. En el contexto del servicio *backend*, este componente resulta útil para tareas como la limpieza periódica de modelos temporales, el envío de notificaciones programadas o la ejecución de análisis recurrentes sobre los **FM** (Celery Project, 2025c).

La integración de Celery en la arquitectura del servicio *backend* aporta múltiples ventajas al sistema:

- **Mejora de la experiencia de usuario:** Las operaciones costosas (como la validación de un **FM** con cientos de características) se ejecutan en segundo plano, devolviendo al cliente una respuesta inmediata con el identificador de la tarea. El cliente puede consultar posteriormente el estado y los resultados de la tarea sin bloquear la interfaz.
- **Escalabilidad horizontal:** Al aumentar la carga de trabajo, es posible añadir más trabajadores (incluso en servidores distintos) para procesar las tareas en paralelo, sin modificar el código de la aplicación principal.
- **Resiliencia:** Si un trabajador falla durante la ejecución de una tarea, Celery puede reencolar automáticamente la tarea para que sea ejecutada por otro trabajador, garantizando la finalización de las operaciones críticas.
- **Desacoplamiento:** La lógica de negocio que invoca tareas asíncronas no necesita conocer los detalles de cómo, cuándo o dónde se ejecutan, lo que reduce el acoplamiento entre componentes.

Ventajas de la arquitectura para la propuesta de solución

La elección de la arquitectura responde directamente a las necesidades identificadas en el análisis de requisitos. Esta representación arquitectónica permite abordar de manera efectiva los desafíos específicos, tales como:

- **Integración con sistemas existentes:** La capa de acceso permite que sistemas como los mencionados en el estudio de homólogos u otros consuman las funcionalidades del *backend* de variabilidad curricular sin necesidad de modificaciones profundas.
- **Evolución del modelo curricular:** La separación entre la lógica de negocio y la persistencia facilita la implementación del versionamiento de modelos, permitiendo almacenar múltiples versiones históricas sin afectar las operaciones actuales.
- **Seguridad y auditoría:** La capa de lógica de negocio centraliza el control de acceso y la validación de permisos, garantizando que solo usuarios autorizados puedan modificar los modelos.
- **Reutilización de componentes:** La capa de persistencia puede ser reutilizada en futuros proyectos que requieran almacenar **FM**, mientras que la capa de lógica de negocio encapsula las reglas de análisis de variabilidad.
- **Rendimiento y escalabilidad:** La incorporación de un sistema de procesamiento asíncrono basado en Celery, junto con una estrategia de caché implementada sobre Redis, permite que la solución maneje eficientemente cargas de trabajo intensivas en cómputo sin degradar la experiencia del usuario.

2.6. Patrones de diseños

Mientras el patrón de arquitectura define la macro-estructura del sistema, los patrones de diseño ofrecen soluciones a problemas de diseño recurrentes a un nivel de granularidad menor, específicamente en el diseño de componentes y sus interacciones. La aplicación consciente de estos patrones es crucial para lograr un código flexible, reusable y fácil

de mantener (Aratchige; Manujaya y Weerasinghe, 2024). Esta sección se divide en dos sub-apartados para abordar las dos categorías principales de patrones utilizadas en el desarrollo de la plataforma.

2.6.1. Patrones GRASP

Los patrones **GRASP** constituyen un conjunto de principios y pautas fundamentales para la asignación de responsabilidades en el diseño orientado a objetos (Larman, 2004). A diferencia de los patrones de diseño concretos, los **GRASP** no ofrecen soluciones específicas codificadas, sino que proporcionan un marco conceptual que guía a los diseñadores en la toma de decisiones sobre qué clase debe ser responsable de qué tarea o información dentro del sistema (Begel, 2020). Su propósito es lograr un diseño con alta cohesión, bajo acoplamiento y una clara separación de preocupaciones, atributos esenciales para la mantenibilidad y extensibilidad del software. A continuación se muestran los patrones **GRASP** aplicados en el diseño de la propuesta de solución y sus usos específicos:

- Repository (Repositorio): Aísla el acceso a datos detrás de una interfaz/repositorio, separando persistencia de lógica de negocio.
 - Repositorio base y concretos para cada tabla de la **BD**, con métodos **CRUD** genéricos y específicos.
 - Inyección de repositorios vía dependencias.
- Service Layer (Capa de Servicios): Encapsula lógica de negocio en servicios de aplicación, evitando que los controladores conozcan detalles de dominio/persistencia.
 - Lógica de negocio referente a los servicios de los **FM**.
- Controller (Controlador): Centraliza la gestión de eventos y solicitudes, delegando a servicios para procesar la lógica.
 - Presente en los *routers* de la capa de presentación, recibiendo peticiones HTTP, inyectando dependencias, validando permisos y delegando a servicios/repositorios.
- Low Coupling (Bajo Acoplamiento): Minimiza las dependencias entre componentes, facilitando el mantenimiento y la evolución del sistema.
 - En la creación de las dependencias del sistema, las cuales son inyectadas en los controladores, repositorios y servicios.
 - Sesión de **BD** inyectada en los repositorios, evitando que la lógica de negocio tenga conocimiento de la infraestructura de persistencia.
- High Cohesion (Alta Cohesión): Asegura que cada componente tenga una única responsabilidad bien definida, mejorando la claridad y mantenibilidad del código.
 - Servicio enfocado en el versionado de los **FM**.
 - Servicio enfocado en la construcción del árbol de los **FM**.
 - Repositorios bases para cada tabla de la **BD**, con métodos específicos para cada entidad que contienen métodos de validación y soporte.

2.6.2. Patrones GoF

Los patrones de diseño **GoF** son soluciones clásicas y ampliamente reconocidas para problemas específicos de diseño en software orientado a objetos. Gamma; Helm; Johnson y Vlissides, 1995 subdividieron esta categoría de

patrones en tres grupos según su propósito. En este sub-apartado se detallan los patrones [GoF](#) específicos que se han empleado en el proyecto, explicando el problema que resuelve cada uno en el contexto de la aplicación.

Patrones creacionales

Los patrones creacionales abstractizan el proceso de instanciación de objetos, encapsulando el conocimiento sobre qué clases concretas crear y cómo se relacionan entre sí (Gamma; Helm; Johnson y Vlissides, 1995). Su objetivo principal es hacer que el sistema sea independiente de cómo se crean, componen y representan sus objetos, favoreciendo la reutilización del código y reduciendo el acoplamiento entre las clases que utilizan los objetos y las clases que los instancian (Seguridad PY, 2022).

- Singleton (Instancia Única): Controla una única instancia compartida para acceso global.
 - Empleado para instanciar únicamente los clientes de MinIO, Redis y Celery, evitando múltiples conexiones y garantizando un punto de acceso centralizado a estos servicios.
 - Instancia global para la configuración del sistema, permitiendo que todas las partes del código accedan a los mismos parámetros de configuración sin necesidad de pasarlos explícitamente.
- Factory Method (Método de Fábrica): Delega la creación de objetos a métodos especializados para desacoplar construcción de uso.
 - Empleado para instanciar componentes reutilizables y servicios de infraestructura bajo una política de creación centralizada.
 - Evita la duplicación de lógica de inicialización y reduce el acoplamiento entre consumidores e implementaciones concretas.
- Builder (Constructor): Separa la construcción de un objeto complejo de su representación final, permitiendo crear diferentes versiones del mismo objeto usando el mismo proceso de construcción paso a paso.
 - Empleado en la construcción del árbol completo del [FM](#).

Patrones estructurales

Los patrones estructurales se centran en la composición de clases y objetos para formar estructuras más grandes y flexibles, manteniendo la eficiencia y la independencia de los componentes involucrados (Gamma; Helm; Johnson y Vlissides, 1995). Estos patrones permiten resolver problemas relacionados con la interfaz que exponen los objetos, la manera en que se agregan responsabilidades adicionales y cómo se adaptan componentes con interfaces incompatibles para que puedan colaborar (Refactoring Guru, 2021).

- Facade (Fachada): Proporciona una interfaz simplificada y unificada a un conjunto de interfaces más complejas dentro de un subsistema.
 - Fachada de análisis que combina validación lógica, estructural y manejo de [UML](#) con el sistema homólogo FlamapyIDE.
- Decorator (Decorador): Permite añadir responsabilidades adicionales a un objeto de forma dinámica, sin modificar su estructura interna ni afectar a otros objetos de la misma clase.
 - Decoradores de caché y ruteo para los controladores.
 - Decoradores para registrar tareas en Celery.

Patrones de comportamiento

Los patrones de comportamiento gestionan la comunicación y asignación de responsabilidades entre objetos, encapsulando la lógica de interacción y distribución de tareas dentro del sistema (Gamma; Helm; Johnson y Vlissides, 1995). Su propósito es elevar el nivel de abstracción en el que se describe el flujo de control, permitiendo que los objetos se comuniquen de manera flexible y desacoplada, sin necesidad de conocer explícitamente las implementaciones concretas de sus colaboradores (Seguridad PY, 2022).

- *Strategy* (Estrategia): Define una familia de algoritmos intercambiables, los encapsula individualmente y permite que el algoritmo varíe de forma independiente a los clientes que lo usan.
 - Generación/optimización de configuraciones con estrategias.
 - Validación lógica multi-motor (*sympy/pysat/z3*) como estrategia interna.
- *State* (Estado): Permite que un objeto modifique su comportamiento cuando su estado interno cambia. El objeto parecerá cambiar su clase.
 - Transiciones de estado para los recursos y los FM (ambos tienen los siguientes estados: borrador, en revisión, publicado, archivado).

2.7. Conclusiones del capítulo

Luego del desarrollo de este capítulo se arribaron a las siguientes conclusiones:

- La solución cuenta con 88 RF encapsulados en 14 HU, organizadas en 7 iteraciones con un tiempo total estimado de desarrollo de 15 semanas aproximadamente. Se definieron 29 RnF que abarcan categorías de rendimiento, seguridad, usabilidad, integración, mantenibilidad, disponibilidad, fiabilidad, monitoreo, escalabilidad y portabilidad.
- El diseño del sistema responsable de propiciar una base sólida para la implementación de la aplicación se describe a través de una arquitectura en capas, apoyada con una arquitectura de Celery para potenciar el rendimiento.
- La aplicación de los principios GRASP y los patrones GoF eleva significativamente la mantenibilidad, flexibilidad y escalabilidad de la solución propuesta, facilitando la modularidad del código y la adaptación a futuros cambios en los requisitos.

Resultados

Escribir aquí el capítulo 3. Otra prueba más [engine](#). Esto es un ejemplo de cómo se puede *linkear* el anexo [Controlador Industrial](#).

Esto es otra cita de la inicial como ejemplo también (Ou y Xu, 2013). Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue

quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetur. Otra prueba más [engine](#) Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa. Otra prueba más [engine](#) Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris. Esto es una prueba de link al Anexo [D.2](#). Esto es una prueba de link a la palabra [sentencia](#)

Conclusiones

Escribir aquí las conclusiones.

Recomendaciones

Escribe aquí las recomendaciones de tu trabajo.

Borrador

engine part of a program which handles certain types of data (Computers). [54](#), [67](#), [68](#)

AAFM Análisis Automatizado de Feature Models. [14](#)

ACID Atomicidad, Consistencia, Aislamiento, Durabilidad. [35](#)

API Interfaz de Programación de Aplicaciones. [30](#), [31](#), [33](#), [34](#), [38](#), [39](#), [45](#)

BDD Diagrama de Decisión Binaria. [30](#)

BPMN Notación de Modelado de Procesos de Negocio. [45](#)

CASE Ingeniería de Software Asistida por Computadora. [45](#)

CI/CD Integración Continua y Despliegue Continuo. [37](#), [44](#)

CRC Clase, Responsabilidad y Colaboración. [52](#)

DRF Django REST Framework. [33](#), [34](#)

FM Modelo de Características. [ix](#), [x](#), [3](#), [4](#), [7–18](#), [20–22](#), [24–35](#), [37–42](#), [52](#)

FODA Análisis de Dominio Orientado a Características. [9](#)

HTTP Protocolo de Transferencia de Hipertexto. [30](#), [39](#)

HU Historias de Usuario. [50–52](#)

I/O Entrada/Salida. [34](#), [40](#)

IDE Entorno de Desarrollo Integrado. [45](#)

JSON Notación de Objetos JavaScript. [35](#)

JSONB JSON Binario. [35](#)

LMS Sistema de Gestión del Aprendizaje. [28](#), [29](#), [31](#)

MOOC Curso en Línea Masivo y Abierto. [3](#)

ORM Mapeo Objeto - Relacional. [35](#)

RAM Memoria de Acceso Aleatorio. [37](#)

REST Transferencia de Estado Representacional. [45](#)

S3 Simple Storage Service. [38](#)

SAT Satisfacibilidad Booleana. [12](#), [38](#), [42](#)

SHARPSAT Conteo de Modelos. [12](#)

SMT Satisfacibilidad Módulo Teorías. [38](#), [42](#)

SPL Línea de Productos de Software. [x](#), [3](#), [4](#), [6–8](#), [15](#), [18–22](#), [27–31](#), [33](#), [42](#), [54](#)
SPLE Ingeniería de Líneas de Productos de Software. [4](#), [6](#), [7](#), [9](#), [19](#), [21](#), [26](#), [29](#), [30](#), [52](#)
SQL Lenguaje de Consulta Estructurado. [35](#), [36](#)

TIC Tecnologías de la Información y Comunicación. [3](#)
TTL Tiempo de Vida. [38](#)

UML Lenguaje de Modelado Unificado. [44](#), [45](#)
URL Localizador Uniforme de Recursos. [38](#)
UVL Lenguaje de Variabilidad Universal. [xii](#), [14–16](#), [22](#), [27](#), [30](#), [31](#), [33](#), [52–54](#)

XP Extreme Programming. [48](#), [50–53](#)

Referencias bibliográficas

- ACADEMIALAB, [s.f.]. *Línea de productos de software*. Url: <https://academia-lab.com/enciclopedia/linea-de-productos-de-software/>. Recuperado 25 de octubre de 2025 (vid. pág. 6).
- ADLER, David y ZHANG, Kai, 2023. Evaluating Distributed Object Storage for Cloud-Native Applications: A Comparative Study of S3-Compatible Systems. *Journal of Cloud Computing*. Vol. 12, n.º 4, págs. 1-19 (vid. pág. 37).
- ANTHOLOGY INC., 2025. *Blackboard Learn: Plataforma para la enseñanza y el aprendizaje*. Url: <https://www.anthology.com/blackboard-learn>. Recuperado 25 de octubre de 2025 (vid. pág. 27).
- APEL, S.; BATORY, D.; KÄSTNER, C. y SAAKE, G., 2013. *Feature-Oriented Software Product Lines: Concepts and Implementation*. Springer (vid. págs. 7, 17, 26).
- ARATCHIGE, R.; MANUJAYA, K. y WEERASINGHE, P., 2024. An Overview of Structural Design Patterns in Object-Oriented Software Engineering. *Software Modeling*, págs. 1-3. vid. pág. 38 (vid. pág. 85).
- AUTHORS, Poetry, 2024. *Poetry: Python Dependency Management and Packaging* [<https://python-poetry.org>]. Accessed: 2025-01-10 (vid. pág. 42).
- AWAD, Mohamed A, 2005. A comparison between agile and traditional software development methodologies. *University of Western Australia*. Vol. 30, págs. 1-69 (vid. págs. 46, 47).
- BASS, Len; CLEMENTS, Paul y KAZMAN, Rick, 2012. *Software Architecture in Practice*. 3rd. Addison-Wesley Professional. ISBN 978-0321815736 (vid. págs. 79, 80).
- BATES, A. W., 2019. *Teaching in a Digital Age: Guidelines for Designing Teaching and Learning*. Tony Bates Associates Ltd (vid. pág. 2).
- BECK, Kent, 2000. *Extreme programming explained: embrace change*. Addison-Wesley Professional. Vid. pág. 33 (vid. pág. 76).
- BECK, Kent y ANDRES, Cynthia, 2004. *Extreme Programming Explained: Embrace Change*. 2nd. Addison-Wesley Professional. ISBN 978-0321278654. Capítulo sobre planificación de entregas (vid. pág. 78).
- BEGEL, Andrew, 2020. *GRASP Patterns and Principles Memo* [online]. [visitado 2026-04-16]. Disponible en: <http://cs-comm.lib.muohio.edu/items/show/72> (vid. pág. 85).

- BENAVIDES, D.; SEGURA, S. y RUIZ-CORTÉS, A., 2010. Automated analysis of feature models 20 years later: A literature review. *Information Systems*. Vol. 35, n.º 6, págs. 615-636 (vid. págs. 8, 11, 13, 17, 41).
- BENAVIDES, David; SEGURA, Sergio y RUIZ-CORTÉS, Antonio, 2010. Principles of Feature Modeling. En: *Proceedings of the 7th International Conference on Software Product Lines (SPLC)*. Springer, págs. 13-22 (vid. pág. 8).
- BENAVIDES, David; SUNDERMANN, Chico; FEICHTINGER, Kevin; GALINDO, José A.; RABISER, Rick y THÜM, Thomas, 2025. UVL: Feature modelling with the Universal Variability Language. *Journal of Systems and Software*. Disp. desde doi: [10.1016/j.jss.2024.112326](https://doi.org/10.1016/j.jss.2024.112326) (vid. pág. 14).
- BENAVIDES, David; TRINIDAD, José-Antonio y RUIZ-CORTÉS, Antonio, 2005. A taxonomy of variability constraints for feature models. En: *SPLC*. Springer, págs. 99-108 (vid. págs. 10-12).
- BOETTIGER, Carl, 2015. An introduction to Docker for reproducible research. *ACM SIGOPS Operating Systems Review* (vid. págs. 35, 36).
- CARLSON, Josiah L, 2013. *Redis in Action*. Manning (vid. págs. 36, 37).
- CELERY PROJECT, 2025a. *Celery - Distributed Task Queue Documentation* [online]. [visitado 2026-04-18]. Disponible en: <https://docs.celeryq.dev/> (vid. págs. 38, 39, 83).
- CELERY PROJECT, 2025b. *Celery - First Steps with Celery* [online]. [visitado 2026-04-18]. Disponible en: <https://docs.celeryq.dev/en/stable/getting-started/first-steps-with-celery.html> (vid. pág. 83).
- CELERY PROJECT, 2025c. *Celery - Periodic Tasks* [online]. [visitado 2026-04-18]. Disponible en: <https://docs.celeryq.dev/en/stable/userguide/periodic-tasks.html> (vid. pág. 84).
- CELERY PROJECT, 2025d. *Celery - Using Redis* [online]. [visitado 2026-04-18]. Disponible en: <https://docs.celeryq.dev/en/stable/getting-started/backends-and-brokers/redis.html> (vid. pág. 83).
- CELERY PROJECT, 2025e. *Celery - Workers Guide* [online]. [visitado 2026-04-18]. Disponible en: <https://docs.celeryq.dev/en/stable/userguide/workers.html> (vid. pág. 83).
- CHEN, L.; BABAR, M. A. y ALI, N., 2020. Variability management in software product lines: A systematic review. *ACM Computing Surveys*. Vol. 53, n.º 4, págs. 1-45 (vid. págs. 17, 29).
- CHIMALAKONDA, Sridhar y NORI, Kesav V., 2012. Accelerating educational technologies using software product lines. En: *2012 IEEE International Conference on Technology Enhanced Education (ICTEE)*, págs. 1-4. Disp. desde doi: [10.1109/ICTEE.2012.6208608](https://doi.org/10.1109/ICTEE.2012.6208608) (vid. págs. 18, 19).
- CLEMENTS, Paul; BACHMANN, Felix; BASS, Len; GARLAN, David; IVERS, James; LITTLE, Reed; NORD, Robert y STAFFORD, Judith, 2010. *Documenting Software Architectures: Views and Beyond*. 2nd. Addison-Wesley Professional. ISBN 978-0321552686 (vid. pág. 80).

- COHN, Mike, 2004. *User Stories Applied: For Agile Software Development*. Addison-Wesley Professional. ISBN 978-0321205681. Capítulo 10: Epics (vid. pág. 76).
- COLANTONIO, Alessandro; DI PIETRO, Roberto; OCELLO, Alberto y VERDE, Nino Vincenzo, 2010. Role Mining in Business: Taming Role-Based Access Control Administration. *World Scientific Publishing*. Disp. desde doi: [10.1142/9789814289986_0002](https://doi.org/10.1142/9789814289986_0002) (vid. pág. 54).
- CORPORATION, Microsoft, 2025a. *Visual Studio Code* [<https://code.visualstudio.com/>]. Editor de código fuente multiplataforma, abierto y extensible. Accedido: 2025-12-08 (vid. pág. 45).
- CORPORATION, Microsoft, 2025b. *Visual Studio Code Documentation* [<https://code.visualstudio.com/Docs>]. Documentación oficial. Accedido: 2025-12-08 (vid. pág. 45).
- Database Management System - an overview*, 2024 [ScienceDirect Topics]. [visitado 2026-03-08]. Disponible en: <https://www.sciencedirect.com/topics/biochemistry-genetics-and-molecular-biology/database-management-system> (vid. pág. 34).
- DE MOURA, Leonardo y BJØRNER, Nikolaj, 2008. Z3: An Efficient SMT Solver. En: *TACAS 2008 - Tools and Algorithms for the Construction and Analysis of Systems*. Springer. Disp. desde doi: [10.1007/978-3-540-78800-3_24](https://doi.org/10.1007/978-3-540-78800-3_24) (vid. pág. 41).
- DEL ÁGUILA, L. et al., 2014. La gestión por procesos en las Instituciones de Educación Superior. *UT-Ciencia*. Vol. 1, n.º 3, págs. 140-149. Url: <https://dialnet.unirioja.es/servlet/articulo?codigo=9596104>. [citation:1] (vid. pág. 1).
- DÍAZ, V., 2002. Flexibilidad y educación superior en Colombia. *Instituto Colombiano para el Fomento de la Educación Superior*. Citado en: https://www.mineducacion.gov.co/1621/articles-106706_archivo_pdf.pdf (vid. pág. 18).
- ELLUCIAN COMPANY, 2015. *Banner de Ellucian. Características generales* (vid. pág. 27).
- EUGENE, Michael y ANDERSON, Robert, 2023. FastAPI Framework Performance Analysis for Microservices. *International Journal of Computer Applications* (vid. pág. 33).
- FERRAILOLO, David F.; KUHN, D. Richard y CHANDRAMOULI, Ramaswamy, 2007. *Role-Based Access Control*. Artech House. ISBN 978-1596932069 (vid. pág. 53).
- FOWLER, Martin, 2003. Who Needs an Architect? *IEEE Software*. Vol. 20, n.º 5, págs. 11-13. Disp. desde doi: [10.1109/MS.2003.1231144](https://doi.org/10.1109/MS.2003.1231144) (vid. pág. 80).
- GALÁN, David y OTROS, 2014. Building Families of Software Products for e-Learning Platforms: A Case Study. *IEEE Revista Iberoamericana de Tecnologías del Aprendizaje*. Vol. 9, n.º 2, págs. 64-71. Disp. desde doi: [10.1109/RITA.2014.2317524](https://doi.org/10.1109/RITA.2014.2317524) (vid. pág. 19).

- GALINDO, José A.; HORCAS, Jose-Miguel; FELFERNING, Alexander; FERNANDEZ-AMOROS, David y BENAVIDES, David, 2023. FLAMA: A collaborative effort to build a new framework for the automated analysis of feature models. En: *Proceedings of the 27th ACM International Systems and Software Product Line Conference (SPLC '23) - Tool Track*. Disp. desde doi: [10.1145/3579028.3609008](https://doi.org/10.1145/3579028.3609008) (vid. pág. 28).
- GAMMA, Erich; HELM, Richard; JOHNSON, Ralph y VLISSIDES, John, 1995. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional. ISBN 978-0201633610 (vid. págs. 32, 80, 85-87).
- GEEKSFORGEES, [s.f.]. *What is Feature Engineering?* Url: <https://www.geeksforgeeks.org/machine-learning/what-is-feature-engineering/> (vid. pág. 8).
- GHODKE, G. M. y CHAVAN, T., 2024. An Overview of Git. *International Journal of Scientific Research in Modern Science and Technology*. Vol. 3, n.º 6, págs. 17-23 (vid. págs. 42, 43).
- GÓMEZ, YA, 2023. Innovación educativa y gestión curricular. *Anales de la Real Academia de Doctores. Obtenido de https://www.rade.es/imageslib/PUBLICACIONES/ARTICULOS/V8N3*. Vol. 2 (vid. págs. 1, 2, 18).
- GUTAMA, Jonnathan Fabricio; ORTIZ GONZÁLEZ, Jhordi Joel; MOSCOSO BERNA, Santiago Arturo y AYABACA LANDI, David Santiago, 2025. Diseño de gestión de procesos para la Secretaria y Gestores de la Unidad Académica de Ingeniería Industria y Construcción de la Universidad Católica de Cuenca. *South Florida Journal of Development*. Disp. desde doi: [10.46932/sfjdv6n3-004](https://doi.org/10.46932/sfjdv6n3-004). [citation:4] (vid. pág. 1).
- HAGBERG, Aric A; SCHULT, Daniel A y SWART, Pieter J, 2008. Exploring network structure, dynamics, and function using NetworkX. *SC* (vid. pág. 40).
- HU, Vincent C.; FERRAILOLO, David F. y KUHN, D. Richard, 2006. Assessment of Access Control Systems. En: *National Institute of Standards and Technology (NIST) Interagency Report 7316*. Url: <https://csrc.nist.gov/publications/detail/nistir/7316/final> (vid. pág. 53).
- IBM CORPORATION, 2023. *What is an Integrated Development Environment (IDE)?* IBM. Url: <https://www.ibm.com/think/topics/integrated-development-environment>. Accedido: 10 de diciembre de 2025 (vid. pág. 44).
- INC., Docker, 2024. *Docker Documentation* [<https://docs.docker.com/>]. Accessed: 2025-01-10 (vid. págs. 35, 36).
- INC., GitHub, 2024. *GitHub Documentation* [<https://docs.github.com>]. Accessed: 2025-01-10 (vid. pág. 43).
- INC., Postman, 2024. *Postman API Development Platform: Documentation* [<https://learning.postman.com>]. Accessed: 2025-01-10 (vid. pág. 44).

- INGENIERÍA DE SOFTWARE, [s.f.]. *Líneas de Producto Software para acelerar el desarrollo de aplicaciones relacionadas*. Url: <https://ingenieriadesoftware.es/lineas-producto-software/>. Recuperado 25 de octubre de 2025 (vid. pág. 6).
- INSTRUCTURE, 2025. *Canvas LMS: Enseñanza y aprendizaje en cualquier lugar*. Url: <https://www.instructure.com/canvas>. Recuperado 25 de octubre de 2025 (vid. pág. 27).
- KANG, Kyo C.; COHEN, Sholom; HESS, James A.; NOVAK, William E. y PETERSON, A. Spencer, 1990. *Feature-Oriented Domain Analysis (FODA) Feasibility Study*. Inf. téc., CMU/SEI-90-TR-21. Software Engineering Institute, Carnegie Mellon University (vid. pág. 8).
- KREKEL, Holger y CONTRIBUTORS, 2024. *pytest Documentation* [<https://docs.pytest.org>]. Accessed: 2025-01-10 (vid. pág. 44).
- LARMAN, Craig, 2004. *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development*. 3rd. Prentice Hall. ISBN 978-0131489066 (vid. pág. 85).
- LOELIGER, Jon y MCCULLOUGH, Matthew, 2012. *Version Control with Git*. O'Reilly (vid. pág. 42).
- LTD., Visual Paradigm International, 2024. *Visual Paradigm Documentation* [<https://www.visual-paradigm.com/documentation/>]. Accessed: 2025-01-10 (vid. pág. 44).
- LUBANOVIC, Bill, 2019. *Introducing Python: Modern Computing in Simple Packages*. 2nd. O'Reilly Media (vid. pág. 34).
- LUTZ, Mark, 2013. *Learning Python*. En: *O'Reilly Media* (vid. pág. 31).
- MARSH, Charlie, 2024. *UV: Extremely fast Python package installer* [<https://github.com/astral-sh/uv>]. Accessed: 2025-01-10 (vid. pág. 42).
- MENDONCA, M y WASOWSKI, A, 2009. SAT-based analysis of feature models is easy. En: *SPLC*. ACM, págs. 231-240 (vid. pág. 11).
- MENZINSKY, Alexander; LÓPEZ, Gertrudis; PALACIO, Juan; SOBRINO, Miguel Ángel; ÁLVAREZ, Rubén y RIVAS, Verónica, 2018. Historias de usuario. En: *Ingeniería de requisitos ágil*, pág. 33. Vid. pág. 33 (vid. pág. 76).
- MEURER, Aaron et al., 2017. SymPy: symbolic computing in Python. *PeerJ Computer Science* (vid. pág. 40).
- MinIO Documentation*, 2024. MinIO Inc. Url: <https://min.io/docs>. Accessed: 2025-01-15 (vid. pág. 37).
- MOMJIAN, Bruce, 2001. *PostgreSQL: Introduction and Concepts*. Addison-Wesley (vid. pág. 34).
- MOODLE PROJECT, 2025. *¿Qué es Moodle? Visión general de la plataforma*. Url: <https://moodle.com/es/what-is-moodle/>. Recuperado 25 de octubre de 2025 (vid. págs. 27, 28).

- OFICINA DE TRANSFERENCIA DE RESULTADOS DE INVESTIGACIÓN - UNIVERSIDAD DE SEVILLA, 2025. *Investigadores de la US lideran un nuevo lenguaje universal de variabilidad del software* [online]. [visitado 2026-03-05]. Disponible en: <https://www.investigacion.us.es/noticias/investigadores-de-la-us-lideran-un-nuevo-lenguaje-universal-de-variabilidad-del-software> (vid. pág. 14).
- OKADA, Takahiro; HISAZUMI, Kenji; NAKANISHI, Tsuneo y FUKUDA, Akira, 2009. A Methodology for improving reusability of educational materials using product line software engineering. En: *Proceedings of the 17th International Conference on Computers in Education (ICCE 2009)*, págs. 170-172. SCOPUS ID: 84857079693 (vid. págs. 18, 19).
- OU, Li-Ming y XU, Xun, 2013. Relationship matrix based automatic assembly sequence generation from a CAD model. *Computer-Aided Design*. Vol. 45, n.º 7, págs. 1053-1067. ISSN 0010-4485. Disp. desde doi: <http://dx.doi.org/10.1016/j.cad.2013.04.002> (vid. pág. 88).
- PACIENZIA, J., 2015. *Metodologías de desarrollo de software*. UCA (vid. págs. 45, 46, 79).
- PESCE, Luigi, 2021. Performance Benchmarking of Python Web Frameworks. *Journal of Systems and Software* (vid. pág. 33).
- POHL, K.; BÖCKLE, G. y LINDEN, F. J. van der, 2005. *Software Product Line Engineering: Foundations, Principles and Techniques*. Springer (vid. págs. 7, 8).
- PONUTHORAI, P. K. y LOELIGER, J., 2022. *Version Control with Git*. O'Reilly Media (vid. págs. 42, 43).
- POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2024. *PostgreSQL Official Documentation* [<https://www.postgresql.org/docs/>]. Accessed: 2025-01-10 (vid. pág. 34).
- PRESSMAN, R. S., 2014. *Software Engineering: A Practitioner's Approach*. 8.^a ed. McGraw-Hill Education (vid. págs. 32, 43, 49-51, 59).
- PRIETO, F., 2021. Estimación de esfuerzo en desarrollo de software ágil: Estudio del estado actual en Bogotá. *Iteckne*. Vol. 13 (vid. pág. 77).
- PTC, 2025. *Feature Models and Features: What Are They*. Url: <https://www.ptc.com/en/blogs/alm/modelling-features>. Recuperado 25 de octubre de 2025 (vid. pág. 8).
- PURE-SYSTEMS GMBH, 2024. *pure::variants Product Overview*. Url: <https://www.pure-systems.com/en/products/purevariants>. Recuperado 25 de octubre de 2025 (vid. pág. 28).
- RAMEL, David, 2021. New Open Source Index: VS Code Is No. 1 Code Editor. *Visual Studio Magazine* (vid. pág. 45).
- RAMÍREZ, Sebastián, 2019. *FastAPI: Modern, fast (high-performance) web framework for building APIs with Python 3.6+* [<https://fastapi.tiangolo.com>]. Accessed: 2025-01-10 (vid. págs. 37, 39).
- RAMÍREZ, Sebastián, 2021. FastAPI: The right tool for your Python API? *Python Papers Monograph*. Vol. 1, págs. 45-52 (vid. pág. 33).

- RAMÍREZ, Sebastián, 2023. *FastAPI Documentation*. Url: <https://fastapi.tiangolo.com>. Accedido: 2024 (vid. pág. 34).
- RAY, Baishakhi; POSNETT, Daryl; FILKOV, Vladimir y DEVANBU, Premkumar, 2014. A large-scale study of programming languages and code quality in GitHub. *Communications of the ACM* (vid. págs. 31, 32).
- REFACTORING GURU, 2021. *Catalog of Design Patterns* [online]. [visitado 2026-04-17]. Disponible en: <https://refactoring.guru/design-patterns/catalog> (vid. pág. 86).
- RODRÍGUEZ, P.; VIZCAÍNO, A. y PIATTINI, M., 2019. Herramientas de gestión curricular: Una evaluación de la situación actual. *IEEE Revista Iberoamericana de Tecnologías del Aprendizaje*. Vol. 14, n.º 3, págs. 112-120 (vid. pág. 17).
- ROJAS, Juan Carlos y PÉREZ, Miguel Ángel, 2021. *Arquitectura de Software: Fundamentos y Prácticas*. Alfaomega. ISBN 978-6075381752 (vid. pág. 80).
- RUMBAUGH, James; JACOBSON, Ivar y BOOCH, Grady, 2004. *Unified Modeling Language Reference Manual*. Pearson (vid. pág. 43).
- SALTZER, Jerome H. y SCHROEDER, Michael D., 1975. The Protection of Information in Computer Systems. *Proceedings of the IEEE*. Vol. 63, n.º 9, págs. 1278-1308. Disp. desde doi: [10.1109/PROC.1975.9939](https://doi.org/10.1109/PROC.1975.9939) (vid. pág. 54).
- SÁNCHEZ, J.; GUTIÉRREZ, F. L. y CABRERA, M., 2020. Hacia una metodología para la gestión de la variabilidad en planes de estudio de ingeniería de software. *Journal of Educational Computing Research*. Vol. 58, n.º 5, págs. 1020-1045 (vid. págs. 2, 17, 27, 29).
- SANDHU, Ravi S.; COYNE, Edward J.; FEINSTEIN, Hal L. y YOUMAN, Charles E., 1996. Role-Based Access Control Models. *IEEE Computer*. Vol. 29, n.º 2, págs. 38-47. Disp. desde doi: [10.1109/2.485845](https://doi.org/10.1109/2.485845) (vid. pág. 53).
- SEBESTA, Robert W., 2016. *Concepts of Programming Languages*. 11.ª ed. Pearson. ISBN 978-0133943023 (vid. pág. 31).
- SEGURIDAD PY, 2022. *Patrones de diseño: Qué son, tipos y porqué los debes conocer* [online]. [visitado 2026-04-17]. Disponible en: <https://seguridadpy.com/patrones-de-diseno-que-son-tipos/> (vid. págs. 86, 87).
- SERNA-MONTOYA, Édgar, 2012. Estado actual de la investigación en requisitos no funcionales. *Ingeniería y Universidad*. Vol. 16, n.º 1, págs. 225-246. Ver p.ágina 32 (vid. pág. 73).
- SG BUZZ, [s.f.]. *Líneas de Productos de Software*. Url: <https://sg.com.mx/revista/34/lineas-productos-software>. Recuperado 25 de octubre de 2025 (vid. pág. 5).

- SILVA MARCOLINO, Anderson y FRANCINE BARBOSA, Ellen, 2017. Towards a Software Product Line Architecture to Build M-learning Applications for the Teaching of Programming. En: *Proceedings of the 50th Hawaii International Conference on System Sciences (HICSS 2017)*. Disp. desde doi: [10125/41922](https://doi.org/10.125/41922) (vid. págs. 18, 19).
- SOLIS PINO, Andrés Felipe; RUIZ, Pablo H. y HURTADO ALEGRIA, Julio Ariel, 2022. A Software Products Line as Educational Tool to Learn Industrial Robots Programming with Arduino. *Electronics*. Vol. 11, n.º 5, pág. 769. Disp. desde doi: [10.3390/electronics11050769](https://doi.org/10.3390/electronics11050769) (vid. pág. 19).
- SOMMERVILLE, I., 2016. *Software Engineering*. 10.^a ed. Pearson Education (vid. págs. 31, 32, 44).
- SOMMERVILLE, Ian, 2005. *Software Engineering*. 8.^a ed. Addison-Wesley. ISBN 978-0321210250 (vid. pág. 73).
- SOOS, Mate; GOCHT, Stephan y AL., et, 2020. Ganak: A scalable probabilistic model counter. En: *SAT Conference*. Springer (vid. pág. 11).
- SOTO GRANT, Andrea, 2022. La gestión por procesos como herramienta fundamental en el aseguramiento de la calidad de las carreras universitarias. *Revista Electrónica Actualidades Investigativas en Educación*. Vol. 22, n.º 2, págs. 1-24. Disp. desde doi: [10.15517/aie.v22i2.48906](https://doi.org/10.15517/aie.v22i2.48906). [citation:2] (vid. pág. 1).
- SUNDERMANN, C.; DÖRR, L.; JUNKER, M. y KEHRER, T., 2023. Evaluating state-of-the-art #SAT solvers on industrial configuration spaces. *Empirical Software Engineering*. Vol. 28, n.º 29 (vid. págs. 10, 11).
- SUNDERMANN, Chico et al., 2021. UVL: A uniform variability language. En: *Proceedings of the 25th ACM International Systems and Software Product Line Conference*, págs. 2-5 (vid. pág. 32).
- SUNDERMANN, Chico; FEICHTINGER, Kevin; ENGELHARDT, Dominik; RABISER, Rick y THÜM, Thomas, 2021. Yet another textual variability language? a community effort towards a unified language. En: *Proceedings of the 25th ACM International Systems and Software Product Line Conference (SPLC '21)*. Disp. desde doi: [10.1145/3461001.3471145](https://doi.org/10.1145/3461001.3471145) (vid. pág. 14).
- SUNDERMANN, Chico; VILL, Stefan; THÜM, Thomas; FEICHTINGER, Kevin; AGARWAL, Prankur; RABISER, Rick; GALINDO, José A. y BENAVIDES, David, 2023. UVLParser: Extending UVL with Language Levels and Conversion Strategies. En: *Proceedings of the 27th ACM International Systems and Software Product Line Conference (SPLC '23) - Tool Track*. Disp. desde doi: [10.1145/3579028.3609013](https://doi.org/10.1145/3579028.3609013) (vid. pág. 14).
- TEAM, Flamapy, 2024. *FLAMAPY rest API* [online]. [visitado 2026-03-08]. Disponible en: <https://pypi.org/project/flamapy-rest/> (vid. pág. 29).
- TEAM, Flamapy, 2025a. *FeatureIDE Reader - Flamapy Documentation* [online]. [visitado 2026-03-08]. Disponible en: https://docs.flamapy.org/framework/transformations/featureide_reader (vid. pág. 29).

- TEAM, Flamapy, 2025b. *Flamapy Documentation* [online]. [visitado 2026-03-08]. Disponible en: <https://docs.flamapy.org/> (vid. pág. 29).
- TEAM, Flamapy, 2025c. *FM to BDD - Flamapy Documentation* [online]. [visitado 2026-03-08]. Disponible en: https://docs.flamapy.org/framework/transformations/fm_to_bdd (vid. pág. 29).
- THÜM, T.; KÄSTNER, C.; BENDUHN, F.; MEINICKE, J.; SAAKE, G. y LEICH, T., 2014. FeatureIDE: An extensible framework for feature-oriented software desarrollo. *Science of Computer Programming*. Vol. 79, págs. 70-85 (vid. págs. 11, 28).
- UNESCO, 2022. *Reimagining our futures together: A new social contract for education*. UNESCO (vid. pág. 1).
- Unified Modeling Language (UML) Specification*, 2017. Object Management Group. Url: <https://www.omg.org/spec/UML>. Accedido: 10 de diciembre de 2025 (vid. pág. 43).
- UVL COMMUNITY, [s.f.]. *Universal Variability Language (UVL): Community-Driven Language for Variability Models* [online]. [visitado 2026-03-05]. Disponible en: <https://universal-variability-language.github.io/> (vid. págs. 14, 15).
- VAN ROSSUM, Guido y DRAKE, Fred L, 1995. *Python Tutorial*. Centrum voor Wiskunde en Informatica (CWI) (vid. pág. 31).
- VEIGA MÉNDEZ, Antonio José, 2024. Norma ISO 9001 en Universidades: Un Enfoque para Mejorar la Gestión de la Calidad Académica. *UP•NIVERSITY Perspectivas*. Url: <https://es.linkedin.com/pulse/norma-iso-9001-en-universidades-un-enfoque-para-la-de-veiga-m%C3%A9ndez-q70uf>. [citation:3] (vid. pág. 1).
- WANG, Hongyu; ZHAO, Lu y HU, Jun, 2007. Formal Semantics and Verification for Feature Modeling. *Computer Software and Applications Conference (COMPSAC)*, págs. 148-155 (vid. pág. 8).
- WELLS, Don, 2009. *Extreme Programming: A Gentle Introduction* [online]. [visitado 2026-04-15]. Disponible en: <http://www.extremeprogramming.org/> (vid. pág. 78).
- ZHAO, Han; LIFFITON, Michael; EIJK, Patrick; MALIKOV, Alexey y PREVITI, Alessandro, 2018. Py-SAT: A Python Toolkit for SAT-Based Prototyping. En: *SAT 2018 - 21st International Conference on Theory and Applications of Satisfiability Testing*. Springer. Disp. desde doi: [10.1007/978-3-319-94144-8_38](https://doi.org/10.1007/978-3-319-94144-8_38) (vid. pág. 41).

Apéndices

Historias de usuario

Épica 1: Gestión de Identidad y Acceso

Cuadro A.1. Historia de usuario # 2

Historia de usuario	
Número: 2	Nombre: Autenticación y Autogestión de cuenta
Épica: Gestión de Identidad y Acceso	
Prioridad en negocio: Alta	Riesgo en desarrollo: Media
Puntos estimados: 0.4	
RFs cubiertos: 1, 2, 3, 4, 5, 8	
Programador responsable: Jose Luis Echemendia López	
Descripción: El usuario del sistema debe poder iniciar sesión, cerrar sesión, gestionar su perfil y recuperar su acceso para garantizar la seguridad de sus datos y su identidad en la plataforma.	
Criterios de aceptación: • Permitir registro e inicio con correo y contraseña. • Procesar recuperación de acceso mediante <i>token</i> enviado por correo. • Permitir la edición del perfil propio sin intervención administrativa.	
Observaciones: La historia concentra autenticación, recuperación de acceso y autogestión del perfil como capacidades base del módulo de identidad. Campos de entrada asociados: RF#1 y RF#3 (correo electrónico, contraseña), RF#5 (token y nueva contraseña; o contraseña actual y nueva contraseña), RF#8 (correo electrónico opcional).	

Cuadro A.2. Historia de usuario # 3

Historia de usuario	
Número: 3	Nombre: Administración de Usuarios y Permisos
Épica: Gestión de Identidad y Acceso	
Prioridad en negocio: Alta	Riesgo en desarrollo: Media
Puntos estimados: 0.3	
RFs cubiertos: 6, 7, 9, 10, 11	
Programador responsable: Jose Luis Echemendia López	
Descripción: El administrador debe poder gestionar el listado de usuarios, sus roles y su estado activo o inactivo para controlar quién accede a la plataforma y qué acciones puede realizar según la jerarquía estática.	
Criterios de aceptación: • Listar y filtrar usuarios por rol o nombre. • Modificar el rol asignado a cada usuario. • Desactivar y activar cuentas sin eliminar la información persistida.	
Observaciones: La gestión de cuentas debe preservar la trazabilidad administrativa y evitar borrados físicos sobre usuarios existentes. Campos de entrada asociados: RF#9 (identificador de usuario en ruta y rol en consulta).	

Épica 2: Gobernanza de Dominios Curriculares

Cuadro A.3. Historia de usuario # 4

Historia de usuario	
Número: 4	Nombre: Gestión de Dominios de Conocimiento
Épica: Gobernanza de Dominios Curriculares	
Prioridad en negocio: Alta	Riesgo en desarrollo: Media
Puntos estimados: 0.4	
RFs cubiertos: 12, 13, 14, 15, 16, 17, 18, 19	
Programador responsable: Jose Luis Echemendia López	
Descripción: El administrador debe poder crear, actualizar y supervisar los dominios curriculares para establecer el contexto de alto nivel donde se alojarán los FM .	

Continúa en la próxima página

Cuadro A.3. Continuación de la página anterior.

Criterios de aceptación: • Crear dominios con nombre y descripción. • Activar y desactivar dominios conforme a su estado de uso. • Obtener cada dominio junto con sus modelos asociados.
Observaciones: El dominio funciona como unidad de organización para agrupar y consultar los modelos vinculados. Campos de entrada asociados: RF#14 (nombre, descripción opcional) y RF#15 (identificador de dominio en ruta; nombre y descripción opcionales).

Épica 3: Ciclo de Vida del FM

Cuadro A.4. Historia de usuario # 5

Historia de usuario	
Número: 5	Nombre: Gestión Operativa de FM
Épica: Ciclo de Vida del FM	
Prioridad en negocio: Alta	Riesgo en desarrollo: Alta
Puntos estimados: 1.0	
RFs cubiertos: 20, 21, 22, 23, 24, 25	
Programador responsable: Jose Luis Echemendia López	
Descripción: El diseñador de modelo debe poder crear, listar y modificar los metadatos de los modelos de características dentro de un dominio para mantener un catálogo organizado de planes de estudio o arquitecturas curriculares.	
Criterios de aceptación: • Crear modelos asociados a un dominio válido. • Actualizar metadatos preservando la integridad del modelo. • Ofrecer paginación y filtrado para la consulta de modelos.	
Observaciones: Esta historia cubre la administración operativa del catálogo de modelos, sin entrar todavía en su estructura interna. Campos de entrada asociados: RF#21 (nombre, descripción opcional, identificador de dominio) y RF#23 (identificador de modelo en ruta; nombre, descripción y estado activo opcionales).	

Cuadro A.5. Historia de usuario # 6

Historia de usuario	
Número: 6	Nombre: Gestión de Versiones y Publicación
Épica: Ciclo de Vida del FM	
Prioridad en negocio: Alta	Riesgo en desarrollo: Alta
Puntos estimados: 2.5	
RFs cubiertos: 26, 27, 28, 29, 30, 31, 32	
Programador responsable: Jose Luis Echemendia López	
Descripción: El diseñador de modelo y el revisador deben poder crear versiones, publicar instantáneas y archivar versiones antiguas para garantizar la trazabilidad y evolución del currículo sin afectar configuraciones previas.	
Criterios de aceptación: • Generar nuevas versiones mediante técnica de copia al escribir. • Validar la versión antes de su publicación oficial. • Permitir la recuperación de versiones archivadas.	
Observaciones: La publicación debe conservar el historial del modelo y permitir volver a estados previos sin pérdida de información. Campos de entrada asociados: RF#28 (identificador de modelo en ruta; identificador de versión origen y descripción opcionales).	

Cuadro A.6. Historia de usuario # 7

Historia de usuario	
Número: 7	Nombre: Análisis de Métricas y Exportación
Épica: Ciclo de Vida del FM	
Prioridad en negocio: Media	Riesgo en desarrollo: Media
Puntos estimados: 0.8	
RFs cubiertos: 33, 34	
Programador responsable: Jose Luis Echemendia López	

Continúa en la próxima página

Cuadro A.6. Continuación de la página anterior.

Descripción: El diseñador y el observador deben poder obtener estadísticas y exportar el modelo en múltiples formatos para compartirlo con otras herramientas o analizar su complejidad. Los formatos a soportar son: • UVL. • XML. • TVL. • DIMACS. • JSON. • DOT. • mermaid.
Criterios de aceptación: • Calcular métricas de la versión publicada más reciente o de una versión específica. • Exportar el modelo en formatos técnicos como UVL y dimacs. • Exportar el modelo en formatos visuales como mermaid y dot. • Guardar el resultado de la exportación en un archivo.
Observaciones: La exportación debe ser consistente con la versión consultada y servir tanto para intercambio técnico como para inspección visual.

Épica 4: Modelado de Variabilidad Estructural

Cuadro A.7. Historia de usuario # 8

Historia de usuario	
Número: 8	Nombre: Edición de la Jerarquía de Características
Épica: Modelado de Variabilidad Estructural	
Prioridad en negocio: Alta	Riesgo en desarrollo: Alta
Puntos estimados: 1.0	
RFs cubiertos: 53, 54, 55, 56, 60, 61, 62, 63	
Programador responsable: Jose Luis Echemendia López	
Descripción: El diseñador y el editor de modelo deben poder crear, mover, actualizar y desactivar características para construir el árbol de variabilidad que representa el plan de estudios.	

Continúa en la próxima página

Cuadro A.7. Continuación de la página anterior.

Criterios de aceptación: • Reubicar una <i>feature</i> cambiando su padre sin perder consistencia. • Recuperar subárboles completos de una característica. • Aplicar desactivación lógica con soporte de versionado.
Observaciones: Las operaciones sobre la jerarquía deben preservar integridad estructural y trazabilidad de cambios. Campos de entrada asociados: RF#53 (nombre, tipo, identificador de versión, y opcionales de padre, grupo, recurso y propiedades) y RF#60 (identificador de característica en ruta; nombre, tipo, padre y grupo opcionales).

Cuadro A.8. Historia de usuario # 9

Historia de usuario	
Número: 9	Nombre: Modelado de Grupos y Relaciones Estructurales
Épica: Modelado de Variabilidad Estructural	
Prioridad en negocio: Alta	Riesgo en desarrollo: Alta
Puntos estimados: 1.0	
RFs cubiertos: 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75	
Programador responsable: Jose Luis Echemendia López	
Descripción: El diseñador y el editor de modelo deben poder definir grupos de características, como XOR y OR, y relaciones jerárquicas para restringir cómo se pueden combinar los módulos educativos.	
Criterios de aceptación: • Crear grupos con validaciones sobre el tipo de agrupación. • Definir relaciones semánticas entre características. • Permitir la definición de cardinalidades mínimas y máximas. • Mantener consistencia al activar, desactivar o actualizar dichas estructuras.	
Observaciones: Esta historia cubre la semántica estructural del árbol y la coherencia entre agrupaciones y relaciones. Campos de entrada asociados: RF#66 (tipo de grupo, característica padre, cardinalidades), RF#67 (identificador de grupo en ruta y campos opcionales), RF#72 (tipo, origen, destino) y RF#73 (identificador de relación en ruta y campos opcionales).	

Épica 5: Restricciones y Lógica Avanzada

Cuadro A.9. Historia de usuario # 10

Historia de usuario	
Número: 10	Nombre: Gestión de Restricciones Transversales
Épica: Restricciones y Lógica Avanzada	
Prioridad en negocio: Alta	Riesgo en desarrollo: Alta
Puntos estimados: 0.8	
RFs cubiertos: 76, 77, 78, 79, 80, 81	
Programador responsable: Jose Luis Echemendia López	
Descripción: El diseñador de modelo debe poder crear y gestionar reglas de exclusión e implicación complejas para modelar dependencias que no son jerárquicas, por ejemplo, si se elige A, no se puede elegir B.	
Criterios de aceptación: • Permitir crear restricciones complejas con validación de sintaxis. • Permitir expresiones lógicas complejas. • Mantener control de versiones sobre las restricciones creadas.	
Observaciones: Las restricciones deben integrarse con el modelo sin romper la evolución histórica del mismo. Campos de entrada asociados: RF#78 (identificador de versión, expresión lógica y descripción opcional) y RF#79 (identificador de restricción en ruta; expresión lógica y descripción opcionales).	

Cuadro A.10. Historia de usuario # 11

Historia de usuario	
Número: 11	Nombre: Integración y Sincronización UVL
Épica: Restricciones y Lógica Avanzada	
Prioridad en negocio: Alta	Riesgo en desarrollo: Alta
Puntos estimados: 1.0	
RFs cubiertos: 36, 37, 38, 39	
Programador responsable: Jose Luis Echemendia López	
Descripción: El diseñador deben poder operar directamente sobre el lenguaje UVL o sincronizarlo desde la interfaz gráfica para mantener la potencia del modelado por código y la facilidad del modelado visual.	
Criterios de aceptación: • Sincronizar el modelo visual al editar el código UVL. • Persistir el UVL analizado y validado. • Aplicar UVL a la estructura para regenerar el árbol desde el código UVL.	

Continúa en la próxima página

Cuadro A.10. Continuación de la página anterior.

Observaciones: La sincronización debe mantener equivalencia entre la representación textual y la representación visual del modelo. Campos de entrada asociados: RF#36 y RF#38 (identificador de modelo y versión en ruta, contenido [UVL](#)).

Épica 6: Validación y Análisis de Modelos

Cuadro A.11. Historia de usuario # 12

Historia de usuario	
Número: 12	Nombre: Validación Lógica y Estructural del Modelo
Épica: Validación y Análisis de Modelos	
Prioridad en negocio: Alta	Riesgo en desarrollo: Alta
Puntos estimados: 1.0	
RFs cubiertos: 39, 40, 41, 42	
Programador responsable: Jose Luis Echemendia López	
Descripción: El revisador y el diseñador deben poder ejecutar análisis de consistencia, detectar características muertas y redundancias para asegurar que el plan de estudios sea lógicamente posible y no presente errores.	
Criterios de aceptación: • Evaluar la satisfacibilidad global del modelo mediante un solucionador SAT. • Detectar ciclos y nodos huérfanos. • Identificar características muertas y redundancias.	
Observaciones: El análisis debe advertir problemas estructurales y semánticos antes de publicar o derivar configuraciones.	

Épica 7: Configuración, Optimización y Recursos

Cuadro A.12. Historia de usuario # 13

Historia de usuario	
Número: 13	Nombre: Gestión de Configuraciones
Épica: Configuración, Optimización y Recursos	
Prioridad en negocio: Alta	Riesgo en desarrollo: Alta
Puntos estimados: 1.0	
RFs cubiertos: 44, 45, 46, 47, 48, 49	

Continúa en la próxima página

Cuadro A.12. Continuación de la página anterior.

Programador responsable: Jose Luis Echemendia López
Descripción: El configurador debe poder crear, actualizar y persistir selecciones de características para generar itinerarios educativos específicos para un perfil o estudiante.
Criterios de aceptación: • Guardar configuraciones asociadas a una versión publicada. • Ofrecer listado paginado de configuraciones almacenadas. • Mantener operaciones de actualización y activación sobre configuraciones persistidas.
Observaciones: La configuración debe quedar ligada a una versión concreta del modelo para conservar su validez histórica. Campos de entrada asociados: RF#44 (nombre, descripción opcional, identificador de versión y lista de características) y RF#47 (identificador de configuración en ruta; nombre, descripción y lista de características opcionales).

Cuadro A.13. Historia de usuario # 14

Historia de usuario	
Número: 14	Nombre: Validación y Generación Automática de Configuraciones
Épica: Configuración, Optimización y Recursos	
Prioridad en negocio: Alta	Riesgo en desarrollo: Alta
Puntos estimados: 1.0	
RFs cubiertos: 49, 50, 51, 52	
Programador responsable: Jose Luis Echemendia López	
Descripción: El configurador y el estudiante deben poder validar si una selección es consistente y generar automáticamente opciones válidas u optimizadas para encontrar el mejor camino formativo sin violar prerequisites.	
Criterios de aceptación: • Validar selecciones sin necesidad de persistirlas. • Ejecutar algoritmos de optimización para proponer mejores configuraciones.	
Observaciones: La generación automática debe apoyar la toma de decisiones sin obligar a guardar resultados intermedios.	

Cuadro A.14. Historia de usuario # 15

Historia de usuario	
Número: 15	Nombre: Enriquecimiento con Recursos y Taxonomías
Épica: Configuración, Optimización y Recursos	
Prioridad en negocio: Media	Riesgo en desarrollo: Media
Puntos estimados: 0.8	
RFs cubiertos: 57, 58, 59, 82, 83, 84, 85, 86, 87, 88	
Programador responsable: Jose Luis Echemendia López	
Descripción: El editor de modelo y el configurador deben poder etiquetar características y vincular recursos externos, como archivos o enlaces, para añadir contenido pedagógico real a la estructura lógica del modelo.	
Criterios de aceptación: • Asociar y desasociar etiquetas a características. • Cargar recursos y asignar propietarios. • Mantener versionada la información asociada a los recursos. • Aceptar metadatos como idioma, licencia y otros de interés en el registro de los recursos.	
Observaciones: Los recursos y etiquetas deben servir como apoyo semántico y documental para el uso del modelo. Campos de entrada asociados: RF#84 (nombre, descripción opcional), RF#87 (metadatos completos del recurso y contenido) y RF#88 (identificador de recurso en ruta y campos opcionales, con restricción sobre propietario).	

Tarjetas CRC

Cuadro B.1. Tarjeta CRC # 2

Tarjeta CRC	
Clase: User	
Responsabilidad	Colaboración
• Representar usuarios y roles. • Mantener estado activo/inactivo. • Soportar auditoría (creado/actualizado/eliminado por).	FeatureModel (owner/collaborator) FeatureModelCollaborator Repositorios de usuario

Cuadro B.2. Tarjeta CRC # 3

Tarjeta CRC	
Clase: Domain	
Responsabilidad	Colaboración
• Agrupar modelos por dominio. • Mantener metadatos (nombre, descripción). • Gestionar relación con modelos.	FeatureModel

Cuadro B.3. Tarjeta CRC # 4

Tarjeta CRC
Clase: FeatureModel

Continúa en la próxima página

Cuadro B.3. Continuación de la página anterior.

Responsabilidad	Colaboración
- Representar un FM (nombre, descripción, dominio). - Enlazar propietario y versiones. - Mantener colaboradores.	Domain User FeatureModelVersion FeatureModelCollaborator

Cuadro B.4. Tarjeta CRC # 5

Tarjeta CRC	
Clase: FeatureModelVersion	
Responsabilidad	Colaboración
- Versionar un FM (estado, snapshot, UVL). - Enlazar estructura completa (features, grupos, relaciones, restricciones y configuraciones).	FeatureModel Feature FeatureGroup FeatureRelation Constraint Configuration

Cuadro B.5. Tarjeta CRC # 6

Tarjeta CRC	
Clase: Feature	
Responsabilidad	Colaboración
- Representar característica (nombre, tipo, propiedades). - Mantener jerarquía padre/hijos. - Enlazar grupo y recurso. - Soportar tags y configuraciones.	FeatureModelVersion FeatureGroup FeatureRelation Tag Configuration Resource

Cuadro B.6. Tarjeta CRC # 7

Tarjeta CRC
Clase: FeatureGroup

Continúa en la próxima página

Cuadro B.6. Continuación de la página anterior.

Responsabilidad	Colaboración
- Definir agrupaciones XOR/OR. - Controlar cardinalidades. - Enlazar feature padre y miembros.	FeatureModelVersion Feature

Cuadro B.7. Tarjeta CRC # 8

Tarjeta CRC	
Clase: FeatureRelation	
Responsabilidad	Colaboración
Representar relaciones cross-tree (requires/excludes).	FeatureModelVersion Feature

Cuadro B.8. Tarjeta CRC # 9

Tarjeta CRC	
Clase: Constraint	
Responsabilidad	Colaboración
Persistir restricciones lógicas complejas (expr_text, expr_cnf).	FeatureModelVersion

Cuadro B.9. Tarjeta CRC # 10

Tarjeta CRC	
Clase: Configuration	
Responsabilidad	Colaboración
- Persistir selección de features. - Enlazar tags. - Mantener estado activo/inactivo.	FeatureModelVersion Feature Tag

Cuadro B.10. Tarjeta CRC # 11

Tarjeta CRC	
Clase: Tag	
Responsabilidad	Colaboración
• Clasificar features/configuraciones. • Mantener unicidad por nombre.	Feature Configuration

Cuadro B.11. Tarjeta CRC # 12

Tarjeta CRC	
Clase: Resource	
Responsabilidad	Colaboración
• Representar recursos educativos. • Mantener metadatos de contenido/licencia. • Enlazar propietario.	User Feature

Cuadro B.12. Tarjeta CRC # 13

Tarjeta CRC	
Clase: FeatureModelVersionManager	
Responsabilidad	Colaboración
• Crear versiones (copy-on-write). • Publicar/archivar/restaurar versiones. • Generar snapshot y validaciones de transición.	FeatureModelVersionRepository FeatureModelVersion Feature User FeatureModel

Cuadro B.13. Tarjeta CRC # 14

Tarjeta CRC	
Clase: FeatureModelExportService	
Responsabilidad	Colaboración

Continúa en la próxima página

Cuadro B.13. Continuación de la página anterior.

• Exportar FM a UVL/XML/TVL/DIMACS/JSON/DOT/Mezclador • Construir mapeo UUID ↔ int.	FeatureModelVersion Feature Constraint Enums de exportación
--	--

Cuadro B.14. Tarjeta CRC # 15

Tarjeta CRC	
Clase: FeatureModelUVLImporter	
Responsabilidad	Colaboración
• Validar UVL. • Construir estructura desde UVL. • Crear relaciones/restricciones y nueva versión.	FeatureModelVersionManager Feature FeatureGroup FeatureRelation Constraint FeatureModelVersion User

Cuadro B.15. Tarjeta CRC # 16

Tarjeta CRC	
Clase: FeatureModelLogicalValidator	
Responsabilidad	Colaboración
• Ejecutar validación SAT/SMT (sympy/pysat/z3). • Verificar consistencia global y configuraciones.	InvalidConstraintException y otras excepciones Dependencias SAT/SMT

Cuadro B.16. Tarjeta CRC # 17

Tarjeta CRC	
Clase: FeatureModelConfigurationGenerator	
Responsabilidad	Colaboración

Continúa en la próxima página

Cuadro B.16. Continuación de la página anterior.

• Generar configuraciones válidas. • Aplicar estrategias GREEDY/RANDOM/BEAM/GENETIC/NSGA2.	FeatureModelLogicalValidator GenerationStrategy Dependencias DEAP/OR-Tools/BDD
---	--

Cuadro B.17. Tarjeta CRC # 18

Tarjeta CRC	
Clase: FeatureModelStructuralAnalyzer	
Responsabilidad	Colaboración
• Analizar estructura (dead features, redundancias, SCC). • Calcular métricas de complejidad.	AnalysisType networkx Excepciones estructurales

Cuadro B.18. Tarjeta CRC # 19

Tarjeta CRC	
Clase: FeatureModelTreeBuilder	
Responsabilidad	Colaboración
• Construir árbol completo para respuesta. • Generar estadísticas. • Generar UVL efectivo.	FeatureModelVersion Feature FeatureGroup FeatureModelExportService FeatureModelCompleteResponse

Cuadro B.19. Tarjeta CRC # 20

Tarjeta CRC	
Clase: FeatureModelAnalysisFacade	
Responsabilidad	Colaboración
• Orquestar análisis lógico y estructural. • Calcular commonality/core/atomic sets. • Ejecutar validación UVL opcional (Flamapy).	FeatureModelLogicalValidator FeatureModelStructuralAnalyzer FeatureModelUVLImporter

Cuadro B.20. Tarjeta CRC # 21

Tarjeta CRC	
Clase: SettingsService	
Responsabilidad	Colaboración
• Consultar y actualizar settings. • Cachear en Redis.	AppSetting redis_client

Cuadro B.21. Tarjeta CRC # 22

Tarjeta CRC	
Clase: AppSetting	
Responsabilidad	Colaboración
• Persistir claves de configuración. • Mantener descripción y valor.	SettingsService Redis cache

Proyectos y Avaless



REPORTE DEL PLANIFICADOR DE SECUENCIA DE ENSAMBLE

Ing. Maybel Díaz Capote
Universidad de las Ciencias Informáticas, La Habana, Cuba
Fecha: sábado 5 de octubre de 2013

DETALLES DEL REPORTE:

Caso de estudio: Bloque Corona
Reorientaciones: sí
Cambios de herramientas: no
Vértices: 17
Aristas: 136
Número total de hormigas: 11

Secuencias de ensamble:

Hormiga: 1
Reorientaciones: 2

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(1, +z)

Hormiga: 2
Reorientaciones: 3

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(7, +y)-(17, +y)-(1, +y)

Hormiga: 3
Reorientaciones: 3

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(3, +z)-(6, +z)-(17, +z)-(1, +z)-(13, +z)-(7, +y)-(4, +y)-(2, +y)

Hormiga: 4
Reorientaciones: 2

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(13, +z)-(1, +z)-(6, +z)-(3, +z)-(2, +z)

Hormiga: 5
Reorientaciones: 3

Salida:
(11, +z)-(10, +z)-(16, +x)-(14, +x)-(15, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(1, +z)-(2, +z)-(13, +z)-(8, +x)-(3, +x)

Hormiga: 6
Reorientaciones: 2

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(1, +z)-(6, +z)-(2, +z)-(13, +z)-(3, +z)

Otro reporte

D.1. Otra sección de muestra

De una sola sentencia

D.2. Otro ensamble Industrial



REPORTE DEL PLANIFICADOR DE SECUENCIA DE ENSAMBLE

Ing. Maybel Díaz Capote
Universidad de las Ciencias Informáticas, La Habana, Cuba
Fecha: sábado 5 de octubre de 2013

DETALLES DEL REPORTE:

Caso de estudio: Bloque Corona
Reorientaciones: sí
Cambios de herramientas: no
Vértices: 17
Aristas: 136
Número total de hormigas: 11

Secuencias de ensamble:

Hormiga: 1
Reorientaciones: 2

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(1, +z)

Hormiga: 2
Reorientaciones: 3

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(7, +z)-(17, +z)-(1, +z)

Hormiga: 3
Reorientaciones: 3

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(3, +z)-(6, +z)-(17, +z)-(1, +z)-(13, +z)-(7, +z)-(4, +z)-(2, +z)

Hormiga: 4
Reorientaciones: 2

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(13, +z)-(1, +z)-(6, +z)-(3, +z)-(2, +z)

Hormiga: 5
Reorientaciones: 3

Salida:
(11, +z)-(10, +z)-(16, +x)-(14, +x)-(15, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(1, +z)-(2, +z)-(13, +z)-(8, +x)-(3, +x)

Hormiga: 6
Reorientaciones: 2

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(1, +z)-(6, +z)-(2, +z)-(13, +z)-(3, +z)



REPORTE DEL PLANIFICADOR DE SECUENCIA DE ENSAMBLE

Ing. Maybel Díaz Capote
Universidad de las Ciencias Informáticas, La Habana, Cuba
Fecha: sábado 5 de octubre de 2013

DETALLES DEL REPORTE:

Caso de estudio: Bloque Corona
Reorientaciones: sí
Cambios de herramientas: no
Vértices: 17
Aristas: 136
Número total de hormigas: 11

Secuencias de ensamble:

Hormiga: 1

Reorientaciones: 2

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(17, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(1, +z)

Hormiga: 2

Reorientaciones: 3

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(17, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(7, -y)-(17, -y)-(1, -y)

Hormiga: 3

Reorientaciones: 3

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(3, +z)-(6, +z)-(17, +z)-(1, +z)-(13, +z)-(7, +y)-(4, +y)-(2, +y)

Hormiga: 4

Reorientaciones: 2

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(13, +z)-(1, +z)-(6, +z)-(3, +z)-(2, +z)

Hormiga: 5

Reorientaciones: 3

Salida:

(11, +z)-(10, +z)-(16, +x)-(14, +x)-(15, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(1, +z)-(2, +z)-(13, +z)-(8, +x)-(3, +x)

Hormiga: 6

Reorientaciones: 2

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(1, +z)-(6, +z)-(2, +z)-(13, +z)-(3, +z)

Hormiga: 7

Reorientaciones: 2

Salida:

(8, -x)-(14, -x)-(15, -x)-(16, -x)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(2, +z)-(13, +z)-(1, +z)-(9, -z)-(10, -z)-(11, -z)-(12, -z)-(3, -z)

Hormiga: 8

Reorientaciones: 2

Salida:

(8, -x)-(14, -x)-(15, -x)-(16, -x)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(2, +z)-(13, +z)-(1, +z)-(3, -z)-(9, -z)-(10, -z)-(11, -z)

Hormiga: 9

Reorientaciones: 3

Salida:

(8, -x)-(14, -x)-(15, -x)-(16, -x)-(10, -z)-(11, -z)-(9, +z)-(5, +z)-(4, +z)-(7, +z)-(17, +z)-(6, +z)-(2, +z)-(1, +z)-(12, -x)

Hormiga: 10

Reorientaciones: 2

Salida:

(8, -x)-(14, -x)-(15, -x)-(16, -x)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(2, +z)-(13, +z)-(1, +z)-(3, -z)-(9, -z)-(10, -z)-(11, -z)-(12, -z)

Hormiga: 11

Reorientaciones: 2

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(1, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)

Secuencia de salida premiada

Reorientaciones: 2

Salida:

(8, -x)-(15, -x)-(14, -x)-(16, -x)-(10, -z)-(11, -z)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(7, +z)-(6, +z)-(13, +z)-(3, +z)-(17, +z)-(2, +z)-(1, +z)